

Repository Audit

File: eslint.config.mjs

```
import { defineConfig, globalIgnores } from "eslint/config";
import nextVitals from "eslint-config-next/core-web-vitals";
import nextTs from "eslint-config-next/typescript";

const eslintConfig = defineConfig([
  ...nextVitals,
  ...nextTs,
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    ".next/**",
    "out/**",
    "build/**",
    "next-env.d.ts",
  ]),
]);

export default eslintConfig;
```

File: next.config.ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
  reactCompiler: true,
};

export default nextConfig;
```

File: package.json

```
{
  "name": "axiom",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint",
    "test": "node --import tsx --test src/lib/**/*.test.ts"
  },
  "dependencies": {
    "@supabase/auth-ui-react": "^0.4.7",
    "@supabase/auth-ui-shared": "^0.1.8",
    "@supabase/ssr": "^0.10.3",
    "@supabase/supabase-js": "^2.105.4",
    "framer-motion": "^12.40.0",
    "lenis": "^1.3.23",
    "lucide-react": "^1.17.0",
    "next": "16.2.6",
    "react": "19.2.4",
    "react-dom": "19.2.4"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
  }
}
```

Repository Audit

```
"babel-plugin-react-compiler": "1.0.0",
"eslint": "^9",
"eslint-config-next": "16.2.6",
"tailwindcss": "^4",
"tsx": "^4.21.0",
"typescript": "^5"
}
}
```

File: postcss.config.mjs

```
const config = {
  plugins: {
    '@tailwindcss/postcss': {},
  },
};

export default config;
```

File: src/app/auth/auth-code-error/page.tsx

```
"use client";

import Link from "next/link";

export default function AuthCodeError() {
  return (
    <main className="min-h-screen flex items-center justify-center bg-background text-foreground p-6">
      <div className="w-full max-w-md bg-background border-2 border-red-500/20 rounded-[2.5rem] p-10 shadow-2xl text-center">
        <div className="w-20 h-20 bg-red-500/10 rounded-3xl flex items-center justify-center mx-auto mb-6">
          <svg
            width="40"
            height="40"
            viewBox="0 0 24 24"
            fill="none"
            stroke="currentColor"
            strokeWidth="3"
            className="text-red-500"
          >
            <path d="M10.29 3.86L11.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"
            />
            <line x1="12" y1="9" x2="12" y2="13" />
            <line x1="12" y1="17" x2="12.01" y2="17" />
          </svg>
        </div>

        <h1 className="text-3xl font-black tracking-tighter uppercase mb-4">
          Auth Failed
        </h1>

        <p className="text-gray-500 text-sm font-bold uppercase tracking-widest mb-8 leading-relaxed">
          The secure handshake with the authentication provider was interrupted.
        </p>

        <Link
          href="/"
          className="w-full bg-foreground text-background px-4 py-5 rounded-2xl font-black text-lg
            hover:scale-[1.02] active:scale-[0.98] transition-all shadow-xl shadow-foreground/10 block uppercase
            text-center"
        >
          Return to Login
        </Link>
      </div>
    </main>
  );
}
```

Repository Audit

```
        </Link>
      </div>
    </main>
  );
}
```

File: src/app/auth/callback/route.ts

```
import { NextResponse } from "next/server";
// The client you created in Step 1
import { createClient } from "@lib/supabase/server";

export async function GET(request: Request) {
  const { searchParams, origin } = new URL(request.url);
  const code = searchParams.get("code");
  // if "next" is in search params, use it as the redirection URL
  const next = searchParams.get("next") ?? "/dashboard";

  if (code) {
    const supabase = await createClient();
    const { error } = await supabase.auth.exchangeCodeForSession(code);
    if (!error) {
      return NextResponse.redirect(`${origin}${next}`);
    }
  }

  // return the user to an error page with instructions
  return NextResponse.redirect(`${origin}/auth/auth-code-error`);
}
```

File: src/app/dashboard/AccountSection.tsx

```
"use client";

import { useState, useSyncExternalStore } from "react";
import { ACCOUNT_TYPES, type Account } from "@lib/finance/accounts";
import { createAccountAction, type DashboardSnapshot } from "./actions";
import { formatCurrency } from "@lib/utils/formatters";
import { AccountMap } from "@lib/utils/taxonomy";

interface AccountSectionProps {
  accounts: Account[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

const emptySubscribe = () => () => {};

export function AccountSection({ accounts, onSnapshot }: AccountSectionProps) {
  const isClient = useSyncExternalStore(
    emptySubscribe,
    () => true,
    () => false,
  );

  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [sourceForm, setSourceForm] = useState({
    account_name: "",
    account_type: "checking",
    current_balance: "",
  });
```

Repository Audit

```
    institution: "",
  });

async function handleSourceSubmit(e: React.FormEvent) {
  e.preventDefault();
  setIsLoading(true);
  setError(null);
  try {
    const snapshot = await createAction({
      account_name: sourceForm.account_name,
      account_type: sourceForm.account_type,
      current_balance: Number(sourceForm.current_balance),
      institution: sourceForm.institution || undefined,
    });
    setSourceForm({
      account_name: "",
      account_type: "checking",
      current_balance: "",
      institution: "",
    });
    setIsAdding(false);
    onSnapshot(snapshot);
  } catch (err: unknown) {
    setError(err instanceof Error ? err.message : "Failed to create source");
  } finally {
    setIsLoading(false);
  }
}

return (
  <div className="space-y-12">
    <div className="flex items-center justify-between">
      <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
        Capital Repositories
      </h2>
      <button
        onClick={() => setIsAdding(!isAdding)}
        className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/40 hover:text-white
transition-colors"
        >
        {isAdding ? "? CANCEL" : "+ APPEND REPOSITORY"}
      </button>
    </div>

    {isAdding && (
      <div className="bg-white/[0.02] border border-white/5 p-8 animate-in fade-in slide-in-from-top-4
duration-500">
        <form onSubmit={handleSourceSubmit} className="space-y-8">
          <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                Repository Name
              </label>
              <input
                type="text"
                required
                placeholder="e.g. M-Pesa, Savings"
                value={sourceForm.account_name}
                onChange={(e) =>
                  setSourceForm({
                    ...sourceForm,
                    account_name: e.target.value,
                  })
                }
              />
            </div>
          </div>
        </form>
      </div>
    )}
  </div>
);
```

Repository Audit

```
        className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
      />
    </div>
    <div className="space-y-2">
      <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
        Current Liquidity
      </label>
      <input
        type="number"
        required
        placeholder="0.00"
        value={sourceForm.current_balance}
        onChange={(e) =>
          setSourceForm({
            ...sourceForm,
            current_balance: e.target.value,
          })
        }
        className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
      />
    </div>
  </div>

  <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
    <div className="space-y-2">
      <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
        Repository Type
      </label>
      <select
        value={sourceForm.account_type}
        onChange={(e) =>
          setSourceForm({
            ...sourceForm,
            account_type: e.target.value,
          })
        }
      >
        <div className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
        >
          {ACCOUNT_TYPES.map((t) => (
            <option key={t.value} value={t.value} className="bg-black">
              {t.label}
            </option>
          ))}
        </select>
      </div>
      <div className="space-y-2">
        <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
          Financial Institution
        </label>
        <input
          type="text"
          placeholder="Optional"
          value={sourceForm.institution}
          onChange={(e) =>
            setSourceForm({
              ...sourceForm,
              institution: e.target.value,
            })
          }
          className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
```

Repository Audit

```
    />
  </div>
</div>

{error && (
  <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
    {error}
  </p>
)}

<button
  type="submit"
  disabled={isLoading}
  className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px]
  hover:bg-white/90 transition-colors disabled:opacity-50"
  >
  {isLoading ? "INITIALIZING..." : "CONFIRM REPOSITORY"}
</button>
</form>
</div>
)}

<div className="space-y-2">
  {accounts.length === 0 ? (
    <div className="py-12 text-center opacity-20">
      <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
        No repositories active
      </p>
    </div>
  ) : (
    accounts.map((account) => (
      <div
        key={account.id}
        className="flex items-center justify-between py-6 border-b border-white/5 group
        hover:bg-white/[0.01] transition-all px-2"
        >
        <div className="space-y-1">
          <span className="text-[8px] font-mono tracking-widest uppercase text-white/20">
            {AccountMap[account.account_type] || account.account_type}
          </span>
          <h3 className="font-cormorant text-xl text-white transition-transform
          group-hover:translate-x-2">
            {account.account_name}
          </h3>
        </div>
        <div className="text-right">
          <p className="font-cormorant text-xl text-white">
            {formatCurrency(account.current_balance)}
          </p>
          {account.institution && (
            <p className="text-[8px] font-mono text-white/10 uppercase tracking-widest mt-1">
              {account.institution}
            </p>
          )}
        </div>
      </div>
    ))
  )}
</div>
</div>
);
}
```

Repository Audit

File: src/app/dashboard/BaselineSection.tsx

```
"use client";

import { useState } from "react";
import { TAXONOMY_CATEGORIES } from "@lib/finance/taxonomy";
import {
  createOperationalBaselineAction,
  type DashboardSnapshot,
} from "../actions";
import { OperationalBaseline, BaselineCadence } from "@lib/analytics/types";
import { formatCurrency } from "@lib/utils/formatters";

interface BaselineSectionProps {
  baseline: OperationalBaseline[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

const CADENCES = [
  { value: "daily", label: "Daily" },
  { value: "weekly", label: "Weekly" },
  { value: "biweekly", label: "Bi-weekly" },
  { value: "monthly", label: "Monthly" },
  { value: "quarterly", label: "Quarterly" },
  { value: "yearly", label: "Yearly" },
];

export function BaselineSection({
  baseline,
  onSnapshot,
}: BaselineSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [form, setForm] = useState({
    title: "",
    amount: "",
    category: "Maintenance",
    cadence: "monthly" as BaselineCadence,
    baseline_type: "expense" as "expense" | "allocation",
    notes: "",
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      const snapshot = await createOperationalBaselineAction({
        title: form.title,
        amount: Number(form.amount),
        category: form.category,
        cadence: form.cadence,
        baseline_type: form.baseline_type,
        is_active: true,
        notes: form.notes || undefined,
      });
      setForm({
        title: "",
        amount: "",
        category: "Maintenance",
        cadence: "monthly",
      });
    } catch {
      // Error handling
    }
  }
}
```

Repository Audit

```
    baseline_type: "expense",
    notes: "",
  });
  setIsAdding(false);
  onSnapshot(snapshot);
} catch (err: unknown) {
  setError(
    err instanceof Error ? err.message : "Failed to create baseline entry",
  );
} finally {
  setIsLoading(false);
}
}

return (
  <div className="space-y-12">
    <div className="flex items-center justify-between">
      <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
        Operational Maintenance
      </h2>
      <button
        onClick={() => setIsAdding(!isAdding)}
        className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/40 hover:text-white
transition-colors"
        >
        {isAdding ? "? CANCEL" : "+ APPEND RECURRING"}
      </button>
    </div>

    {isAdding && (
      <div className="bg-white/[0.02] border border-white/5 p-8 animate-in fade-in slide-in-from-top-4
duration-500">
        <form onSubmit={handleSubmit} className="space-y-8">
          <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                Baseline Name
              </label>
              <input
                type="text"
                required
                placeholder="e.g. Rent, Netflix"
                value={form.title}
                onChange={(e) => setForm({ ...form, title: e.target.value })}
                className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
              />
            </div>
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                Amount
              </label>
              <input
                type="number"
                required
                placeholder="0.00"
                value={form.amount}
                onChange={(e) => setForm({ ...form, amount: e.target.value })}
                className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
              />
            </div>
          </div>
        </form>
      </div>
    )}
  </div>
);
```

Repository Audit

```
<div className="grid grid-cols-1 md:grid-cols-2 gap-8">
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Category
    </label>
    <select
      value={form.category}
      onChange={(e) =>
        setForm({ ...form, category: e.target.value })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
    >
      {TAXONOMY_CATEGORIES.map((cat) => (
        <option
          key={cat.value}
          value={cat.value}
          className="bg-black"
        >
          {cat.label}
        </option>
      ))}
    </select>
  </div>
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Frequency
    </label>
    <select
      value={form.cadence}
      onChange={(e) =>
        setForm({
          ...form,
          cadence: e.target.value as BaselineCadence,
        })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
    >
      {CADENCES.map((c) => (
        <option key={c.value} value={c.value} className="bg-black">
          {c.label}
        </option>
      ))}
    </select>
  </div>
</div>

{error && (
  <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
    {error}
  </p>
)}

<button
  type="submit"
  disabled={isLoading}
  className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px]
hover:bg-white/90 transition-colors disabled:opacity-50"
>
  {isLoading ? "SAVING..." : "CONFIRM RECURRING"}
</button>
</form>
</div>
```

Repository Audit

```
    })

    <div className="space-y-2">
      {baseline.length === 0 ? (
        <div className="py-12 text-center opacity-20">
          <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
            No recurring flows active
          </p>
        </div>
      ) : (
        baseline.map((item) => (
          <div
            key={item.id}
            className="flex items-center justify-between py-6 border-b border-white/5 group
            hover:bg-white/[0.01] transition-all px-2"
            >
            <div className="space-y-1">
              <span className="text-[8px] font-mono tracking-widest uppercase text-white/20">
                {item.cadence}
              </span>
              <h3 className="font-cormorant text-xl text-white transition-transform
              group-hover:translate-x-2">
                {item.title}
              </h3>
            </div>
            <div className="text-right">
              <p className="font-cormorant text-xl text-white">
                {formatCurrency(item.amount)}
              </p>
            </div>
          </div>
        ))
      )}
    </div>
  </div>
);
}
```

File: src/app/dashboard/GoalSection.tsx

```
"use client";

import { useState } from "react";
import {
  calculateGoalProgressPercentage,
  type FinancialGoal,
} from "@lib/finance/goals";
import { createGoalAction, type DashboardSnapshot } from "../actions";
import { formatCurrency } from "@lib/utils/formatters";

interface GoalSectionProps {
  goals: FinancialGoal[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function GoalSection({ goals, onSnapshot }: GoalSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [form, setForm] = useState({
    goal_name: "",
    goal_type: "emergency_fund",
```

Repository Audit

```
target_amount: "",
current_progress: "",
priority: "medium",
status: "active",
notes: "",
});

async function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  setIsLoading(true);
  setError(null);
  try {
    const snapshot = await createGoalAction({
      goal_name: form.goal_name,
      goal_type: form.goal_type,
      target_amount: Number(form.target_amount),
      current_progress: form.current_progress
        ? Number(form.current_progress)
        : 0,
      priority: form.priority,
      status: form.status,
      notes: form.notes || undefined,
    });
    setForm({
      goal_name: "",
      goal_type: "emergency_fund",
      target_amount: "",
      current_progress: "",
      priority: "medium",
      status: "active",
      notes: "",
    });
    setIsAdding(false);
    onSnapshot(snapshot);
  } catch (err: unknown) {
    setError(
      err instanceof Error ? err.message : "Failed to record financial goal",
    );
  } finally {
    setIsLoading(false);
  }
}

return (
  <div className="space-y-12">
    <div className="flex items-center justify-between">
      <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
        Wealth Milestones
      </h2>
      <button
        onClick={() => setIsAdding(!isAdding)}
        className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/40 hover:text-white
transition-colors"
      >
        {isAdding ? "? CANCEL" : "+ APPEND MILESTONE"}
      </button>
    </div>

    {isAdding && (
      <div className="bg-white/[0.02] border border-white/5 p-8 animate-in fade-in slide-in-from-top-4
duration-500">
        <form onSubmit={handleSubmit} className="space-y-8">
          <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
            <div className="space-y-2">
```

Repository Audit

```
<label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
  Milestone Name
</label>
<input
  type="text"
  required
  placeholder="e.g. Emergency Fund"
  value={form.goal_name}
  onChange={(e) =>
    setForm({ ...form, goal_name: e.target.value })
  }
  className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
/>
</div>
<div className="space-y-2">
  <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
    Target Amount
  </label>
  <input
    type="number"
    required
    placeholder="0.00"
    value={form.target_amount}
    onChange={(e) =>
      setForm({ ...form, target_amount: e.target.value })
    }
    className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
  />
</div>
</div>

{error && (
  <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
    {error}
  </p>
)}

<button
  type="submit"
  disabled={isLoading}
  className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px]
hover:bg-white/90 transition-colors disabled:opacity-50"
>
  {isLoading ? "SAVING..." : "CONFIRM MILESTONE"}
</button>
</form>
</div>
)}

<div className="space-y-8">
  {goals.length === 0 ? (
    <div className="py-12 text-center opacity-20">
      <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
        No milestones active
      </p>
    </div>
  ) : (
    goals.map((goal) => {
      const progress = calculateGoalProgressPercentage(goal);
      return (
        <div
          key={goal.id}

```

Repository Audit

```
        className="space-y-4 group cursor-default pb-8 border-b border-white/5 last:border-0"
    >
    <div className="flex items-center justify-between mb-2">
      <div className="space-y-1">
        <span
          className={`text-[8px] font-mono tracking-widest uppercase ${
            goal.priority === "critical"
              ? "text-red-500"
              : "text-white/20"
          }`}
        >
          {goal.priority}
        </span>
        <h3 className="font-cormorant text-2xl text-white transition-transform
group-hover:translate-x-2">
          {goal.goal_name}
        </h3>
      </div>
      <div className="text-right">
        <p className="font-cormorant text-3xl text-white tabular-nums">
          {Math.round(progress)}%
        </p>
      </div>
    </div>

    <div className="h-[1px] w-full bg-white/5 relative overflow-hidden">
      <div
        className="absolute inset-y-0 left-0 bg-white/60 transition-all duration-1000 ease-out"
        style={{ width: `${Math.min(100, progress)}%` }}
      />
    </div>

    <div className="flex justify-between items-end pt-2 opacity-30 group-hover:opacity-60
transition-opacity">
      <p className="text-[9px] font-mono uppercase tracking-widest">
        {formatCurrency(goal.current_progress)}
      </p>
      <p className="text-[9px] font-mono uppercase tracking-widest">
        Target: {formatCurrency(goal.target_amount)}
      </p>
    </div>
  </div>
);
})
})
</div>
</div>
);
}
```

File: src/app/dashboard/HistoricalAudit.tsx

```
"use client";

import React, { useEffect, useState } from "react";
import { fetchHistoricalAuditAction } from "../actions";
import { AuditRecord } from "@lib/db/audit";
import { formatCurrency } from "@lib/utils/formatters";

export function HistoricalAudit() {
  const [auditRecords, setAuditRecords] = useState<AuditRecord[]>([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
```

Repository Audit

```
useEffect(() => {
  async function loadAudit() {
    try {
      const records = await fetchHistoricalAuditAction();
      setAuditRecords(records);
    } catch (err) {
      console.error("Failed to fetch audit history:", err);
      setError("Forensic layer synchronization failure.");
    } finally {
      setIsLoading(false);
    }
  }
  loadAudit();
}, []);

if (isLoading) {
  return (
    <div className="p-6 text-center animate-pulse">
      <p className="text-[10px] font-black uppercase tracking-[0.3em] text-foreground/20">
        Synchronizing Forensic Layer...
      </p>
    </div>
  );
}

if (error) {
  return (
    <div className="p-6 text-center">
      <p className="text-[10px] font-black uppercase tracking-[0.3em] text-red-500/40">
        {error}
      </p>
    </div>
  );
}

if (auditRecords.length === 0) {
  return (
    <div className="p-8 text-center border-2 border-dashed border-foreground/5 rounded-3xl">
      <p className="text-[10px] font-black uppercase tracking-[0.3em] text-foreground/20">
        No audit history found
      </p>
    </div>
  );
}

return (
  <div className="space-y-6">
    <div className="flex items-center gap-3 opacity-20">
      <h3 className="text-[11px] font-black uppercase tracking-[0.2em] text-foreground">
        Historical Audit Trail
      </h3>
      <div className="h-0.5 flex-1 bg-foreground/10"></div>
    </div>

    <div className="grid grid-cols-1 gap-3">
      {auditRecords.map((record) => (
        <div
          key={`${record.type}-${record.id}`}
          className="group relative bg-foreground/[0.02] border border-foreground/5 rounded-2xl p-4 flex
items-center justify-between transition-colors hover:bg-foreground/[0.04]"
        >
          <div className="flex items-center gap-4">
            <div className="w-10 h-10 bg-foreground/5 rounded-xl flex items-center justify-center grayscale
```

Repository Audit

```
opacity-40">
  <svg
    width="18"
    height="18"
    viewBox="0 0 24 24"
    fill="none"
    stroke="currentColor"
    strokeWidth="2.5"
    className="text-foreground"
  >
    {record.type === "deployment" && (
      <path d="M12 2v20M17 5H9.5a3.5 3.5 0 0 0 0 7h5a3.5 3.5 0 0 1 0 7H6" />
    )}
    {record.type === "income" && (
      <path d="M12 20V10M18 20V4M6 20v-4" />
    )}
    {record.type === "liability" && (
      <path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8 10z" />
    )}
  </svg>
</div>
<div>
  <div className="flex items-center gap-2 mb-0.5">
    <h4 className="font-bold text-sm text-foreground/40 line-through decoration-foreground/20">
      {record.name}
    </h4>
    <span className="text-[8px] font-black px-2 py-0.5 bg-foreground/5 rounded-full uppercase tracking-tighter text-foreground/30">
      {record.type}
    </span>
  </div>
  <div className="text-[9px] font-bold text-foreground/30 uppercase tracking-tighter">
    Deleted:{" "}
    {new Date(record.deleted_at).toLocaleDateString(undefined, {
      month: "short",
      day: "numeric",
      year: "numeric",
    })}
  </div>
</div>
<div>
  <div className="text-right">
    <p className="font-black text-sm tabular-nums text-foreground/30 line-through decoration-foreground/10">
      {formatCurrency(record.amount)}
      {record.cadence && (
        <span className="text-[9px] ml-1 uppercase">
          / {record.cadence}
        </span>
      )}
    </p>
    <p className="text-[8px] font-black text-foreground/20 uppercase tracking-widest mt-0.5">
      Archived
    </p>
  </div>
</div>
  )}}
</div>

  <p className="text-center text-[9px] font-black text-foreground/10 uppercase tracking-[0.2em] mt-4">
    Forensic view only :: Data immutable
  </p>
</div>
);
```

Repository Audit

```
}
```

File: src/app/dashboard/IncomeSection.tsx

```
"use client";

import { useState } from "react";
import {
  INCOME_TYPES,
  CADENCES,
  type IncomeStream,
  type IncomeType,
  type Cadence,
} from "@lib/finance/income";
import { createIncomeAction, type DashboardSnapshot } from "../actions";
import { formatCurrency } from "@lib/utils/formatters";
import { IncomeMap } from "@lib/utils/taxonomy";

interface IncomeSectionProps {
  incomeStreams: IncomeStream[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

interface FormEntry {
  income_name: string;
  income_type: IncomeType;
  amount: string;
  cadence: Cadence;
  execution_day: number;
  is_recurring: boolean;
  source: string;
}

export function IncomeSection({
  incomeStreams,
  onSnapshot,
}: IncomeSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [entries, setEntries] = useState<FormEntry[]>([
    {
      income_name: "",
      income_type: "salary",
      amount: "",
      cadence: "monthly",
      execution_day: 1,
      is_recurring: true,
      source: "",
    },
  ]);

  const addEntry = () => {
    setEntries([
      ...entries,
      {
        income_name: "",
        income_type: "salary",
        amount: "",
        cadence: "monthly",
        execution_day: 1,
        is_recurring: true,
      },
    ]);
  };
}
```

Repository Audit

```
        source: "",
      },
    ]);
  };

const updateEntry = <K extends keyof FormEntry>({
  index: number,
  field: K,
  value: FormEntry[K],
}) => {
  const newEntries = [...entries];
  newEntries[index][field] = value;
  setEntries(newEntries);
};

async function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  setIsLoading(true);
  setError(null);
  try {
    const snapshot = await createIncomeAction(
      entries.map((entry) => ({
        ...entry,
        amount: Number(entry.amount),
        execution_day:
          entry.cadence === "monthly" ||
          entry.cadence === "weekly" ||
          entry.cadence === "biweekly"
            ? Number(entry.execution_day)
            : null,
        source: entry.source || undefined,
      })),
    );
    setEntries([
      {
        income_name: "",
        income_type: "salary",
        amount: "",
        cadence: "monthly",
        execution_day: 1,
        is_recurring: true,
        source: "",
      },
    ]);
    setIsAdding(false);
    onSnapshot(snapshot);
  } catch (err: unknown) {
    setError(
      err instanceof Error ? err.message : "Failed to record income streams",
    );
  } finally {
    setIsLoading(false);
  }
}

return (
  <div className="space-y-12">
    <div className="flex items-center justify-between">
      <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
        Inflow Velocity
      </h2>
      <button
        onClick={() => setIsAdding(!isAdding)}
        className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/40 hover:text-white

```

Repository Audit

```
transition-colors"
  >
    {isAdding ? "? CANCEL" : "+ APPEND INFLOW"}
  </button>
</div>

{isAdding && (
  <div className="bg-white/[0.02] border border-white/5 p-8 animate-in fade-in slide-in-from-top-4
duration-500">
    <form onSubmit={handleSubmit} className="space-y-12">
      {entries.map((entry, index) => (
        <div key={index} className="space-y-8 relative">
          {index > 0 && <div className="border-t border-white/5 pt-8" />}

          <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                Source Name
              </label>
              <input
                type="text"
                required
                placeholder="e.g. Salary"
                value={entry.income_name}
                onChange={(e) =>
                  updateEntry(index, "income_name", e.target.value)
                }
                className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
              />
            </div>
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                Amount
              </label>
              <input
                type="number"
                required
                placeholder="0.00"
                value={entry.amount}
                onChange={(e) =>
                  updateEntry(index, "amount", e.target.value)
                }
                className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
              />
            </div>
          </div>

          <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                Type
              </label>
              <select
                value={entry.income_type}
                onChange={(e) =>
                  updateEntry(
                    index,
                    "income_type",
                    e.target.value as IncomeType,
                  )
                }
                className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
```

Repository Audit

```
text-[10px] tracking-widest uppercase focus:outline-none"
  >
    {INCOME_TYPES.map((t) => (
      <option
        key={t.value}
        value={t.value}
        className="bg-black"
      >
        {t.label}
      </option>
    ))}
  </select>
</div>
<div className="space-y-2">
  <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
    Frequency
  </label>
  <select
    value={entry.cadence}
    onChange={(e) =>
      updateEntry(index, "cadence", e.target.value as Cadence)
    }
    className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
  >
    {CADENCES.map((c) => (
      <option
        key={c.value}
        value={c.value}
        className="bg-black"
      >
        {c.label}
      </option>
    ))}
  </select>
</div>
<div className="flex items-center pt-6">
  <label className="flex items-center gap-3 cursor-pointer">
    <input
      type="checkbox"
      checked={entry.is_recurring}
      onChange={(e) =>
        updateEntry(index, "is_recurring", e.target.checked)
      }
      className="w-4 h-4 border border-white/20 accent-white"
    />
    <span className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Recurring
    </span>
  </label>
</div>
</div>
</div>
)))

<div className="space-y-6">
  <button
    type="button"
    onClick={addEntry}
    className="text-[9px] font-mono tracking-widest uppercase text-white/20 hover:text-white
transition-colors"
  >
    + APPEND ANOTHER SOURCE
  </button>
```

Repository Audit

```
{error && (
  <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
    {error}
  </p>
)}

<button
  type="submit"
  disabled={isLoading}
  className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px]
  hover:bg-white/90 transition-colors disabled:opacity-50"
  >
  {isLoading ? "SAVING..." : "CONFIRM INFLOWS"}
</button>
</div>
</form>
</div>
)}

<div className="space-y-2">
  {incomeStreams.length === 0 ? (
    <div className="py-12 text-center opacity-20">
      <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
        No inflow streams active
      </p>
    </div>
  ) : (
    incomeStreams.map((stream) => (
      <div
        key={stream.id}
        className="flex items-center justify-between py-6 border-b border-white/5 group
        hover:bg-white/[0.01] transition-all px-2"
        >
        <div className="space-y-1">
          <span className="text-[8px] font-mono tracking-widest uppercase text-white/20">
            {IncomeMap[stream.income_type] || stream.income_type}
          </span>
          <h3 className="font-cormorant text-xl text-white transition-transform
          group-hover:translate-x-2">
            {stream.income_name}
          </h3>
        </div>
        <div className="text-right">
          <p className="font-cormorant text-xl text-white">
            {formatCurrency(stream.amount)}
            <span className="text-[10px] ml-2 text-white/20 font-sans tracking-widest uppercase">
              / {stream.cadence}
            </span>
          </p>
        </div>
        </div>
      </div>
    ))
  )}
</div>
</div>
);
}
```

File: src/app/dashboard/LiabilitySection.tsx

"use client";

Repository Audit

```
import { useState } from "react";
import { LIABILITY_TYPES, type Liability } from "@lib/finance/liabilities";
import {
  createLiabilityAction,
  deleteLiabilityAction,
  type DashboardSnapshot,
} from "../actions";
import { formatCurrency } from "@lib/utils/formatters";
import { LiabilityMap } from "@lib/utils/taxonomy";

interface LiabilitySectionProps {
  liabilities: Liability[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function LiabilitySection({
  liabilities,
  onSnapshot,
}: LiabilitySectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [form, setForm] = useState({
    liability_name: "",
    liability_type: "credit_card",
    outstanding_balance: "",
    interest_rate: "",
    minimum_payment: "",
    institution: "",
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      const snapshot = await createLiabilityAction({
        liability_name: form.liability_name,
        liability_type: form.liability_type,
        outstanding_balance: Number(form.outstanding_balance),
        interest_rate: form.interest_rate ? Number(form.interest_rate) : 0,
        minimum_payment: form.minimum_payment
          ? Number(form.minimum_payment)
          : 0,
        institution: form.institution || undefined,
      });
      setForm({
        liability_name: "",
        liability_type: "credit_card",
        outstanding_balance: "",
        interest_rate: "",
        minimum_payment: "",
        institution: "",
      });
      setIsAdding(false);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      setError(
        err instanceof Error ? err.message : "Failed to record obligation",
      );
    } finally {
      setIsLoading(false);
    }
  }
}
```

Repository Audit

```
}

return (
  <div className="space-y-12">
    <div className="flex items-center justify-between">
      <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
        Financial Commitments
      </h2>
      <button
        onClick={() => setIsAdding(!isAdding)}
        className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/40 hover:text-white
transition-colors"
      >
        {isAdding ? "? CANCEL" : "+ APPEND COMMITMENT"}
      </button>
    </div>

    {isAdding && (
      <div className="bg-white/[0.02] border border-white/5 p-8 animate-in fade-in slide-in-from-top-4
duration-500">
        <form onSubmit={handleSubmit} className="space-y-8">
          <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                Commitment Name
              </label>
              <input
                type="text"
                required
                placeholder="e.g. Equity Bank Loan"
                value={form.liability_name}
                onChange={(e) =>
                  setForm({ ...form, liability_name: e.target.value })
                }
                className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
              />
            </div>
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                Outstanding Balance
              </label>
              <input
                type="number"
                required
                placeholder="0.00"
                value={form.outstanding_balance}
                onChange={(e) =>
                  setForm({ ...form, outstanding_balance: e.target.value })
                }
                className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
              />
            </div>
          </div>

          <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
            <div className="space-y-2">
              <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
                Type
              </label>
              <select
                value={form.liability_type}
                onChange={(e) =>
```

Repository Audit

```
        setForm({ ...form, liability_type: e.target.value })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
    >
      {LIABILITY_TYPES.map((t) => (
        <option key={t.value} value={t.value} className="bg-black">
          {t.label}
        </option>
      ))}
    </select>
  </div>
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Min. Payment
    </label>
    <input
      type="number"
      placeholder="0.00"
      value={form.minimum_payment}
      onChange={(e) =>
        setForm({ ...form, minimum_payment: e.target.value })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
    />
  </div>
  <div className="space-y-2">
    <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
      Interest (%)
    </label>
    <input
      type="number"
      step="0.01"
      placeholder="0.00"
      value={form.interest_rate}
      onChange={(e) =>
        setForm({ ...form, interest_rate: e.target.value })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
    />
  </div>
</div>

{error && (
  <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
    {error}
  </p>
)}

<button
  type="submit"
  disabled={isLoading}
  className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px]
hover:bg-white/90 transition-colors disabled:opacity-50"
  >
  {isLoading ? "INITIALIZING..." : "CONFIRM COMMITMENT"}
</button>
</form>
</div>
)}

<div className="space-y-2">
```

Repository Audit

```
{liabilities.length === 0 ? (  
  <div className="py-12 text-center opacity-20">  
    <p className="text-[10px] font-mono uppercase tracking-[0.5em]">  
      No active commitments  
    </p>  
  </div>  
) : (  
  liabilities.map((liability) => (  
    <div  
      key={liability.id}  
      className="flex items-center justify-between py-6 border-b border-white/5 group  
      hover:bg-white/[0.01] transition-all px-2"  
    >  
      <div className="space-y-1">  
        <span className="text-[8px] font-mono tracking-widest uppercase text-white/20">  
          {liabilityMap[liability.liability_type] ||  
            liability.liability_type}  
        </span>  
        <h3 className="font-cormorant text-xl text-white transition-transform  
group-hover:translate-x-2">  
          {liability.liability_name}  
        </h3>  
      </div>  
      <div className="text-right">  
        <p className="font-cormorant text-xl text-white">  
          {formatCurrency(liability.outstanding_balance)}  
        </p>  
        {liability.interest_rate > 0 && (  
          <p className="text-[8px] font-mono text-red-500/40 uppercase tracking-widest mt-1">  
            {liability.interest_rate}% APR  
          </p>  
        )}  
      </div>  
    </div>  
  ))  
)}  
</div>  
</div>  
);  
}
```

File: src/app/dashboard/NewEntryForm.tsx

```
"use client";  
  
import { useState } from "react";  
import { TAXONOMY_CATEGORIES } from "@lib/finance/taxonomy";  
import { DeploymentMap } from "@lib/utils/taxonomy";  
import { formatCurrency } from "@lib/utils/formatters";  
import { Account } from "@lib/analytics/types";  
  
interface NewEntryFormProps {  
  accounts: Account[];  
  liquidity: number;  
  isActionLoading: boolean;  
  onSubmit: (data: {  
    title: string;  
    amount: number;  
    category: string;  
    accountId?: string;  
  }) => Promise<void>;  
  onCancel?: () => void;  
}
```

Repository Audit

```
export function NewEntryForm({
  accounts,
  liquidity,
  isActionLoading,
  onSubmit,
  onCancel,
}: NewEntryFormProps) {
  const [title, setTitle] = useState("");
  const [amount, setAmount] = useState("");
  const [category, setCategory] = useState("Unclassified");
  const [accountId, setAccountId] = useState("");
  const [error, setError] = useState<string | null>(null);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);

    if (category === "Unclassified") {
      setError("Strategic intent required.");
      return;
    }

    try {
      await onSubmit({
        title,
        amount: Number(amount),
        category,
        accountId: accountId || undefined,
      });
      setTitle("");
      setAmount("");
      setCategory("Unclassified");
      setAccountId("");
    } catch (err: unknown) {
      setError(err instanceof Error ? err.message : "Execution failed");
    }
  };

  return (
    <form onSubmit={handleSubmit} className="space-y-12">
      <div className="space-y-12">
        <div className="space-y-4">
          <label className="text-[10px] font-mono text-white/20 uppercase tracking-[0.4em] ml-1">
            Intent
          </label>
          <input
            type="text"
            disabled={isActionLoading}
            placeholder="Objective of this deployment?"
            value={title}
            onChange={(e) => setTitle(e.target.value)}
            className="w-full bg-transparent border-b border-white/10 py-6 font-cormorant text-4xl text-white
placeholder:text-white/5 focus:outline-none focus:border-white transition-all"
            required
          />
        </div>

        <div className="grid grid-cols-1 md:grid-cols-2 gap-16">
          <div className="space-y-8">
            <div className="space-y-4">
              <label className="text-[10px] font-mono text-white/20 uppercase tracking-[0.4em] ml-1">
                Amount (KES)
              </label>

```

Repository Audit

```
<input
  type="number"
  disabled={isActionLoading}
  placeholder="0.00"
  value={amount}
  onChange={(e) => setAmount(e.target.value)}
  className="w-full bg-transparent border-b border-white/10 py-4 font-cormorant text-3xl
text-white placeholder:text-white/5 focus:outline-none focus:border-white transition-all tabular-nums"
  required
/>
</div>
<div className="space-y-4">
  <label className="text-[10px] font-mono text-white/20 uppercase tracking-[0.4em] ml-1">
    Funding Source
  </label>
  <select
    disabled={isActionLoading}
    value={accountId}
    onChange={(e) => setAccountId(e.target.value)}
    className="w-full bg-transparent border-b border-white/10 py-3 font-mono text-[10px]
tracking-widest uppercase text-white/60 focus:outline-none"
  >
    <option value="" className="bg-black">
      External / Manual
    </option>
    {accounts.map((acc) => (
      <option key={acc.id} value={acc.id} className="bg-black">
        {acc.account_name} ({formatCurrency(acc.current_balance)})
      </option>
    ))}
  </select>
</div>
</div>

<div className="space-y-6">
  <label className="text-[10px] font-mono text-white/20 uppercase tracking-[0.4em] ml-1">
    Strategic Classification
  </label>
  <div className="grid grid-cols-2 gap-4">
    {TAXONOMY_CATEGORIES.map((cat) => (
      <button
        key={cat.value}
        type="button"
        onClick={() => setCategory(cat.value)}
        className={`p-4 border transition-all text-left space-y-2 ${
          category === cat.value
            ? "border-white bg-white text-black"
            : "border-white/5 bg-white/[0.02] text-white/40 hover:border-white/20 hover:text-white"
        }`}
      >
        <span className="block text-[8px] font-mono tracking-widest uppercase">
          {DeploymentMap[cat.value] || cat.label}
        </span>
        <span
          className={`block text-[7px] font-light leading-tight ${category === cat.value ?
"text-black/60" : "text-white/20"}}`
        >
          {cat.definition}
        </span>
      </button>
    ))}
  </div>
</div>
</div>
```

Repository Audit

```
</div>

{error && (
  <div className="p-4 border border-white/10 font-mono text-[9px] text-white/60 uppercase
tracking-widest">
    ERROR: {error}
  </div>
)}

<div className="pt-8">
  <button
    type="submit"
    disabled={isActionLoading}
    className="w-full bg-white text-black py-6 font-medium tracking-[0.3em] uppercase text-xs
hover:bg-white/90 transition-all hover:scale-[1.02] active:scale-[0.98] shadow-2xl shadow-white/5
disabled:opacity-50"
  >
    {isActionLoading ? "EXECUTING..." : "AUTHORIZE DEPLOYMENT"}
  </button>
</div>
</form>
);
}
```

File: src/app/dashboard/PendingInflows.tsx

```
"use client";

import { useState } from "react";
import type { IncomeStream, Account } from "@lib/analytics/types";
import { resolvePendingInflowAction, type DashboardSnapshot } from "../actions";
import { formatCurrency } from "@lib/utils/formatters";
import { IncomeMap } from "@lib/utils/taxonomy";

interface PendingInflowsProps {
  incomeStreams: IncomeStream[];
  accounts: Account[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function PendingInflows({
  incomeStreams,
  accounts,
  onSnapshot,
}: PendingInflowsProps) {
  const [loadingId, setLoadingId] = useState<string | null>(null);
  const [selectedAccounts, setSelectedAccounts] = useState<
    Record<string, string>
  >({});

  const today = new Date();
  const currentDayOfWeek = today.getDay();
  const currentDayOfMonth = today.getDate();
  const mappedDayOfWeek = currentDayOfWeek === 0 ? 7 : currentDayOfWeek;

  const pendingInflows = incomeStreams.filter((stream) => {
    if (!stream.is_recurring || !stream.execution_day) return false;

    if (stream.last_executed_at) {
      const lastExecuted = new Date(stream.last_executed_at);
      if (
        lastExecuted.getDate() === today.getDate() &&
        lastExecuted.getMonth() === today.getMonth() &&

```

Repository Audit

```
    lastExecuted.getFullYear() === today.getFullYear()
  ) {
    return false;
  }
}

if (stream.cadence === "weekly") {
  return mappedDayOfWeek === stream.execution_day;
}
if (stream.cadence === "monthly" || stream.cadence === "biweekly") {
  return currentDayOfMonth === stream.execution_day;
}

return false;
});

if (pendingInflows.length === 0) return null;

async function handleVerify(stream: IncomeStream) {
  const accountId = selectedAccounts[stream.id];
  if (!accountId) {
    alert("Please select a destination account.");
    return;
  }

  setLoadingId(stream.id);
  try {
    const snapshot = await resolvePendingInflowAction({
      incomeId: stream.id,
      accountId,
      amount: stream.amount,
    });
    onSnapshot(snapshot);
  } catch (err: unknown) {
    alert(err instanceof Error ? err.message : "Verification failed");
  } finally {
    setLoadingId(null);
  }
}

return (
  <div className="space-y-6 animate-in fade-in duration-700">
    <div className="flex items-center gap-4">
      <div className="w-2 h-2 rounded-full bg-white animate-pulse"></div>
      <h2 className="text-[10px] font-mono text-white/60 uppercase tracking-[0.4em]">
        Verification Required
      </h2>
    </div>

    {pendingInflows.map((stream) => (
      <div
        key={stream.id}
        className="bg-white p-8 md:p-12 flex flex-col md:flex-row md:items-center justify-between gap-8
text-black"
      >
        <div className="space-y-2">
          <p className="text-[10px] font-mono tracking-widest uppercase opacity-40">
            Expected Inflow Detected
          </p>
          <h3 className="font-cormorant text-4xl">
            {formatCurrency(stream.amount)}
          </h3>
          <p className="text-xs font-light tracking-wide opacity-60">
            {stream.income_name} (
```

Repository Audit

```
        {IncomeMap[stream.income_type] || stream.income_type}))
    </p>
</div>

<div className="flex flex-col sm:flex-row items-stretch sm:items-center gap-4">
  <select
    disabled={loadingId === stream.id}
    value={selectedAccounts[stream.id] || ""}
    onChange={(e) =>
      setSelectedAccounts({
        ...selectedAccounts,
        [stream.id]: e.target.value,
      })
    }
    className="bg-transparent border-b border-black/20 py-2 font-mono text-[10px] tracking-widest uppercase focus:outline-none focus:border-black transition-colors min-w-[200px]"
  >
    <option value="">Destination Account</option>
    {accounts.map((acc) => (
      <option key={acc.id} value={acc.id}>
        {acc.account_name}
      </option>
    ))}
  </select>

  <button
    disabled={loadingId === stream.id}
    onClick={() => handleVerify(stream)}
    className="bg-black text-white px-10 py-4 font-medium tracking-widest uppercase text-[10px] hover:bg-black/90 transition-all disabled:opacity-50"
  >
    {loadingId === stream.id ? "VERIFYING..." : "CONFIRM RECEIPT"}
  </button>
</div>
</div>
))}
</div>
);
}
```

File: src/app/dashboard/RunwayCard.tsx

```
"use client";

import React from "react";

interface RunwayCardProps {
  runwayDays: number | null;
  netWorth?: number;
}

export function RunwayCard({ runwayDays, netWorth = 0 }: RunwayCardProps) {
  // 1. Logic for determining visual state
  const isInfiniteRunway = runwayDays === null;
  const isCritical = !isInfiniteRunway && runwayDays! < 30;
  const isInsolvent = netWorth < 0;

  // 2. Formatting
  const displayValue = isInfiniteRunway
    ? isInsolvent
      ? "INSOLVENT STABLE"
      : "SYSTEM STABLE"
    : `${(runwayDays! / 30).toFixed(1)} MONTHS`;
```

Repository Audit

```
// 3. Conditional Styles
const containerClasses = `bg-foreground/5 border rounded-2xl p-6 md:p-8 flex flex-col justify-between
transition-all duration-500 ${
  isInfiniteRunway
    ? isInsolvent
      ? "border-rose-500/20"
      : "border-emerald-500/10"
    : isCritical
      ? "border-rose-500/30 bg-rose-500/5 shadow-inner"
      : "border-foreground/10"
};

const labelClasses = `text-[11px] font-black uppercase tracking-[0.2em] mb-4 ${
  isInfiniteRunway
    ? isInsolvent
      ? "text-rose-500/50"
      : "text-emerald-500/50"
    : isCritical
      ? "text-rose-500/60"
      : "text-foreground/40"
};

const valueClasses = `text-4xl font-black tabular-nums transition-colors duration-500 ${
  isInfiniteRunway
    ? isInsolvent
      ? "text-rose-500/80 font-mono tracking-tighter"
      : "text-emerald-500/80 font-mono tracking-tighter"
    : isCritical
      ? "text-rose-600"
      : "text-foreground"
};

const description = isInfiniteRunway
  ? isInsolvent
    ? "Burn is absorbed by inflows, but total commitments exceed assets."
    : "Net inbound capital completely absorbs current structural burn."
  : isCritical
    ? "A recalibration of The Foundation is recommended."
    : "Duration of sustained operation based on current structural burn and liquidity.";

return (
  <div className={containerClasses}>
    <div className="flex items-center justify-between mb-4">
      <span className={labelClasses}>
        {isInfiniteRunway
          ? isInsolvent
            ? "Insolvent Stability"
            : "Infinite Stability"
          : "Operating Horizon"}
      </span>
      {isInfiniteRunway && !isInsolvent && (
        <div className="flex items-center gap-1.5">
          <span className="w-1.5 h-1.5 bg-emerald-500/40 rounded-full animate-pulse"></span>
          <span className="text-[8px] font-black text-emerald-500/40 uppercase tracking-widest">
            Active Protection
          </span>
        </div>
      )}
    </div>
    {isInfiniteRunway && isInsolvent && (
      <div className="flex items-center gap-1.5">
        <span className="w-1.5 h-1.5 bg-rose-500/40 rounded-full animate-pulse"></span>
        <span className="text-[8px] font-black text-rose-500/40 uppercase tracking-widest">
          Solvency Risk
        </span>
      </div>
    )}
  </div>
);
```

Repository Audit

```
        </span>
      </div>
    )}
  </div>

  <div className="flex flex-col">
    <span className={valueClasses}>{displayValue}</span>
    <p
      className={`text-[10px] font-bold uppercase tracking-widest mt-4 ${
        isInfiniteRunway ? "text-emerald-600/40" : "text-foreground/40"
      }`}
    >
      {description}
    </p>
  </div>
</div>
);
}
```

File: src/app/dashboard/StrategicObjectiveSection.tsx

```
"use client";

import { useState } from "react";
import {
  calculateObjectiveFundingRatio,
  type StrategicObjective,
} from "@lib/finance/objectives";
import {
  createStrategicObjectiveAction,
  updateStrategicObjectiveAction,
  type DashboardSnapshot,
} from "../actions";
import { formatCurrency } from "@lib/utils/formatters";

interface StrategicObjectiveSectionProps {
  objectives: StrategicObjective[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function StrategicObjectiveSection({
  objectives,
  onSnapshot,
}: StrategicObjectiveSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [editingId, setEditingId] = useState<string | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [form, setForm] = useState({
    objective_name: "",
    objective_type: "emergency_reserve",
    target_amount: "",
    current_amount: "",
    priority_level: "moderate",
    status: "active",
    target_date: "",
    notes: "",
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
```

Repository Audit

```
setError(null);
try {
  let snapshot;
  if (editingId) {
    snapshot = await updateStrategicObjectiveAction(editingId, {
      objective_name: form.objective_name,
      objective_type: form.objective_type,
      target_amount: Number(form.target_amount),
      current_amount: form.current_amount ? Number(form.current_amount) : 0,
      priority_level: form.priority_level,
      status: form.status,
      target_date: form.target_date || null,
      notes: form.notes || undefined,
    });
  } else {
    snapshot = await createStrategicObjectiveAction({
      objective_name: form.objective_name,
      objective_type: form.objective_type,
      target_amount: Number(form.target_amount),
      current_amount: form.current_amount ? Number(form.current_amount) : 0,
      priority_level: form.priority_level,
      status: form.status,
      target_date: form.target_date || null,
      notes: form.notes || undefined,
    });
  }
  setForm({
    objective_name: "",
    objective_type: "emergency_reserve",
    target_amount: "",
    current_amount: "",
    priority_level: "moderate",
    status: "active",
    target_date: "",
    notes: "",
  });
  setIsAdding(false);
  setEditingId(null);
  onSnapshot(snapshot);
} catch (err: unknown) {
  setError(
    err instanceof Error
      ? err.message
      : "Failed to establish strategic intent",
  );
} finally {
  setIsLoading(false);
}
}

return (
  <div className="space-y-12">
    <div className="flex items-center justify-between">
      <h2 className="font-cormorant text-2xl text-white tracking-wide uppercase">
        Strategic Intent
      </h2>
      <button
        onClick={() => setIsAdding(!isAdding)}
        className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/40 hover:text-white transition-colors"
      >
        {isAdding ? "? CANCEL" : "+ APPEND OBJECTIVE"}
      </button>
    </div>
  </div>
);
```

Repository Audit

```
{isAdding && (
  <div className="bg-white/[0.02] border border-white/5 p-8 animate-in fade-in slide-in-from-top-4
duration-500">
    <form onSubmit={handleSubmit} className="space-y-8">
      <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
        <div className="space-y-2">
          <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
            Objective Name
          </label>
          <input
            type="text"
            required
            placeholder="e.g. Emergency Fund"
            value={form.objective_name}
            onChange={(e) =>
              setForm({ ...form, objective_name: e.target.value })
            }
            className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
          />
        </div>
        <div className="space-y-2">
          <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
            Target Date
          </label>
          <input
            type="date"
            value={form.target_date}
            onChange={(e) =>
              setForm({ ...form, target_date: e.target.value })
            }
            className="w-full bg-transparent border-b border-white/10 py-3 text-white/60 font-mono
text-[10px] tracking-widest uppercase focus:outline-none"
          />
        </div>
      </div>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-2 gap-8">
      <div className="space-y-2">
        <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
          Target Amount
        </label>
        <input
          type="number"
          required
          placeholder="0.00"
          value={form.target_amount}
          onChange={(e) =>
            setForm({ ...form, target_amount: e.target.value })
          }
          className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
        />
      </div>
      <div className="space-y-2">
        <label className="text-[9px] font-mono text-white/20 uppercase tracking-widest">
          Current Position
        </label>
        <input
          type="number"
          placeholder="0.00"
          value={form.current_amount}
          onChange={(e) =>
```

Repository Audit

```
        setForm({ ...form, current_amount: e.target.value })
      }
      className="w-full bg-transparent border-b border-white/10 py-3 text-white font-light
focus:outline-none focus:border-white transition-colors"
    />
  </div>
</div>

{error && (
  <p className="text-red-500 font-mono text-[9px] uppercase tracking-widest">
    {error}
  </p>
)}

<button
  type="submit"
  disabled={isLoading}
  className="w-full bg-white text-black py-4 font-medium tracking-[0.2em] uppercase text-[10px]
hover:bg-white/90 transition-colors disabled:opacity-50"
>
  {isLoading ? "ESTABLISHING..." : "SET STRATEGIC INTENT"}
</button>
</form>
</div>
)}

<div className="space-y-8">
  {objectives.length === 0 ? (
    <div className="py-12 text-center opacity-20">
      <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
        No objectives active
      </p>
    </div>
  ) : (
    objectives.map((obj) => {
      const ratio = calculateObjectiveFundingRatio(obj);
      return (
        <div
          key={obj.id}
          className="space-y-4 group cursor-default pb-8 border-b border-white/5 last:border-0"
        >
          <div className="flex items-center justify-between mb-2">
            <div className="space-y-1">
              <span
                className={`text-[8px] font-mono tracking-widest uppercase ${
                  obj.priority_level === "critical"
                    ? "text-red-500"
                    : "text-white/20"
                }`}
              >
                {obj.priority_level}
              </span>
              <h3 className="font-cormorant text-2xl text-white transition-transform
group-hover:translate-x-2">
                {obj.objective_name}
              </h3>
            </div>
            <div className="text-right">
              <p className="font-cormorant text-3xl text-white tabular-nums">
                {Math.round(ratio)}%
              </p>
            </div>
          </div>
        </div>
      )
    })
  )
}
```

Repository Audit

```
<div className="h-[1px] w-full bg-white/5 relative overflow-hidden">
  <div
    className="absolute inset-y-0 left-0 bg-white/60 transition-all duration-1000 ease-out"
    style={{ width: `${Math.min(100, ratio)}%` }}
  />
</div>

    <div className="flex justify-between items-end pt-2 opacity-30 group-hover:opacity-60
transition-opacity">
      <p className="text-[9px] font-mono uppercase tracking-widest">
        {formatCurrency(obj.current_amount)}
      </p>
      <p className="text-[9px] font-mono uppercase tracking-widest">
        Target: {formatCurrency(obj.target_amount)}
      </p>
    </div>
  </div>
);
})
})
</div>
</div>
);
}
```

File: src/app/dashboard/actions.ts

```
"use server";

import { revalidatePath } from "next/cache";
import { createClient } from "@/lib/supabase/server";
import {
  getDeployments,
  createDeployment,
  updateDeployment,
  deleteDeployment,
} from "@/lib/db/deployments";
import {
  getAccounts,
  createAccount,
  updateAccount,
  deleteAccount,
} from "@/lib/db/accounts";
import {
  getLiabilities,
  createLiability,
  updateLiability,
  deleteLiability,
} from "@/lib/db/liabilities";
import {
  getIncomeStreams,
  createIncomeStream,
  updateIncomeStream,
  deleteIncomeStream,
} from "@/lib/db/income";
import { getGoals, createGoal, updateGoal, deleteGoal } from "@/lib/db/goals";
import {
  getStrategicObjectives,
  createStrategicObjective,
  updateStrategicObjective,
  deleteStrategicObjective,
} from "@/lib/db/objectives";
```

Repository Audit

```
import {
  getOperationalBaseline,
  createOperationalBaseline,
  updateOperationalBaseline,
  deleteOperationalBaseline,
} from "@lib/db/baseline";
import { getUserSettings, updateLiquidity } from "@lib/db/settings";
import { getInsights, saveInsight } from "@lib/db/insights";
import { getDeletedLedgerRecords } from "@lib/db/audit";
import { generateKairosAIInsight, type KairosInsight } from "@lib/ai/kairos";
import { generateSummary, type AnalyticsSummary } from "@lib/analytics";
import type {
  Deployment,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  OperationalBaseline,
  BaselineCadence,
} from "@lib/analytics/types";
import type { DeploymentAdvancedContextInput } from "@lib/finance/deploymentContext";

export interface DashboardSnapshot {
  authenticated: boolean;
  deployments: Deployment[];
  accounts: Account[];
  liabilities: Liability[];
  incomeStreams: IncomeStream[];
  goals: FinancialGoal[];
  objectives: StrategicObjective[];
  baseline: OperationalBaseline[];
  analytics: AnalyticsSummary | null;
  liquidity: number;
  kairosInsight: KairosInsight | null;
}

export interface CreateDeploymentActionInput {
  title: string;
  amount: number;
  category: string;
  advancedContext?: DeploymentAdvancedContextInput;
  accountId?: string;
}

export interface UpdateDeploymentActionInput {
  title: string;
  amount: number;
  category: string;
  accountId?: string;
}

export interface CreateAccountActionInput {
  account_name: string;
  account_type: string;
  current_balance: number;
  institution?: string;
}

export interface CreateLiabilityActionInput {
  liability_name: string;
  liability_type: string;
  outstanding_balance: number;
  interest_rate?: number;
}
```

Repository Audit

```
    minimum_payment?: number;
    due_date?: string | null;
    institution?: string;
  }

export interface CreateIncomeActionInput {
  income_name: string;
  income_type: string;
  amount: number;
  cadence: string;
  execution_day?: number | null;
  is_recurring?: boolean;
  source?: string;
  start_date?: string;
  end_date?: string | null;
}

export interface CreateGoalActionInput {
  goal_name: string;
  goal_type: string;
  target_amount: number;
  current_progress?: number;
  target_date?: string | null;
  priority: string;
  status: string;
  notes?: string;
}

export interface CreateObjectiveActionInput {
  objective_name: string;
  objective_type: string;
  target_amount: number;
  current_amount?: number;
  target_date?: string | null;
  priority_level: string;
  status: string;
  notes?: string;
}

function unauthenticatedSnapshot(): DashboardSnapshot {
  return {
    authenticated: false,
    deployments: [],
    accounts: [],
    liabilities: [],
    incomeStreams: [],
    goals: [],
    objectives: [],
    baseline: [],
    analytics: null,
    liquidity: 0,
    kairosInsight: null,
  };
}

const requireAuth = async () => {
  const supabase = await createClient();
  const {
    data: { user },
    error,
  } = await supabase.auth.getUser();
  if (error || !user) throw new Error("Unauthorized mutation attempt.");
  return { userId: user.id, supabase };
};
```

Repository Audit

```
function normalizeSavedInsight(row: Record<string, unknown>): KairosInsight {
  const metadata =
    row.metadata && typeof row.metadata === "object"
      ? (row.metadata as Record<string, unknown>)
      : {};

  return {
    type: row.type as KairosInsight["type"],
    severity: (metadata.severity as KairosInsight["severity"]) || "observation",
    category: row.category as KairosInsight["category"],
    message: String(row.message || ""),
    supportingSignals: Array.isArray(metadata.supporting_signals)
      ? (metadata.supporting_signals as string[])
      : metadata.supporting_signal
        ? [String(metadata.supporting_signal)]
        : [],
    timestamp: String(
      metadata.timestamp || row.created_at || new Date().toISOString(),
    ),
    runway: metadata.runway !== undefined ? Number(metadata.runway) : null,
    capitalEfficiency:
      metadata.capital_efficiency !== undefined
        ? Number(metadata.capital_efficiency)
        : 100,
    isSilent: Boolean(metadata.is_silent),
    metadataQuality:
      metadata.metadata_quality && typeof metadata.metadata_quality === "object"
        ? (metadata.metadata_quality as KairosInsight["metadataQuality"])
        : undefined,
    is_new_signal: false,
  };
}

async function buildDashboardSnapshot(
  options: { forceInsightEvaluation?: boolean } = {},
): Promise<DashboardSnapshot> {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();

  if (!user) return unauthenticatedSnapshot();

  console.log("ACTIVE SESSION USER ID:", user.id);

  const [
    deploymentsData,
    accountsData,
    liabilitiesData,
    incomeStreamsData,
    goalsData,
    objectivesData,
    baselineData,
    settings,
  ] = await Promise.all([
    getDeployments(supabase),
    getAccounts(supabase),
    getLiabilities(supabase),
    getIncomeStreams(supabase),
    getGoals(supabase),
    getStrategicObjectives(supabase),
    getOperationalBaseline(supabase),
    getUserSettings(supabase, user.id),
  ])
```

Repository Audit

```
]);

const deployments = (deploymentsData || []) as Deployment[];
const accounts = (accountsData || []) as Account[];
const liabilities = (liabilitiesData || []) as Liability[];
const incomeStreams = (incomeStreamsData || []) as IncomeStream[];
const goals = (goalsData || []) as FinancialGoal[];
const objectives = (objectivesData || []) as StrategicObjective[];
const baseline = (baselineData || []) as OperationalBaseline[];
const liquidity = Number(settings.total_liquidity);
const analytics = generateSummary(
  deployments,
  liquidity,
  accounts,
  liabilities,
  incomeStreams,
  goals,
  objectives,
  baseline,
);
const savedInsights = await getInsights(supabase, 1);
const previousInsight =
  savedInsights && savedInsights.length > 0
    ? normalizeSavedInsight(savedInsights[0] as Record<string, unknown>)
    : null;

let kairosInsight: KairosInsight | null = null;

if (!options.forceInsightEvaluation && previousInsight) {
  // OPTIMIZATION SHORTCUT: Reuse cached strategic signal
  kairosInsight = previousInsight;
} else {
  // FRESH EVALUATION: Process authoritative telemetry
  kairosInsight = await generateKairosAIInsight({
    deployments,
    liquidity,
    previousInsight,
    objectives,
    accounts,
    liabilities,
    incomeStreams,
    goals,
    baseline,
  });

  // Only save if it's actually a material change
  if (kairosInsight.is_new_signal) {
    await saveInsight(supabase, kairosInsight, user.id);
  }
}

return {
  authenticated: true,
  deployments,
  accounts,
  liabilities,
  incomeStreams,
  goals,
  objectives,
  baseline,
  analytics,
  liquidity,
  kairosInsight,
};
```

Repository Audit

```
}

export async function getDashboardSnapshotAction() {
  return buildDashboardSnapshot();
}

export async function checkOnboardingStatusAction() {
  const { userId, supabase } = await requireAuth();

  const [accounts, baseline] = await Promise.all([
    supabase
      .from("accounts")
      .select("id", { count: "exact", head: true })
      .eq("user_id", userId),
    supabase
      .from("operational_baseline")
      .select("id", { count: "exact", head: true })
      .eq("user_id", userId),
  ]);

  return {
    isOnboarded: (accounts.count ?? 0) > 0 || (baseline.count ?? 0) > 0,
  };
}

export async function createDeploymentAction(
  input: CreateDeploymentActionInput,
) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Create the deployment record (The database trigger handles account balance deduction)
    await createDeployment(
      supabase,
      input.title,
      input.amount,
      userId,
      input.category,
      0,
      input.advancedContext,
      input.accountId,
    );

    // 2. Update global liquidity setting if this was a liquidity deployment
    if (input.accountId) {
      const settings = await getUserSettings(supabase, userId);
      const newLiquidity = Number(settings.total_liquidity) - input.amount;
      await updateLiquidity(supabase, userId, newLiquidity);
    }

    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create deployment:", error);
    throw error;
  }
}

export async function updateDeploymentAction(
  id: string,
  input: UpdateDeploymentActionInput,
) {
  const { userId, supabase } = await requireAuth();
```

Repository Audit

```
try {
  await updateDeployment(supabase, id, userId, {
    title: input.title,
    amount: input.amount,
    category: input.category,
  });
  revalidatePath("/dashboard");
  return buildDashboardSnapshot({ forceInsightEvaluation: true });
} catch (error) {
  console.error("Failed to update deployment:", error);
  throw error;
}
}

export async function deleteDeploymentAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteDeployment(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete deployment:", error);
    throw error;
  }
}

export async function updateLiquidityAction(amount: number) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateLiquidity(supabase, userId, amount);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update liquidity:", error);
    throw error;
  }
}

export async function createAccountAction(input: CreateAccountActionInput) {
  const { userId, supabase } = await requireAuth();

  try {
    await createAccount(supabase, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create account:", error);
    throw error;
  }
}

export async function updateAccountAction(
  id: string,
  input: Partial<CreateAccountActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateAccount(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
```

Repository Audit

```
        console.error("Failed to update account:", error);
        throw error;
    }
}

export async function deleteAccountAction(id: string) {
    const { userId, supabase } = await requireAuth();

    try {
        await deleteAccount(supabase, id, userId);
        revalidatePath("/dashboard");
        return buildDashboardSnapshot({ forceInsightEvaluation: true });
    } catch (error) {
        console.error("Failed to delete account:", error);
        throw error;
    }
}

export async function createLiabilityAction(input: CreateLiabilityActionInput) {
    const { userId, supabase } = await requireAuth();

    try {
        await createLiability(supabase, userId, input);
        revalidatePath("/dashboard");
        return buildDashboardSnapshot({ forceInsightEvaluation: true });
    } catch (error) {
        console.error("Failed to create liability:", error);
        throw error;
    }
}

export async function updateLiabilityAction(
    id: string,
    input: Partial<CreateLiabilityActionInput>,
) {
    const { userId, supabase } = await requireAuth();

    try {
        await updateLiability(supabase, id, userId, input);
        revalidatePath("/dashboard");
        return buildDashboardSnapshot({ forceInsightEvaluation: true });
    } catch (error) {
        console.error("Failed to update liability:", error);
        throw error;
    }
}

export async function deleteLiabilityAction(id: string) {
    const { userId, supabase } = await requireAuth();

    try {
        await deleteLiability(supabase, id, userId);
        revalidatePath("/dashboard");
        return buildDashboardSnapshot({ forceInsightEvaluation: true });
    } catch (error) {
        console.error("Failed to delete liability:", error);
        throw error;
    }
}

export async function createIncomeAction(inputs: CreateIncomeActionInput[]) {
    const { userId, supabase } = await requireAuth();

    try {
```

Repository Audit

```
    await createIncomeStreams(supabase, userId, inputs);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create income streams:", error);
    throw error;
  }
}

export async function updateIncomeAction(
  id: string,
  input: Partial<CreateIncomeActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateIncomeStream(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update income stream:", error);
    throw error;
  }
}

export async function deleteIncomeAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteIncomeStream(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete income stream:", error);
    throw error;
  }
}

export async function resolvePendingInflowAction(payload: {
  incomeId: string;
  accountId: string;
  amount: number;
}) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Get current account balance
    const { data: account, error: accError } = await supabase
      .from("accounts")
      .select("current_balance")
      .eq("id", payload.accountId)
      .eq("user_id", userId)
      .single();

    if (accError || !account) throw new Error("Target account not found.");

    // 2. Update the target account balance
    const newBalance = Number(account.current_balance) + payload.amount;
    await updateAccount(supabase, payload.accountId, userId, {
      current_balance: newBalance,
    });

    // 3. Update the income stream last_executed_at
    await updateIncomeStream(supabase, payload.incomeId, userId, {
```

Repository Audit

```
    last_executed_at: new Date().toISOString(),
  });

  // 4. Update global liquidity if this account is part of it
  const settings = await getUserSettings(supabase, userId);
  const newLiquidity = Number(settings.total_liquidity) + payload.amount;
  await updateLiquidity(supabase, userId, newLiquidity);

  revalidatePath("/dashboard");
  return buildDashboardSnapshot({ forceInsightEvaluation: true });
} catch (error) {
  console.error("Failed to resolve pending inflow:", error);
  throw error;
}
}

export async function createGoalAction(input: CreateGoalActionInput) {
  const { userId, supabase } = await requireAuth();

  try {
    await createGoal(supabase, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create goal:", error);
    throw error;
  }
}

export async function updateGoalAction(
  id: string,
  input: Partial<CreateGoalActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateGoal(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update goal:", error);
    throw error;
  }
}

export async function deleteGoalAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteGoal(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete goal:", error);
    throw error;
  }
}

export async function createStrategicObjectiveAction(
  input: CreateObjectiveActionInput,
) {
  const { userId, supabase } = await requireAuth();

  try {
```

Repository Audit

```
    await createStrategicObjective(supabase, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create strategic objective:", error);
    throw error;
  }
}

export const createObjectiveAction = createStrategicObjectiveAction;

export async function updateStrategicObjectiveAction(
  id: string,
  input: Partial<CreateObjectiveActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateStrategicObjective(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update strategic objective:", error);
    throw error;
  }
}

export const updateObjectiveAction = updateStrategicObjectiveAction;

export async function deleteStrategicObjectiveAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteStrategicObjective(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete strategic objective:", error);
    throw error;
  }
}

export const deleteObjectiveAction = deleteStrategicObjectiveAction;

export async function createOperationalBaselineAction(
  data: Omit<
    OperationalBaseline,
    "id" | "user_id" | "created_at" | "updated_at"
  >,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await createOperationalBaseline(supabase, userId, data);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create operational baseline:", error);
    throw error;
  }
}

export async function updateOperationalBaselineAction(
  id: string,
```

Repository Audit

```
updates: Partial<
  Omit<OperationalBaseline, "id" | "user_id" | "created_at" | "updated_at">
>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateOperationalBaseline(supabase, id, userId, updates);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update operational baseline:", error);
    throw error;
  }
}

export async function deleteOperationalBaselineAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteOperationalBaseline(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete operational baseline:", error);
    throw error;
  }
}

export async function submitDayZeroBaselineAction(payload: {
  accounts: {
    account_name: string;
    account_type: string;
    current_balance: number;
  }[];
  incomes: {
    income_name: string;
    income_type: string;
    amount: number;
    cadence: string;
  }[];
  liabilities: {
    liability_name: string;
    liability_type: string;
    outstanding_balance: number;
  }[];
  baseline: { amount: number; cadence: string };
}) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Insert Accounts
    for (const acc of payload.accounts) {
      await createAccount(supabase, userId, acc);
    }

    // 2. Insert Incomes (if any)
    if (payload.incomes.length > 0) {
      await createIncomeStreams(
        supabase,
        userId,
        payload.incomes.map((inc) => ({
          ...inc,
          is_recurring: true,

```

Repository Audit

```
    })),
  );
}

// 3. Insert Liabilities (if any)
for (const liab of payload.liabilities) {
  if (liab.outstanding_balance > 0) {
    await createLiability(supabase, userId, liab);
  }
}

// 4. Insert Operational Baseline
await createOperationalBaseline(supabase, userId, {
  title: "Core Survival Baseline",
  amount: payload.baseline.amount,
  category: "Maintenance",
  cadence: payload.baseline.cadence as BaselineCadence,
  baseline_type: "expense",
  is_active: true,
});

// 5. Update initial liquidity pool to match sum of all accounts
const totalLiquidity = payload.accounts.reduce(
  (sum, acc) => sum + acc.current_balance,
  0,
);
await updateLiquidity(supabase, userId, totalLiquidity);

revalidatePath("/dashboard");
return buildDashboardSnapshot({ forceInsightEvaluation: true });
} catch (error) {
  console.error("Day Zero Onboarding failed:", error);
  throw error;
}
}

export async function fetchHistoricalAuditAction() {
  const { userId, supabase } = await requireAuth();
  return getDeletedLedgerRecords(supabase, userId);
}

export interface AuditLog {
  id: string;
  rule_id: string;
  severity: string;
  evaluation_time_ms: number;
  created_at: string;
}

export interface TelemetrySummary {
  logs: AuditLog[];
  averageLatency: number;
  matchRate: number;
  totalCycles: number;
  ruleHits: Record<string, number>;
}

export async function fetchTelemetryLogsAction(): Promise<TelemetrySummary> {
  const { userId, supabase } = await requireAuth();

  const { data, error } = await supabase
    .from("kairos_insights")
    .select("*")
    .eq("user_id", userId)
```

Repository Audit

```
.order("created_at", { ascending: false })
.limit(50);

if (error) {
  console.error("[Telemetry] Fetch failed:", error.message);
  return {
    logs: [],
    averageLatency: 0,
    matchRate: 0,
    totalCycles: 0,
    ruleHits: {},
  };
}

const logs = (data || []) as AuditLog[];
const totalLogs = logs.length;

if (totalLogs === 0) {
  return {
    logs: [],
    averageLatency: 0,
    matchRate: 0,
    totalCycles: 0,
    ruleHits: {},
  };
}

const totalLatency = logs.reduce(
  (sum, log) => sum + (log.evaluation_time_ms || 0),
  0,
);
const averageLatency = totalLatency / totalLogs;

const matches = logs.filter((log) => log.severity !== "none").length;
const matchRate = (matches / totalLogs) * 100;

const ruleHits: Record<string, number> = {};
logs.forEach((log) => {
  if (log.severity && log.severity !== "none") {
    ruleHits[log.rule_id] = (ruleHits[log.rule_id] || 0) + 1;
  }
});

return {
  logs,
  averageLatency: Math.round(averageLatency * 100) / 100,
  matchRate: Math.round(matchRate * 10) / 10,
  totalCycles: totalLogs,
  ruleHits,
};
}
```

File: src/app/dashboard/error.tsx

```
"use client";

import { useEffect } from "react";

export default function DashboardError({
  error,
  reset,
}: {
  error: Error & { digest?: string };
}) {
```

Repository Audit

```
    reset: () => void;
  }) {
    useEffect(() => {
      // Log the error to an error reporting service
      console.error("Dashboard Route Error:", error);
    }, [error]);

    return (
      <main className="min-h-[80vh] flex items-center justify-center p-6">
        <div className="w-full max-w-md bg-background border-2 border-red-500/20 rounded-[2.5rem] p-10 shadow-2xl text-center">
          <div className="w-20 h-20 bg-red-500/10 rounded-3xl flex items-center justify-center mx-auto mb-6">
            <svg
              width="40"
              height="40"
              viewBox="0 0 24 24"
              fill="none"
              stroke="currentColor"
              strokeWidth="3"
              className="text-red-500"
            >
              <path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"
            />
            <line x1="12" y1="9" x2="12" y2="13" />
            <line x1="12" y1="17" x2="12.01" y2="17" />
          </svg>
        </div>

        <h1 className="text-3xl font-black tracking-tighter uppercase mb-4 text-foreground">
          System Interrupted
        </h1>

        <p className="text-foreground/60 text-sm font-bold uppercase tracking-widest mb-8 leading-relaxed">
          The intelligence engine encountered an unhandled exception.
        </p>

        <div className="bg-foreground/5 rounded-2xl p-4 mb-8 text-left">
          <p className="text-[10px] font-black text-foreground/40 uppercase tracking-widest mb-2">
            Error Diagnostic
          </p>
          <p className="text-xs font-mono text-red-400 break-all leading-tight">
            {error.message || "Unknown cryptographic failure"}
          </p>
        </div>

        <button
          onClick={() => reset()}
          className="w-full bg-foreground text-background px-4 py-5 rounded-2xl font-black text-lg
          hover:scale-[1.02] active:scale-[0.98] transition-all shadow-xl shadow-foreground/10 uppercase"
        >
          Attempt Recovery
        </button>
      </div>
    </main>
  );
}
```

File: src/app/dashboard/layout.tsx

```
"use client";

import { supabase } from "@/lib/supabase";
import { useEffect, useState } from "react";
```

Repository Audit

```
import { User } from "@supabase/supabase-js";
import Link from "next/link";
import { SmoothScrollProvider } from "@components/ui/smooth-scroll-provider";
import { checkOnboardingStatusAction } from "../actions";
import { BrandMark } from "@components/ui/brand-mark";

export const dynamic = "force-dynamic";

export default function DashboardLayout({
  children,
}): {
  children: React.ReactNode;
}) {
  const [user, setUser] = useState<User | null>(null);
  const [isProfileOpen, setIsProfileOpen] = useState(false);
  const [activeSection, setActiveSection] = useState("overview");
  const [isOnboarded, setIsOnboarded] = useState<boolean | null>(null);

  useEffect(() => {
    async function init() {
      const {
        data: { user },
      } = await supabase.auth.getUser();
      setUser(user);

      try {
        const { isOnboarded } = await checkOnboardingStatusAction();
        setIsOnboarded(isOnboarded);
      } catch {
        setIsOnboarded(false);
      }
    }
    init();

    const handleScroll = () => {
      const sections = ["overview", "deploy", "intelligence", "ledger"];
      const scrollTop = window.scrollY + 200;

      for (const section of sections) {
        const element = document.getElementById(section);
        if (element) {
          const { offsetTop, offsetHeight } = element;
          if (
            scrollTop >= offsetTop &&
            scrollTop < offsetTop + offsetHeight
          ) {
            setActiveSection(section);
            break;
          }
        }
      }
    };

    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll);
  }, []);

  async function handleLogout() {
    await supabase.auth.signOut();
    window.location.href = "/";
  }

  const navItems = [
    { id: "overview", label: "Overview", roman: "I" },
  ]
```

Repository Audit

```
{ id: "deploy", label: "Architecture", roman: "II" },
{ id: "intelligence", label: "Intelligence", roman: "III" },
{ id: "ledger", label: "Ledger", roman: "IV" },
];

const scrollToSection = (id: string) => {
  const element = document.getElementById(id);
  if (element) {
    const offset = 100;
    const bodyRect = document.body.getBoundingClientRect().top;
    const elementRect = element.getBoundingClientRect().top;
    const elementPosition = elementRect - bodyRect;
    const offsetPosition = elementPosition - offset;

    window.scrollTo({
      top: offsetPosition,
      behavior: "smooth",
    });
  }
};

return (
  <div className="min-h-screen bg-black text-white flex flex-col md:flex-row relative selection:bg-white selection:text-black">
    /* Background Atmosphere */
    <div className="fixed inset-0 z-0">
      <div className="absolute top-0 right-0 w-[40%] h-[40%] bg-white/[0.02] rounded-full blur-[120px]" />
      <div className="absolute bottom-0 left-0 w-[40%] h-[40%] bg-white/[0.02] rounded-full blur-[120px]" />
      <div className="absolute inset-0 bg-[url('https://www.transparenttextures.com/patterns/carbon-fibre.png')] opacity-10 pointer-events-none" />
    </div>

    {isOnboarded === true && (
      <>
        /* SIDEBAR */
        <aside className="hidden md:flex flex-col fixed left-0 top-0 bottom-0 w-80 bg-black border-r border-white/5 z-50 p-12 animate-in fade-in slide-in-from-left-4 duration-1000">
          <div className="flex flex-col h-full justify-between">
            <div className="space-y-24">
              /* Logo */
              <Link href="/" className="group">
                <div className="flex items-center gap-4">
                  <BrandMark />
                  <div className="flex flex-col">
                    <span className="font-mono text-[10px] tracking-[0.4em] uppercase font-bold text-white">
                      AXIOM
                    </span>
                    <span className="text-[7px] font-mono tracking-[0.2em] uppercase text-white/30">
                      Intelligence Layer
                    </span>
                  </div>
                </div>
              </Link>

              /* Navigation */
              <nav className="space-y-12">
                <p className="text-[10px] font-mono tracking-[0.5em] text-white/20 uppercase mb-8">
                  Client Portal
                </p>
                {navItems.map((item) => (
                  <button
                    key={item.id}
                    onClick={() => scrollToSection(item.id)}
                    className={`w-full group text-left transition-all ${`

```

Repository Audit

```
        activeSection === item.id
        ? "opacity-100"
        : "opacity-30 hover:opacity-100"
    }`}}
  >
  <div className="flex items-center gap-6">
    <span className="font-cormorant italic text-2xl text-white/60">
      {item.roman}.
    </span>
    <span
      className={`font-cormorant text-2xl tracking-wide transition-all ${
        activeSection === item.id
        ? "translate-x-2"
        : "group-hover:translate-x-2"
      }`}
    >
      {item.label}
    </span>
  </div>
</button>
)}}
</nav>
</div>

{/* User Profile */}
<div className="pt-12 border-t border-white/5 relative">
  <button
    onClick={() => setIsProfileOpen(!isProfileOpen)}
    className="flex items-center gap-4 w-full group"
  >
    <div className="w-10 h-10 border border-white/10 flex items-center justify-center font-mono
text-xs text-white/60 group-hover:bg-white group-hover:text-black transition-all">
      {user?.email?.[0].toUpperCase()}
    </div>
    <div className="flex flex-col text-left">
      <span className="text-[10px] font-mono tracking-widest text-white/80 uppercase truncate
max-w-[120px]">
        {user?.email?.split("@")[0]}
      </span>
      <span className="text-[8px] font-mono tracking-widest text-white/20 uppercase">
        Verified Session
      </span>
    </div>
  </button>

  {isProfileOpen && (
    <div className="absolute bottom-full left-0 mb-4 w-full bg-black border border-white/10 p-4
space-y-2 animate-in fade-in slide-in-from-bottom-2 duration-200">
      <button
        onClick={handleLogout}
        className="w-full text-left font-mono text-[9px] tracking-widest text-red-500
hover:text-red-400 uppercase py-2"
      >
        Terminate Session
      </button>
    </div>
  )}
</div>
</div>
</aside>

{/* MOBILE HEADER */}
<header className="md:hidden sticky top-0 bg-black/80 backdrop-blur-xl border-b border-white/5 z-50
px-6 py-6 flex justify-between items-center">
```

Repository Audit

```
<div className="flex items-center gap-3">
  <BrandMark className="w-8 h-8" />
  <span className="font-mono text-[10px] tracking-widest uppercase text-white/40">
    Axiom // Portal
  </span>
</div>
<button onClick={() => setIsProfileOpen(!isProfileOpen)}>
  <div className="w-8 h-8 border border-white/10 flex items-center justify-center font-mono
text-[10px]">
    {user?.email?.[0].toUpperCase()}
  </div>
</button>
</header>
</>
)}

{/* MAIN CONTENT AREA */}
<main
  className={`flex-1 min-h-screen relative z-10 transition-all duration-1000 ${isOnboarded === true ?
"md:ml-80" : "md:ml-0"}`}
>
  <div
    className={`max-w-5xl mx-auto px-6 md:px-16 py-12 md:py-24 ${isOnboarded === false ? "h-full flex
items-center justify-center" : ""}`}
  >
    {children}
  </div>
</main>

{/* MOBILE BOTTOM NAV */}
{isOnboarded === true && (
  <nav className="md:hidden fixed bottom-0 left-0 right-0 bg-black/80 backdrop-blur-xl border-t
border-white/5 z-50 px-6 py-4 flex justify-between items-center">
    {navItems.map((item) => (
      <button
        key={item.id}
        onClick={() => scrollToSection(item.id)}
        className={`font-mono text-[8px] tracking-widest uppercase transition-all ${
          activeSection === item.id ? "text-white" : "text-white/20"}
        `}
      >
        {item.label}
      </button>
    ))}
  </nav>
)}
</div>
);
}
```

File: src/app/dashboard/loading.tsx

```
export default function DashboardLoading() {
  return (
    <div className="max-w-6xl mx-auto p-6 pb-20 animate-pulse">
      <div className="flex flex-col md:flex-row md:items-end justify-between gap-6 mb-12">
        <div className="space-y-3">
          <div className="h-12 w-64 bg-foreground/10 rounded-2xl"></div>
          <div className="h-4 w-48 bg-foreground/10 rounded-full"></div>
        </div>
        <div className="flex gap-4">
          <div className="h-20 w-40 bg-foreground/10 rounded-2xl"></div>
          <div className="h-20 w-40 bg-foreground/10 rounded-2xl"></div>
        </div>
      </div>
    )
  )
}
```

Repository Audit

```
    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-12 gap-8">
    <div className="lg:col-span-4 space-y-6">
      <div className="h-96 bg-foreground/10 rounded-[2.5rem]"></div>
      <div className="h-48 bg-foreground/10 rounded-[2.5rem]"></div>
    </div>
    <div className="lg:col-span-8 space-y-10">
      <div className="space-y-4">
        <div className="h-8 w-48 bg-foreground/10 rounded-full"></div>
        <div className="grid grid-cols-2 gap-4">
          <div className="h-32 bg-foreground/10 rounded-3xl"></div>
          <div className="h-32 bg-foreground/10 rounded-3xl"></div>
        </div>
      </div>
      <div className="space-y-4">
        <div className="h-8 w-48 bg-foreground/10 rounded-full"></div>
        <div className="space-y-4">
          <div className="h-24 bg-foreground/10 rounded-3xl"></div>
          <div className="h-24 bg-foreground/10 rounded-3xl"></div>
          <div className="h-24 bg-foreground/10 rounded-3xl"></div>
        </div>
      </div>
    </div>
  </div>
);
}
```

File: src/app/dashboard/page.tsx

```
"use client";

import { useEffect, useState, useCallback, useRef } from "react";
import {
  getDashboardSnapshotAction,
  createDeploymentAction,
  type DashboardSnapshot,
} from "../actions";
import { AccountSection } from "../AccountSection";
import { LiabilitySection } from "../LiabilitySection";
import { IncomeSection } from "../IncomeSection";
import { GoalSection } from "../GoalSection";
import { StrategicObjectiveSection } from "../StrategicObjectiveSection";
import { BaselineSection } from "../BaselineSection";
import { PendingInflows } from "../PendingInflows";
import { TelemetryDashboard } from "@components/dashboard/TelemetryDashboard";
import DayZeroOnboarding from "@components/dashboard/DayZeroOnboarding";
import { NewEntryForm } from "../NewEntryForm";
import type {
  AnalyticsSummary,
  Deployment,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  OperationalBaseline,
} from "@lib/analytics/types";
import { formatCurrency } from "@lib/utils/formatters";
import { DeploymentMap } from "@lib/utils/taxonomy";
```

Repository Audit

```
interface LedgerState {
  deployments: Deployment[];
  accounts: Account[];
  liabilities: Liability[];
  incomeStreams: IncomeStream[];
  goals: FinancialGoal[];
  objectives: StrategicObjective[];
  baseline: OperationalBaseline[];
  analytics: AnalyticsSummary | null;
}

type KairosInsight = DashboardSnapshot["kairosInsight"];

const EMPTY_ADVANCED_CONTEXT = {
  associatedAccount: "",
  expectedReturnHorizon: "",
  tags: "",
};

export default function Dashboard() {
  const [ledger, setLedger] = useState<LedgerState>({
    deployments: [],
    accounts: [],
    liabilities: [],
    incomeStreams: [],
    goals: [],
    objectives: [],
    baseline: [],
    analytics: null,
  });
  const [liquidity, setLiquidity] = useState<number>(0);
  const [kairosInsight, setKairosInsight] = useState<KairosInsight | null>(
    null,
  );

  const [isInitialLoading, setIsInitialLoading] = useState(true);
  const [isActionLoading, setIsActionLoading] = useState(false);
  const [isIntelligenceSyncing, setIsIntelligenceSyncing] = useState(false);

  const isExecuting = useRef(false);
  const fetchCount = useRef(0);

  const applyDashboardSnapshot = useCallback((snapshot: DashboardSnapshot) => {
    if (!snapshot.authenticated) return;
    setLiquidity(snapshot.liquidity);
    setLedger({
      deployments: snapshot.deployments as Deployment[],
      accounts: snapshot.accounts,
      liabilities: snapshot.liabilities,
      incomeStreams: snapshot.incomeStreams,
      goals: snapshot.goals,
      objectives: snapshot.objectives,
      baseline: snapshot.baseline,
      analytics: snapshot.analytics as AnalyticsSummary | null,
    });
    setKairosInsight(snapshot.kairosInsight);
  }, []);

  const fetchDashboardData = useCallback(async () => {
    const requestId = ++fetchCount.current;
    try {
      const snapshot = await getDashboardSnapshotAction();
      if (requestId !== fetchCount.current) return;
      applyDashboardSnapshot(snapshot);
    }
  }, []);
```

Repository Audit

```
    } catch {
      // Fail silently
    } finally {
      if (requestId === fetchCount.current) setIsInitialLoading(false);
    }
  }, [applyDashboardSnapshot]);

useEffect(() => {
  let mounted = true;
  const load = async () => {
    if (!mounted) return;
    await fetchDashboardData();
  };
  void load();
  return () => {
    mounted = false;
  };
}, [fetchDashboardData]);

if (isInitialLoading) {
  return (
    <div className="max-w-6xl mx-auto py-12 space-y-24 animate-pulse">
      <div className="space-y-4 text-center">
        <div className="h-4 w-32 bg-white/5 mx-auto rounded" />
        <div className="h-24 w-64 bg-white/5 mx-auto rounded" />
      </div>
      <div className="grid grid-cols-3 gap-8">
        <div className="h-40 bg-white/5 rounded" />
        <div className="h-40 bg-white/5 rounded" />
        <div className="h-40 bg-white/5 rounded" />
      </div>
    </div>
  );
}

if (ledger.accounts.length === 0 && ledger.baseline.length === 0) {
  return <DayZeroOnboarding onComplete={applyDashboardSnapshot} />;
}

return (
  <div className="space-y-32">
    </* Zone 0 ? PSYCHOLOGICAL ANCHOR (SOLVENCY) */>
    <section className="text-center space-y-8 animate-in fade-in duration-1000">
      <div className="inline-flex items-center gap-4 px-6 py-2 border border-white/10 backdrop-blur-sm rounded-full mb-4">
        <div className="w-1.5 h-1.5 rounded-full bg-white/40 animate-pulse"></div>
        <span className="text-[10px] font-mono uppercase tracking-[0.4em] text-white/40">
          Structural Solvency Window
        </span>
      </div>

      <div className="flex flex-col items-center">
        <div className="font-cormorant italic text-2xl text-white/40 mb-2">
          Remaining Capital Life
        </div>
        <h1 className="font-cormorant text-8xl md:text-[10rem] text-white tracking-tighter tabular-nums leading-none">
          {ledger.analytics?.runwayDays !== null &&
            ledger.analytics?.runwayDays !== undefined
            ? Math.round(ledger.analytics.runwayDays)
            : "?"}
        <span className="text-2xl font-sans font-light text-white/20 ml-4 tracking-widest uppercase">
          Days
        </span>
      </div>
    </section>
  </div>
);
```

Repository Audit

```
</h1>
</div>

<div className="max-w-xl mx-auto pt-12 space-y-4">
  <div className="h-[1px] w-full bg-white/10 relative overflow-hidden">
    <div
      className="absolute inset-y-0 left-0 bg-white/60 transition-all duration-[2000ms] ease-out"
      style={{
        width: `${Math.min(100, (ledger.analytics?.runwayDays ?? 0) / 3.65)}%`,
      }}
    ></div>
  </div>
  <p className="text-[9px] font-mono text-white/20 uppercase tracking-[0.4em]">
    Deterministic survival projection :: v1.0
  </p>
</div>
</section>

{/* Zone 1 ? FINANCIAL POSITION */}
<section id="overview" className="space-y-16">
  <div className="flex items-center gap-6">
    <span className="font-cormorant italic text-3xl text-white/20">
      I.
    </span>
    <h2 className="font-cormorant text-4xl text-white tracking-wide">
      Financial Position
    </h2>
    <div className="flex-1 h-[1px] bg-white/5" />
  </div>

  <PendingInflows
    incomeStreams={ledger.incomeStreams}
    accounts={ledger.accounts}
    onSnapshot={applyDashboardSnapshot}
  />

  <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
    {[
      {
        label: "Net Worth",
        value: formatCurrency(ledger.analytics?.netWorth || 0),
        sub: "Total Equity",
      },
      {
        label: "Total Capital",
        value: formatCurrency(liquidity),
        sub: "Liquid Assets",
      },
      {
        label: "Monthly Inflow",
        value: formatCurrency(ledger.analytics?.totalMonthlyIncome || 0),
        sub: "Replenishment",
      },
    ]}.map((card) => (
      <div
        key={card.label}
        className="bg-white/[0.02] border border-white/5 p-10 group hover:bg-white hover:text-black
        transition-all duration-500"
      >
        <span className="text-[10px] font-mono text-white/40 group-hover:text-black/40 uppercase
        tracking-[0.3em] mb-8 block">
          {card.label}
        </span>
        <span className="font-cormorant text-4xl md:text-5xl block mb-4 transition-transform
```

Repository Audit

```
group-hover:translate-x-2">
    {card.value}
  </span>
    <p className="text-[9px] font-mono text-white/20 group-hover:text-black/20 uppercase
tracking-widest">
    {card.sub}
  </p>
</div>
)}}
</div>
</section>

{/* Zone 2 ? ARCHITECTURE ENTRY */}
<section id="deploy" className="max-w-4xl mx-auto w-full">
  <div className="bg-white/[0.02] border border-white/5 p-12 md:p-20 relative overflow-hidden">
    <div className="absolute top-0 right-0 p-8 opacity-5">
      <svg width="120" height="120" viewBox="0 0 24 24" fill="white">
        <path d="M12 2L4.5 20.29L7.1 12 12 18L16.79 3 .71-.71z" />
      </svg>
    </div>

    <div className="flex items-center gap-8 mb-16">
      <span className="font-cormorant italic text-3xl text-white/20">
        II.
      </span>
      <div className="space-y-2">
        <h2 className="font-cormorant text-5xl text-white leading-none">
          Architecture Deployment
        </h2>
        <p className="text-white/30 text-[10px] font-mono uppercase tracking-[0.4em]">
          Strategic Capital Allocation
        </p>
      </div>
    </div>

    <NewEntryForm
      accounts={ledger.accounts}
      liquidity={liquidity}
      isActionLoading={isActionLoading}
      onSubmit={async (data) => {
        if (isExecuting.current) return;
        setIsActionLoading(true);
        isExecuting.current = true;
        setIsIntelligenceSyncing(true);
        try {
          const snapshot = await createDeploymentAction({
            title: data.title,
            amount: data.amount,
            category: data.category,
            accountId: data.accountId,
            advancedContext: EMPTY_ADVANCED_CONTEXT,
          });
          applyDashboardSnapshot(snapshot);
        } finally {
          setIsActionLoading(false);
          isExecuting.current = false;
          setIsIntelligenceSyncing(false);
        }
      }}
    />
  </div>
</section>

{/* Subsections Grid */}
```

Repository Audit

```
<div className="grid grid-cols-1 md:grid-cols-2 gap-12">
  <div
    id="accounts"
    className="bg-white/[0.02] border border-white/5 p-10 transition-all hover:border-white/20"
  >
    <AccountSection
      accounts={ledger.accounts}
      onSnapshot={applyDashboardSnapshot}
    />
  </div>
  <div
    id="liabilities"
    className="bg-white/[0.02] border border-white/5 p-10 transition-all hover:border-white/20"
  >
    <LiabilitySection
      liabilities={ledger.liabilities}
      onSnapshot={applyDashboardSnapshot}
    />
  </div>
  <div
    id="income"
    className="bg-white/[0.02] border border-white/5 p-10 transition-all hover:border-white/20"
  >
    <IncomeSection
      incomeStreams={ledger.incomeStreams}
      onSnapshot={applyDashboardSnapshot}
    />
  </div>
  <div
    id="baseline"
    className="bg-white/[0.02] border border-white/5 p-10 transition-all hover:border-white/20"
  >
    <BaselineSection
      baseline={ledger.baseline}
      onSnapshot={applyDashboardSnapshot}
    />
  </div>
  <div
    id="objectives"
    className="bg-white/[0.02] border border-white/5 p-10 transition-all hover:border-white/20"
  >
    <StrategicObjectiveSection
      objectives={ledger.objectives}
      onSnapshot={applyDashboardSnapshot}
    />
  </div>
  <div
    id="goals"
    className="bg-white/[0.02] border border-white/5 p-10 transition-all hover:border-white/20"
  >
    <GoalSection
      goals={ledger.goals}
      onSnapshot={applyDashboardSnapshot}
    />
  </div>
</div>

{/* Zone 3 ? INTELLIGENCE */}
<section id="intelligence" className="space-y-16">
  <div className="flex items-center gap-6">
    <span className="font-cormorant italic text-3xl text-white/20">
      III.
    </span>
    <h2 className="font-cormorant text-4xl text-white tracking-wide">
```

Repository Audit

```
Kairos Intelligence
</h2>
<div className="flex-1 h-[1px] bg-white/5" />
</div>

<div className="grid grid-cols-1 lg:grid-cols-12 gap-12">
  <div
    className={`lg:col-span-8 bg-white p-12 md:p-20 text-black flex flex-col justify-between
transition-all duration-700 ${isIntelligenceSyncing ? "opacity-50 grayscale" : "opacity-100"}}`}
  >
    <div>
      <div className="flex items-center justify-between border-b border-black/10 pb-8 mb-12">
        <div className="flex items-center gap-4">
          <div
            className={`w-2 h-2 rounded-full ${kairosInsight?.severity === "critical" ? "bg-red-600
animate-pulse" : "bg-black/20"}}`}
          ></div>
          <span className="text-[10px] font-mono uppercase tracking-[0.4em] text-black/40">
            {kairosInsight?.category?.replace("_", " ") ||
              "SESSION MONITOR"}
          </span>
        </div>
        <span className="text-[10px] font-mono tracking-widest uppercase font-bold">
          {kairosInsight?.severity.toUpperCase() || "NORMAL"}
        </span>
      </div>

      <div className="space-y-8">
        {kairosInsight ? (
          <div className="animate-in fade-in slide-in-from-bottom-4 duration-1000">
            <p className="font-cormorant text-4xl md:text-5xl leading-tight">
              {kairosInsight.message}
            </p>
            {kairosInsight.supportingSignals?.map((signal, idx) => (
              <p
                key={idx}
                className="mt-8 text-black/50 text-lg font-light italic leading-relaxed pl-8 border-l
border-black/10"
              >
                {signal}
              </p>
            ))}
          </div>
        ) : (
          <div className="space-y-4 opacity-10">
            <div className="h-4 bg-black rounded w-full" />
            <div className="h-4 bg-black rounded w-3/4" />
          </div>
        )}
      </div>
    </div>
  </div>

  <div className="lg:col-span-4 space-y-12">
    <h3 className="font-cormorant italic text-3xl text-white/60">
      Capital Allocation
    </h3>
    <div className="space-y-8">
      {ledger.analytics &&
        Object.entries(ledger.analytics.categoryBreakdown)
          .sort(([, a], [, b]) => b - a)
          .map(([cat, amt]) => {
            const percentage =
              (amt / ledger.analytics!.totalDeployed) * 100;
```

Repository Audit

```
return (
  <div key={cat} className="space-y-4 group cursor-default">
    <div className="flex justify-between items-end">
      <span className="text-[10px] font-mono tracking-widest text-white/40 uppercase">
        {DeploymentMap[cat as keyof typeof DeploymentMap] ||
          cat}
      </span>
      <span className="font-cormorant text-xl text-white group-hover:text-white
transition-colors">
        {Math.round(percentage)}%
      </span>
    </div>
    <div className="h-[1px] w-full bg-white/5 overflow-hidden">
      <div
        className="h-full bg-white/60 transition-all duration-1000"
        style={{ width: `${percentage}%` }}
      />
    </div>
  </div>
);
}}
</div>
</div>
</div>
</section>

{/* Zone 4 ? LEDGER */}
<section id="ledger" className="space-y-16">
  <div className="flex items-center gap-6">
    <span className="font-cormorant italic text-3xl text-white/20">
      IV.
    </span>
    <h2 className="font-cormorant text-4xl text-white tracking-wide">
      Capital Ledger
    </h2>
    <div className="flex-1 h-[1px] bg-white/5" />
  </div>

  <div className="space-y-4">
    {ledger.deployments.map((deployment) => (
      <div
        key={deployment.id}
        className="group flex flex-col md:flex-row md:items-center justify-between p-8 border-b
border-white/5 hover:bg-white/[0.01] transition-all"
      >
        <div className="space-y-1">
          <h3 className="font-cormorant text-2xl text-white transition-transform
group-hover:translate-x-2">
            {deployment.title}
          </h3>
          <p className="text-[9px] font-mono tracking-[0.4em] text-white/20 uppercase">
            {DeploymentMap[
              deployment.category as keyof typeof DeploymentMap
            ] || deployment.category}
          </p>
        </div>
        <div className="text-left md:text-right mt-4 md:mt-0">
          <p className="font-cormorant text-2xl text-white">
            {formatCurrency(deployment.amount)}
          </p>
          <p className="text-[9px] font-mono text-white/10 uppercase tracking-widest">
            {new Date(deployment.created_at).toLocaleDateString()}
          </p>
        </div>
      </div>
    ))}
  </div>
</section>
```

Repository Audit

```
        </div>
    )})
</div>
</section>

    { /* OBSERVABILITY */ }
    <section className="pt-32 opacity-20">
        <TelemetryDashboard />
    </section>
</div>
);
}
```

File: src/app/globals.css

```
@import "tailwindcss";

@theme {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
  --font-cormorant: var(--font-cormorant);

  --radius-none: 0px;
  --radius-xs: 0.125rem;
  --radius-default: 0.375rem;
  --radius-2xl: 1.5rem;
  --radius-3xl: 1.875rem;
  --radius-full: 9999px;
  --color-button-ripple-color: var(--button-ripple-color);
}

:root {
  --background: #000000;
  --foreground: #ffffff;
  --button-ripple-color: oklch(0.145 0 0 / 0.3);
}

.dark {
  --button-ripple-color: oklch(0.985 0 0 / 0.5);
}

body {
  background: var(--background);
  color: var(--foreground);
  font-family: var(--font-sans), ui-sans-serif, system-ui, sans-serif;
  -webkit-font-smoothing: antialiased;
}

/* Custom Scrollbar */
::-webkit-scrollbar {
  width: 4px;
}

::-webkit-scrollbar-track {
  background: transparent;
}

::-webkit-scrollbar-thumb {
  background: rgba(255, 255, 255, 0.1);
}
```

Repository Audit

```
::-webkit-scrollbar-thumb:hover {
  background: rgba(255, 255, 255, 0.2);
}

/* Luxury Transitions */
* {
  transition-timing-function: cubic-bezier(0.4, 0, 0.2, 1);
}

@keyframes luxury-fade-in {
  from {
    opacity: 0;
    transform: translateY(10px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.animate-luxury {
  animation: luxury-fade-in 1.2s ease-out forwards;
}

/* Hide range inputs default look for luxury feel */
input[type="range"] {
  -webkit-appearance: none;
  appearance: none;
  background: transparent;
  cursor: pointer;
  height: 32px; /* Increase hit area */
}

input[type="range"]::-webkit-slider-runnable-track {
  background: rgba(255, 255, 255, 0.1);
  height: 2px;
  border-radius: 9999px; /* Round track */
}

input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  appearance: none;
  margin-top: -11px; /* Center thumb on 2px track */
  background-color: #ffffff;
  height: 24px;
  width: 24px;
  border-radius: 9999px; /* Round thumb */
  box-shadow: 0 0 20px rgba(255, 255, 255, 0.2);
  transition:
    transform 0.2s ease,
    box-shadow 0.2s ease;
}

input[type="range"]:active::-webkit-slider-thumb {
  transform: scale(1.1);
  box-shadow: 0 0 30px rgba(255, 255, 255, 0.4);
}
```

File: src/app/layout.tsx

```
import type { Metadata } from "next";
import { Geist, Geist_Mono, Cormorant_Garamond } from "next/font/google";
import "./globals.css";
```

Repository Audit

```
const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

const cormorant = Cormorant_Garamond({
  variable: "--font-cormorant",
  subsets: ["latin"],
  weight: ["300", "400", "500", "600", "700"],
  style: ["normal", "italic"],
});

export const metadata: Metadata = {
  title: "AXIOM // Strategic Wealth Intelligence",
  description: "Bespoke Liquidity Architecture & Structural Solvency Engine",
};

import { SmoothScrollProvider } from "@components/ui/smooth-scroll-provider";

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html
      lang="en"
      className={`${geistSans.variable} ${geistMono.variable} ${cormorant.variable} h-full antialiased`}
    >
      <body className="min-h-full flex flex-col bg-background text-foreground selection:bg-foreground selection:text-background">
        <SmoothScrollProvider>{children}</SmoothScrollProvider>
      </body>
    </html>
  );
}
```

File: src/app/page.tsx

```
"use client";

import { useEffect, useState } from "react";
import { useRouter } from "next/navigation";
import Link from "next/link";
import { supabase } from "@lib/supabase";
import { LuxuryHero } from "@components/landing/LuxuryHero";
import { SolvencyCalculator } from "@components/landing/SolvencyCalculator";
import { StrategicPillars } from "@components/landing/StrategicPillars";
import { ProcessFlow } from "@components/landing/ProcessFlow";

import { RippleButton } from "@components/ui/multi-type-ripple-buttons";
import { HoverButton } from "@components/ui/hover-glow-button";
import { BrandMark } from "@components/ui/brand-mark";

export default function LandingPage() {
  const router = useRouter();
  const [isScrolled, setIsScrolled] = useState(false);
```

Repository Audit

```
useEffect(() => {
  const handleScroll = () => {
    setIsScrolled(window.scrollY > 20);
  };

  window.addEventListener("scroll", handleScroll);

  const {
    data: { subscription },
  } = supabase.auth.onAuthStateChange((event, session) => {
    if (session) {
      router.push("/dashboard");
    }
  });

  return () => {
    window.removeEventListener("scroll", handleScroll);
    subscription.unsubscribe();
  };
}, [router]);

return (
  <main className="bg-black text-white min-h-screen selection:bg-white selection:text-black overflow-x-hidden antialiased">
    { /* Editorial Navigation - Dynamic Glassmorphism */ }
    <nav
      className={`fixed top-0 left-0 w-full z-50 px-8 flex justify-between items-center transition-all duration-700 ${
        isScrolled
          ? "py-5 bg-black/40 backdrop-blur-xl border-b border-white/5 shadow-2xl"
          : "py-10 bg-transparent border-b border-transparent"
      }`}
    >
      <div className="flex items-center gap-4 group cursor-pointer">
        <BrandMark />
        <div className="flex flex-col">
          <span className="font-mono text-[10px] tracking-[0.6em] uppercase font-bold text-white">
            AXIOM
          </span>
          <span className="text-[7px] font-mono tracking-[0.3em] uppercase text-white/40">
            Sovereign Intel
          </span>
        </div>
      </div>
      <div className="flex gap-8 items-center">
        <Link href="/signup">
          <RippleButton
            variant="ghost"
            className="text-white/40 hover:text-white"
          >
            Access Portal
          </RippleButton>
        </Link>
        <Link href="/signup">
          <RippleButton
            variant="hoverborder"
            hoverBorderEffectColor="rgba(255, 255, 255, 0.4)"
          >
            Initialize
          </RippleButton>
        </Link>
      </div>
    </nav>
  </main>
);
```

Repository Audit

```
{/* ASSEMBLY */}
<LuxuryHero />
<SolvencyCalculator />
<StrategicPillars />
<ProcessFlow />

{/* FINAL CONVERSION SECTION */}
<section className="py-80 px-6 text-center bg-gradient-to-b from-black to-[#080808] relative">
  <div className="absolute inset-0
bg-[url('https://www.transparenttextures.com/patterns/carbon-fibre.png')] opacity-5 pointer-events-none" />

  <div className="max-w-5xl mx-auto space-y-24 relative z-10">
    <div className="space-y-8">
      <span className="font-mono text-[10px] tracking-[0.6em] text-white/20 uppercase">
        Ready to Secure your legacy?
      </span>
      <h2 className="font-cormorant text-7xl md:text-[10rem] text-white leading-[0.8] tracking-tighter">
        Establish your <br />
        <span className="italic">strategic baseline.</span>
      </h2>
    </div>

    <div className="flex flex-col items-center gap-16">
      <Link href="/signup">
        <HoverButton
          glowColor="rgba(255,255,255,0.4)"
          className="px-24 py-10 rounded-full"
        >
          Establish Your Architecture
        </HoverButton>
      </Link>

      <div className="flex flex-col items-center gap-8">
        <div className="h-40 w-[1px] bg-gradient-to-b from-white/20 to-transparent" />
        <p className="font-mono text-[9px] tracking-[0.6em] text-white/20 uppercase">
          Axiom Strategic Protocol // v1.0
        </p>
      </div>
    </div>
  </div>

  <div className="py-32 px-8 md:px-24 border-t border-white/5 bg-[#080808]">
    <div className="max-w-7xl mx-auto grid md:grid-cols-[1.5fr_1fr_1fr] gap-24 items-start">
      <div className="space-y-12">
        <div className="flex items-center gap-4">
          <BrandMark className="w-8 h-8" />
          <span className="font-mono text-xs tracking-[0.5em] uppercase font-bold">
            AXIOM
          </span>
        </div>
        <p className="text-white/20 text-lg font-light leading-relaxed max-w-sm">
          Deterministic financial intelligence for the sovereign individual.
          Precision modeling for a chaotic ecosystem.
        </p>
        <div className="pt-8 border-t border-white/5 flex gap-8 font-mono text-[9px] tracking-widest
text-white/10 uppercase">
          <span>Encrypted Session</span>
          <span>Regional Node: Nairobi</span>
        </div>
      </div>
    </div>
  </div>
</section>

{/* LUXURY FOOTER */}
<footer className="py-32 px-8 md:px-24 border-t border-white/5 bg-[#080808]">
  <div className="max-w-7xl mx-auto grid md:grid-cols-[1.5fr_1fr_1fr] gap-24 items-start">
    <div className="space-y-12">
      <div className="flex items-center gap-4">
        <BrandMark className="w-8 h-8" />
        <span className="font-mono text-xs tracking-[0.5em] uppercase font-bold">
          AXIOM
        </span>
      </div>
      <p className="text-white/20 text-lg font-light leading-relaxed max-w-sm">
        Deterministic financial intelligence for the sovereign individual.
        Precision modeling for a chaotic ecosystem.
      </p>
      <div className="pt-8 border-t border-white/5 flex gap-8 font-mono text-[9px] tracking-widest
text-white/10 uppercase">
        <span>Encrypted Session</span>
        <span>Regional Node: Nairobi</span>
      </div>
    </div>
  </div>
</footer>
```

Repository Audit

```
<div className="space-y-10">
  <h4 className="font-cormorant italic text-2xl text-white/60">
    Intelligence
  </h4>
  <div className="flex flex-col gap-6 font-mono text-[10px] tracking-[0.4em] uppercase
text-white/30">
    <Link href="#" className="hover:text-white transition-colors">
      Methodology
    </Link>
    <Link href="#" className="hover:text-white transition-colors">
      Taxonomy
    </Link>
    <Link href="#" className="hover:text-white transition-colors">
      Risk Protocol
    </Link>
    <Link href="#" className="hover:text-white transition-colors">
      Legacy Design
    </Link>
  </div>
</div>

<div className="space-y-10">
  <h4 className="font-cormorant italic text-2xl text-white/60">
    Legal
  </h4>
  <div className="flex flex-col gap-6 font-mono text-[10px] tracking-[0.4em] uppercase
text-white/30">
    <Link href="#" className="hover:text-white transition-colors">
      Privacy Policy
    </Link>
    <Link href="#" className="hover:text-white transition-colors">
      Terms of Service
    </Link>
    <Link href="#" className="hover:text-white transition-colors">
      Compliance Audit
    </Link>
    <div className="pt-12 text-white/10 italic normal-case font-serif text-sm">
      © 2026 Axiom Labs. All rights reserved.
    </div>
  </div>
</div>
</div>
</footer>
</main>
);
}
```

File: src/app/signup/page.tsx

```
"use client";

import { useEffect } from "react";
import { useRouter } from "next/navigation";
import { Auth } from "@supabase/auth-ui-react";
import { ThemeSupa } from "@supabase/auth-ui-shared";
import { supabase } from "@/lib/supabase";
import Link from "next/link";
import { BrandMark } from "@/components/ui/brand-mark";
import { ScrollReveal } from "@/components/ui/scroll-reveal";

export default function SignUp() {
  const router = useRouter();
```

Repository Audit

```
useEffect(() => {
  const {
    data: { subscription },
  } = supabase.auth.onAuthStateChange((event, session) => {
    if (session) {
      router.push("/dashboard");
    }
  });

  return () => {
    subscription.unsubscribe();
  };
}, [router]);

return (
  <main className="min-h-screen flex flex-col md:flex-row bg-black text-white relative overflow-hidden
selection:bg-white selection:text-black">
    {/* Left Side: Editorial Canvas - Tactical Intelligence */}
    <div className="hidden md:flex md:w-1/2 bg-[#020202] relative items-center justify-center border-r
border-white/5 overflow-hidden">
      {/* The Tactical Schematic Image */}
      <div
        className="absolute inset-0 z-0 opacity-40 mix-blend-screen scale-110 grayscale"
        style={{
          backgroundImage: "url('/images/tactical-schematic.png')",
          backgroundSize: "cover",
          backgroundPosition: "center",
        }}
      />

      {/* Depth & Focus Overlays */}
      <div className="absolute inset-0 z-0">
        <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-[90%] h-[90%]
bg-white/[0.01] rounded-full blur-[140px]" />
        <div className="absolute inset-0 bg-gradient-to-t from-black via-transparent to-black opacity-60" />
      </div>

      <div className="relative z-10 max-w-lg px-12 text-center">
        <ScrollReveal direction="up">
          <h2 className="font-cormorant italic text-5xl lg:text-7xl text-white mb-8 leading-tight">
            Strategic Wealth <br />
            <span className="not-italic font-medium text-white/80">
              Architecture.
            </span>
          </h2>
        </ScrollReveal>

        <ScrollReveal direction="up" delay={0.2}>
          <div className="h-[1px] w-12 bg-white/20 mx-auto mb-8" />
          <p className="text-white/40 font-mono text-[9px] tracking-[0.6em] uppercase">
            Axiom Strategic Protocol // v1.0
          </p>
        </ScrollReveal>
      </div>
    </div>

    {/* Right Side: Authentication Portal */}
    <div className="flex-1 flex items-center justify-center p-8 md:p-12 relative z-10 bg-black">
      <div className="w-full max-w-md">
        {/* Brand & Header */}
        <div className="flex flex-col items-center mb-16">
          <ScrollReveal direction="down" delay={0.3}>
            <Link href="/" className="mb-12 block">
              <BrandMark className="w-14 h-14" />
            </ScrollReveal>
          </div>
        </div>
      </div>
    </div>
  </main>
);
```

Repository Audit

```
</Link>
</ScrollReveal>

<ScrollReveal direction="up" delay={0.4}>
  <h1 className="font-cormorant text-4xl text-white text-center mb-3 tracking-wide">
    Initialize Access
  </h1>
  <p className="text-white/30 text-[10px] font-mono text-center tracking-[0.3em] uppercase">
    Enter Sovereign Credentials
  </p>
</ScrollReveal>
</div>

<ScrollReveal direction="up" delay={0.5}>
  <div className="bg-white/[0.02] border border-white/5 p-10 md:p-14 rounded-3xl backdrop-blur-sm
relative overflow-hidden group">
    {/* Subtle Decorative Glow */}
    <div className="absolute -top-24 -right-24 w-48 h-48 bg-white/5 blur-3xl rounded-full opacity-0
group-hover:opacity-100 transition-opacity duration-1000" />

    <Auth
      supabaseClient={supabase}
      view="sign_up"
      appearance={{
        theme: ThemeSupa,
        variables: {
          default: {
            colors: {
              brand: "white",
              brandAccent: "white",
              brandButtonText: "black",
              defaultButtonBackground: "transparent",
              defaultButtonBackgroundHover: "rgba(255,255,255,0.05)",
              defaultButtonText: "white",
              defaultButtonBorder: "rgba(255,255,255,0.1)",
              inputText: "white",
              inputBackground: "transparent",
              inputBorder: "rgba(255,255,255,0.1)",
              inputPlaceholder: "rgba(255,255,255,0.2)",
              inputLabelText: "rgba(255,255,255,0.4)",
            },
            radii: {
              buttonBorderRadius: "9999px",
              inputBorderRadius: "12px",
            },
            fonts: {
              bodyFontFamily: ` -apple-system, BlinkMacSystemFont, "SF Pro Display", "SF Pro Text",
"Helvetica Neue", Helvetica, Arial, sans-serif`,
              buttonFontFamily: `var(--font-geist-mono), ui-monospace, SFMono-Regular, Roboto Mono,
Menlo, Monaco, Consolas, Liberation Mono, Courier New, monospace`,
              inputFontFamily: ` -apple-system, BlinkMacSystemFont, "SF Pro Display", "SF Pro Text",
"Helvetica Neue", Helvetica, Arial, sans-serif`,
              labelFontFamily: `var(--font-geist-mono), ui-monospace, SFMono-Regular, Roboto Mono,
Menlo, Monaco, Consolas, Liberation Mono, Courier New, monospace`,
            },
          },
        },
        className: {
          button:
            "font-mono uppercase tracking-widest text-[10px] py-4 transition-all hover:scale-[1.02]
active:scale-[0.98]",
          label:
            "font-mono uppercase tracking-widest text-[8px] mb-3 ml-1",
          input:
```

Repository Audit

```
        "border-white/10 focus:border-white/40 transition-colors bg-white/5 py-3 px-4",
      },
    }}
    localization={{
      variables: {
        sign_up: {
          email_label: "EMAIL ADDRESS",
          password_label: "SECURE ACCESS KEY",
          button_label: "INITIALIZE PROTOCOL",
          link_text: "AUTHORIZED? ACCESS PORTAL",
        },
        sign_in: {
          email_label: "EMAIL ADDRESS",
          password_label: "SECURE ACCESS KEY",
          button_label: "AUTHORIZE SESSION",
          link_text: "NO ACCESS? INITIALIZE LEDGER",
        },
      },
    }}
    providers=[["google"]]
    redirectTo={`${typeof window !== "undefined" ? window.location.origin : ""}/auth/callback`}
  />

  <div className="mt-12 text-center pt-10 border-t border-white/5">
    <Link
      href="/"
      className="text-[9px] font-mono text-white/20 uppercase tracking-[0.4em] hover:text-white
transition-colors"
    >
      ? Return to Authorization
    </Link>
  </div>
</div>
</ScrollReveal>

  <ScrollReveal direction="up" delay={0.7}>
    <p className="mt-12 text-white/10 text-[8px] font-mono text-center uppercase tracking-[0.5em]
leading-relaxed">
      All sessions are encrypted. Authorization granted by <br />
      Axiom Strategic Wealth Intelligence.
    </p>
  </ScrollReveal>
</div>
</div>
</main>
);
}
```

File: src/components/dashboard/DayZeroOnboarding.tsx

```
"use client";

import { useState, useEffect } from "react";
import {
  submitDayZeroBaselineAction,
  type DashboardSnapshot,
} from "@app/dashboard/actions";
import { BrandMark } from "@components/ui/brand-mark";
import { RippleButton } from "@components/ui/multi-type-ripple-buttons";
import { HoverButton } from "@components/ui/hover-glow-button";
import { LuxuryCard } from "@components/ui/luxury-card";
import { motion, AnimatePresence } from "framer-motion";
```

Repository Audit

```
interface DayZeroOnboardingProps {
  onComplete: (snapshot: DashboardSnapshot) => void;
}

const ACCOUNT_TYPES = [
  { value: "mobile_money", label: "Mobile Money" },
  { value: "checking", label: "Checking" },
  { value: "savings", label: "Savings" },
  { value: "cash", label: "Cash" },
  { value: "crypto", label: "Crypto" },
];

const INCOME_TYPES = [
  { value: "salary", label: "Salary" },
  { value: "freelance", label: "Freelance" },
  { value: "business", label: "Business" },
  { value: "other", label: "Other" },
];

const LIABILITY_TYPES = [
  { value: "credit_card", label: "Credit Card" },
  { value: "mortgage", label: "Mortgage" },
  { value: "personal_loan", label: "Personal Loan" },
  { value: "student_loan", label: "Student Loan" },
  { value: "business_loan", label: "Business Loan" },
  { value: "line_of_credit", label: "Line of Credit" },
  { value: "other", label: "Other" },
];

const CADENCES = [
  { value: "weekly", label: "Weekly" },
  { value: "biweekly", label: "Bi-weekly" },
  { value: "monthly", label: "Monthly" },
  { value: "annually", label: "Annually" },
];

const BASELINE_CADENCES = [
  { value: "daily", label: "Daily" },
  { value: "weekly", label: "Weekly" },
  { value: "monthly", label: "Monthly" },
  { value: "yearly", label: "Yearly" },
];

const STEPS = [
  {
    roman: "I",
    title: "Repositories",
    desc: "Define your primary liquidity containers.",
  },
  {
    roman: "II",
    title: "Inflow Velocity",
    desc: "Map your yield-generating transaction streams.",
  },
  {
    roman: "III",
    title: "Maintenance",
    desc: "Determine your core structural cost of existence.",
  },
  {
    roman: "IV",
    title: "Obligations",
    desc: "Audit your outstanding liabilities.",
  },
];
```

Repository Audit

```
];

export default function DayZeroOnboarding({
  onComplete,
}: DayZeroOnboardingProps) {
  const [step, setStep] = useState(1);
  const [direction, setDirection] = useState(1);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [accounts, setAccounts] = useState([
    { account_name: "", account_type: "mobile_money", current_balance: "" },
  ]);
  const [incomes, setIncomes] = useState<any[]>([]);
  const [liabilities, setLiabilities] = useState<any[]>([]);
  const [baseline, setBaseline] = useState({ amount: "", cadence: "monthly" });

  const addAccount = () => {
    if (accounts.length < 3)
      setAccounts([
        ...accounts,
        { account_name: "", account_type: "checking", current_balance: "" },
      ]);
  };

  const removeAccount = (index: number) =>
    setAccounts(accounts.filter((_, i) => i !== index));

  const addIncome = () => {
    if (incomes.length < 3)
      setIncomes([
        ...incomes,
        {
          income_name: "",
          income_type: "salary",
          amount: "",
          cadence: "monthly",
        },
      ]);
  };

  const removeIncome = (index: number) =>
    setIncomes(incomes.filter((_, i) => i !== index));

  const addLiability = () => {
    if (liabilities.length < 3)
      setLiabilities([
        ...liabilities,
        {
          liability_name: "",
          liability_type: "credit_card",
          outstanding_balance: "",
          cadence: "monthly",
        },
      ]);
  };

  const removeLiability = (index: number) =>
    setLiabilities(liabilities.filter((_, i) => i !== index));

  const isStepValid = () => {
    if (step === 1)
      return (
        accounts.length > 0 &&
        accounts.every((a) => a.account_name && a.current_balance !== "")
      );
    if (step === 2)
```

Repository Audit

```
    return (
      incomes.length === 0 ||
      incomes.every((i) => i.income_name && i.amount !== "")
    );
    if (step === 3) return baseline.amount !== "";
    if (step === 4)
      return (
        liabilities.length === 0 ||
        liabilities.every(
          (l) => l.liability_name && l.outstanding_balance !== "",
        )
      );
    return false;
  };

const handleNext = () => {
  if (step < 4) {
    setDirection(1);
    setStep(step + 1);
  }
};

const handlePrev = () => {
  if (step > 1) {
    setDirection(-1);
    setStep(step - 1);
  }
};

async function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  if (step < 4) {
    handleNext();
    return;
  }
  setIsLoading(true);
  setError(null);
  try {
    await submitDayZeroBaselineAction({
      accounts: accounts.map((a) => ({
        ...a,
        current_balance: Number(a.current_balance),
      })),
      incomes: incomes.map((i) => ({ ...i, amount: Number(i.amount) })),
      liabilities: liabilities
        .filter((l) => Number(l.outstanding_balance) > 0)
        .map((l) => ({
          ...l,
          outstanding_balance: Number(l.outstanding_balance),
          // Actions.ts needs cadence if we want to store it, but for baseline setup we usually focus on
current_balance.
          // I will pass the object as is.
        })),
      baseline: {
        amount: Number(baseline.amount),
        cadence: baseline.cadence,
      },
    });
    window.location.reload();
  } catch (err: unknown) {
    setError(err instanceof Error ? err.message : "Configuration failed.");
  } finally {
    setIsLoading(false);
  }
}
```

Repository Audit

```
const activeStep = STEPS[step - 1];
const backgroundX = -(step - 1) * 5;

return (
  <div className="fixed inset-0 bg-black z-[100] flex flex-col selection:bg-white selection:text-black
overflow-hidden h-screen w-full font-sans">
    /* Background Atmosphere */
    <div className="absolute inset-0 z-0 pointer-events-none overflow-hidden">
      <motion.div
        animate={{ x: `${backgroundX}%` }}
        transition={{ duration: 1.2, ease: [0.215, 0.61, 0.355, 1] }}
        className="absolute inset-0 w-[150%] h-full"
      >
        <div className="absolute top-[-10%] left-[-10%] w-[40%] h-[40%] bg-white/[0.02] rounded-full
blur-[120px]" />
        <div className="absolute bottom-[-10%] right-[30%] w-[40%] h-[40%] bg-white/[0.02] rounded-full
blur-[120px]" />
        <div
          className="absolute inset-0 opacity-15 mix-blend-screen grayscale"
          style={{
            backgroundImage: "url('/images/tactical-schematic.png')",
            backgroundSize: "cover",
            backgroundPosition: "left center",
          }}
        />
      </motion.div>
    </div>

    /* Header - Absolute position */
    <nav className="absolute top-0 left-0 w-full z-50 px-12 py-10 flex justify-between items-center
mix-blend-difference pointer-events-none">
      <div className="flex items-center gap-6 group cursor-pointer pointer-events-auto">
        <BrandMark className="w-10 h-10" />
        <div className="flex flex-col">
          <span className="font-mono text-[10px] tracking-[0.6em] uppercase font-bold text-white">
            AXIOM
          </span>
          <span className="text-[8px] font-mono tracking-[0.4em] uppercase text-white/40">
            Architecture
          </span>
        </div>
      </div>
      <div className="flex gap-12 items-center pointer-events-auto">
        <div className="hidden md:flex gap-8">
          {STEPS.map((s, idx) => (
            <div
              key={s.roman}
              className="flex flex-col items-center gap-2 text-white"
            >
              <span
                className={`font-mono text-[9px] tracking-widest transition-colors duration-700 ${idx + 1 ===
step ? "opacity-100" : "opacity-10"}`}>
                >
                  {s.roman}
                </span>
                <motion.div
                  animate={{ scaleX: idx + 1 === step ? 1 : 0 }}
                  className="h-[1px] w-4 bg-white origin-left"
                />
              </div>
            ))}
        </div>
        <span className="font-mono text-[9px] tracking-[0.4em] uppercase text-white/20">
```

Repository Audit

```
        v1.0
      </span>
    </div>
  </nav>

  { /* Main Canvas */ }
  <div className="relative z-10 flex-1 flex items-center justify-center px-8 md:px-12 pt-16">
    <div className="max-w-7xl w-full h-[620px] grid lg:grid-cols-[0.7fr_1.3fr] gap-16 md:gap-24
items-stretch">
      { /* Info Side */ }
      <div className="flex flex-col justify-center h-full border-r border-white/5 pr-12">
        <AnimatePresence mode="wait" custom={direction}>
          <motion.div
            key={step}
            custom={direction}
            initial={{ opacity: 0, x: direction * 50 }}
            animate={{ opacity: 1, x: 0 }}
            exit={{ opacity: 0, x: -direction * 50 }}
            transition={{ duration: 0.8, ease: [0.215, 0.61, 0.355, 1] }}
            className="space-y-6"
          >
            <div className="space-y-2">
              <div className="font-cormorant italic text-8xl text-white/5 leading-none select-none">
                {activeStep.roman}
              </div>
              <h1 className="font-cormorant text-6xl text-white leading-none tracking-tight">
                {activeStep.title}
              </h1>
            </div>
            <p className="text-white/40 text-lg font-light leading-relaxed">
              {activeStep.desc}
            </p>
          </motion.div>
        </AnimatePresence>
      </div>

      { /* Form Area */ }
      <LuxuryCard className="h-full flex flex-col border border-white/10 shadow-[0_0_80px_rgba(0,0,0,0.8)]
overflow-hidden">
        <div className="flex-1 p-12 md:p-16 overflow-hidden relative rounded-t-[inherit]">
          <form onSubmit={handleSubmit} className="h-full flex flex-col">
            <AnimatePresence mode="wait" custom={direction}>
              <motion.div
                key={step}
                custom={direction}
                initial={{ opacity: 0, x: direction * 30 }}
                animate={{ opacity: 1, x: 0 }}
                exit={{ opacity: 0, x: -direction * 30 }}
                transition={{ duration: 0.6, ease: "easeOut" }}
                className="flex-1 flex flex-col"
              >
                {step === 1 && (
                  <div className="flex flex-col h-full gap-6">
                    <div className="flex-1 space-y-8">
                      {accounts.map((acc, idx) => (
                        <div
                          key={idx}
                          className="space-y-4 pb-4 border-b border-white/5 relative shrink-0 group/entry"
                        >
                          <input
                            type="text"
                            placeholder="Repository Name"
                            value={acc.account_name}
                            onChange={(e) => {
```

Repository Audit

```
const n = [...accounts];
n[idx].account_name = e.target.value;
setAccounts(n);
}}
className="w-full bg-transparent border-b border-white/10 py-2 font-cormorant
text-4xl text-white focus:outline-none focus:border-white transition-all placeholder:text-white/5
tracking-tight"

/>
<div className="grid grid-cols-2 gap-8">
  <div className="space-y-1">
    <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">
      Type
    </label>
    <select
      value={acc.account_type}
      onChange={(e) => {
        const n = [...accounts];
        n[idx].account_type = e.target.value;
        setAccounts(n);
      }}
      className="w-full bg-transparent border-b border-white/10 py-1 text-[11px]
font-mono tracking-widest uppercase text-white/60 focus:outline-none"
    >
      {ACCOUNT_TYPES.map((t) => (
        <option
          key={t.value}
          value={t.value}
          className="bg-[#080808]"
        >
          {t.label}
        </option>
      ))}
    </select>
  </div>
  <div className="space-y-1">
    <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">
      KES
    </label>
    <input
      type="number"
      placeholder="0.00"
      value={acc.current_balance}
      onChange={(e) => {
        const n = [...accounts];
        n[idx].current_balance = e.target.value;
        setAccounts(n);
      }}
      className="w-full bg-transparent border-b border-white/10 py-1 text-2xl
font-light text-white focus:outline-none focus:border-white transition-all tabular-nums"
    />
  </div>
</div>
{accounts.length > 1 && (
  <button
    type="button"
    onClick={() => removeAccount(idx)}
    className="absolute top-0 right-0 p-4 text-white/10 hover:text-red-400
transition-colors bg-white/5 rounded-full hover:bg-white/10"
  >
    ?
  </button>
)}
}}
```

Repository Audit

```
        </div>
      )}}
    </div>
    {accounts.length < 3 && (
      <button
        type="button"
        onClick={addAccount}
        className="text-[11px] font-mono tracking-[0.6em] uppercase text-white/40
hover:text-white transition-all py-4 px-8 border border-white/10 rounded-full hover:bg-white/5 self-start
active:scale-95"
      >
        + Append Repository
      </button>
    )}
  </div>
)}

{step === 2 && (
  <div className="flex flex-col h-full gap-6">
    <div className="flex-1 space-y-8">
      {incomes.map((inc, idx) => (
        <div
          key={idx}
          className="space-y-4 pb-4 border-b border-white/5 relative shrink-0 group/entry"
        >
          <input
            type="text"
            placeholder="Inflow Source"
            value={inc.income_name}
            onChange={(e) => {
              const n = [...incomes];
              n[idx].income_name = e.target.value;
              setIncomes(n);
            }}
            className="w-full bg-transparent border-b border-white/10 py-2 font-cormorant
text-4xl text-white focus:outline-none focus:border-white transition-all placeholder:text-white/5
tracking-tight"
          />
          <div className="grid grid-cols-3 gap-8">
            <div className="space-y-1">
              <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">
                Type
              </label>
              <select
                value={inc.income_type}
                onChange={(e) => {
                  const n = [...incomes];
                  n[idx].income_type = e.target.value;
                  setIncomes(n);
                }}
                className="w-full bg-transparent border-b border-white/10 py-1 text-[11px]
font-mono tracking-widest uppercase text-white/60 focus:outline-none"
              >
                {INCOME_TYPES.map((t) => (
                  <option
                    key={t.value}
                    value={t.value}
                    className="bg-[#080808]"
                  >
                    {t.label}
                  </option>
                ))}
              </select>
            )}
          </div>
        </div>
      )}
    </div>
  )}
}
```

Repository Audit

```
</div>
<div className="space-y-1">
  <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">
    Cadence
  </label>
  <select
    value={inc.cadence}
    onChange={(e) => {
      const n = [...incomes];
      n[idx].cadence = e.target.value;
      setIncomes(n);
    }}
    className="w-full bg-transparent border-b border-white/10 py-1 text-[11px]
font-mono tracking-widest uppercase text-white/60 focus:outline-none"
  >
    {CADENCES.map((c) => (
      <option
        key={c.value}
        value={c.value}
        className="bg-[#080808]"
      >
        {c.label}
      </option>
    ))}
  </select>
</div>
<div className="space-y-1">
  <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">
    KES
  </label>
  <input
    type="number"
    placeholder="0.00"
    value={inc.amount}
    onChange={(e) => {
      const n = [...incomes];
      n[idx].amount = e.target.value;
      setIncomes(n);
    }}
    className="w-full bg-transparent border-b border-white/10 py-1 text-2xl
font-light text-white focus:outline-none focus:border-white transition-all tabular-nums"
  />
</div>
<button
  type="button"
  onClick={() => removeIncome(idx)}
  className="absolute top-0 right-0 p-4 text-white/10 hover:text-red-400
transition-colors bg-white/5 rounded-full hover:bg-white/10"
>
  ?
</button>
</div>
)}}
</div>
{incomes.length < 3 && (
  <button
    type="button"
    onClick={addIncome}
    className="text-[11px] font-mono tracking-[0.6em] uppercase text-white/40
hover:text-white transition-all py-4 px-8 border border-white/10 rounded-full hover:bg-white/5 self-start
active:scale-95"
```

Repository Audit

```
    >
    + Append Inflow
  </button>
  })
</div>
})

{step === 3 && (
  <div className="flex flex-col h-full justify-center space-y-12 text-center">
    <div className="space-y-4">
      <label className="text-[10px] font-mono tracking-[0.8em] uppercase text-white/20">
        Maintenance Load (KES)
      </label>
      <input
        autoFocus
        type="number"
        placeholder="0.00"
        value={baseline.amount}
        onChange={(e) =>
          setBaseline({
            ...baseline,
            amount: e.target.value,
          })
        }
        className="w-full bg-transparent border-b-2 border-white/10 py-4 text-[8rem]
font-cormorant text-white focus:outline-none focus:border-white transition-all placeholder:text-white/5
tabular-nums leading-none tracking-tighter"
      />
    </div>
    <div className="flex flex-wrap justify-center gap-4">
      {BASELINE_CADENCES.map((c) => (
        <button
          key={c.value}
          type="button"
          onClick={() =>
            setBaseline({ ...baseline, cadence: c.value })
          }
          className={`px-10 py-4 border rounded-full transition-all duration-700
text-[10px] font-mono tracking-[0.6em] uppercase ${baseline.cadence === c.value ? "bg-white text-black
border-white shadow-[0_0_40px_rgba(255,255,255,0.1)] scale-105" : "border-white/5 text-white/20
hover:border-white/40 hover:text-white"}`}
        >
          {c.label}
        </button>
      ))}
    </div>
  </div>
)}

{step === 4 && (
  <div className="flex flex-col h-full gap-6">
    <div className="flex-1 space-y-8">
      {liabilities.map((liab, idx) => (
        <div
          key={idx}
          className="space-y-4 pb-4 border-b border-white/5 relative shrink-0 group/entry"
        >
          <input
            type="text"
            placeholder="Obligation Name"
            value={liab.liability_name}
            onChange={(e) => {
              const n = [...liabilities];
              n[idx].liability_name = e.target.value;
            }}
          />
        </div>
      ))}
    </div>
  </div>
)}
```

Repository Audit

```
        setLiabilities(n);
      }}
      className="w-full bg-transparent border-b border-white/10 py-2 font-cormorant
text-4xl text-white focus:outline-none focus:border-white transition-all placeholder:text-white/5
tracking-tight"
    />
    <div className="grid grid-cols-3 gap-8">
      <div className="space-y-1">
        <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">
          Type
        </label>
        <select
          value={liab.liability_type}
          onChange={(e) => {
            const n = [...liabilities];
            n[idx].liability_type = e.target.value;
            setLiabilities(n);
          }}
          className="w-full bg-transparent border-b border-white/10 py-1 text-[11px]
font-mono tracking-widest uppercase text-white/60 focus:outline-none"
        >
          {LIABILITY_TYPES.map((t) => (
            <option
              key={t.value}
              value={t.value}
              className="bg-[#080808]"
            >
              {t.label}
            </option>
          ))}
        </select>
      </div>
      <div className="space-y-1">
        <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">
          Cadence
        </label>
        <select
          value={liab.cadence}
          onChange={(e) => {
            const n = [...liabilities];
            n[idx].cadence = e.target.value;
            setLiabilities(n);
          }}
          className="w-full bg-transparent border-b border-white/10 py-1 text-[11px]
font-mono tracking-widest uppercase text-white/60 focus:outline-none"
        >
          {CADENCES.map((c) => (
            <option
              key={c.value}
              value={c.value}
              className="bg-[#080808]"
            >
              {c.label}
            </option>
          ))}
        </select>
      </div>
      <div className="space-y-1">
        <label className="text-[9px] font-mono text-white/20 uppercase
tracking-[0.4em]">
          KES
        </label>
```

Repository Audit

```

        <input
          type="number"
          placeholder="0.00"
          value={liab.outstanding_balance}
          onChange={(e) => {
            const n = [...liabilities];
            n[idx].outstanding_balance =
              e.target.value;
            setLiabilities(n);
          }}
          className="w-full bg-transparent border-b border-white/10 py-1 text-2xl
font-light text-white focus:outline-none focus:border-white transition-all tabular-nums"
        />
      </div>
    </div>
    <button
      type="button"
      onClick={() => removeLiability(idx)}
      className="absolute top-0 right-0 p-4 text-white/10 hover:text-red-400
transition-colors bg-white/5 rounded-full hover:bg-white/10"
    >
      ?
    </button>
  </div>
)}}
</div>
{liabilities.length < 3 && (
  <button
    type="button"
    onClick={addLiability}
    className="text-[11px] font-mono tracking-[0.6em] uppercase text-white/40
hover:text-white transition-all py-4 px-8 border border-white/10 rounded-full hover:bg-white/5 self-start
active:scale-95"
  >
    + Append Obligation
  </button>
)}
</div>
)}
</motion.div>
</AnimatePresence>
</form>
</div>

  <div className="h-32 border-t border-white/5 relative z-10 px-14 flex items-center shrink-0
bg-[#080808]/40 backdrop-blur-xl rounded-b-[inherit]">
    <div className="w-full flex items-center justify-between">
      <button
        type="button"
        onClick={handlePrev}
        disabled={step === 1}
        className="text-[11px] font-mono tracking-[0.6em] text-white/20 hover:text-white
transition-all uppercase disabled:opacity-0 py-8 px-12 -ml-12 hover:bg-white/5 rounded-full"
      >
        ? Back
      </button>
      {error && (
        <div className="px-6 py-2 bg-red-500/10 border border-red-500/20 rounded-full text-[8px]
font-mono tracking-widest text-red-400 uppercase max-w-[200px] truncate">
          {error}
        </div>
      )}
    </div>
    <div onClick={handleSubmit}>
      {step === 4 ? (
```

Repository Audit

```
        <HoverButton
          glowColor="rgba(255,255,255,0.4)"
          className="px-24 py-10 rounded-full text-[11px]"
          disabled={isLoading || !isStepValid()}
        >
          {isLoading ? "... " : "Archive Setup"}
        </HoverButton>
      ) : (
        <RippleButton
          variant="hoverborder"
          className="px-24 py-10 border-none text-[11px]"
          disabled={isLoading || !isStepValid()}
        >
          Next Phase ?
        </RippleButton>
      )}
    </div>
  </div>
</div>
</LuxuryCard>
</div>
</div>
</div>
</div>
);
}
```

File: src/components/dashboard/TelemetryDashboard.tsx

```
"use client";

import { useEffect, useState, useCallback } from "react";
import {
  fetchTelemetryLogsAction,
  type TelemetrySummary,
} from "@app/dashboard/actions";

export function TelemetryDashboard() {
  const [telemetry, setTelemetry] = useState<TelemetrySummary | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  const loadTelemetry = useCallback(async (isManualRefresh = false) => {
    if (isManualRefresh) setIsLoading(true);
    setError(null);
    try {
      const summary = await fetchTelemetryLogsAction();
      setTelemetry(summary);
    } catch (err) {
      setError("Failed to retrieve forensic logs.");
      console.error(err);
    } finally {
      setIsLoading(false);
    }
  }, []);

  useEffect(() => {
    loadTelemetry();
  }, [loadTelemetry]);

  if (isLoading) {
    return (
      <div className="py-12 space-y-12 animate-pulse opacity-20">
        <div className="h-[1px] bg-white/20 w-full" />
      </div>
    );
  }
}
```

Repository Audit

```
<div className="grid grid-cols-4 gap-8">
  {[1, 2, 3, 4].map((i) => (
    <div key={i} className="h-24 bg-white/10" />
  ))}
</div>
</div>
);
}

if (error) {
  return (
    <div className="py-12 text-center opacity-40">
      <p className="text-[10px] font-mono uppercase tracking-[0.5em]">
        {error}
      </p>
    </div>
  );
}

return (
  <div className="space-y-24">
    { /* PERFORMANCE SUMMARY */ }
    <div className="grid grid-cols-1 md:grid-cols-4 gap-px bg-white/5">
      [[
        { label: "Evaluation Cycles", value: telemetry?.totalCycles || 0 },
        {
          label: "Avg. Latency",
          value: `${telemetry?.averageLatency || 0}ms`,
        },
        {
          label: "Intelligence Yield",
          value: `${telemetry?.matchRate || 0}%`,
        },
        {
          label: "Active Heuristics",
          value: Object.keys(telemetry?.ruleHits || {}).length,
        },
      ]
      .map((stat) => (
        <div key={stat.label} className="bg-black p-8 group">
          <span className="text-[8px] font-mono text-white/20 uppercase tracking-[0.3em] block mb-4 group-hover:text-white/40 transition-colors">
            {stat.label}
          </span>
          <span className="font-cormorant text-3xl text-white">
            {stat.value}
          </span>
        </div>
      )))
    </div>

    { /* FORENSIC LOG TABLE */ }
    <div className="space-y-8">
      <div className="flex items-center justify-between border-b border-white/5 pb-4">
        <h3 className="font-cormorant italic text-xl text-white/40">
          Evaluation Audit Trail
        </h3>
        <button
          onClick={() => loadTelemetry(true)}
          className="text-[8px] font-mono text-white/20 hover:text-white uppercase tracking-widest transition-colors"
        >
          Refresh Logs
        </button>
      </div>
    </div>
  )
);
```

Repository Audit

```
<div className="overflow-x-auto">
  <table className="w-full text-left">
    <thead>
      <tr className="border-b border-white/5">
        <th className="py-4 text-[9px] font-mono text-white/20 uppercase tracking-widest font-normal">
          Timestamp
        </th>
        <th className="py-4 text-[9px] font-mono text-white/20 uppercase tracking-widest font-normal">
          Heuristic ID
        </th>
        <th className="py-4 text-[9px] font-mono text-white/20 uppercase tracking-widest font-normal">
          Severity
        </th>
        <th className="py-4 text-[9px] font-mono text-white/20 uppercase tracking-widest font-normal
text-right">
          Compute
        </th>
      </tr>
    </thead>
    <tbody className="divide-y divide-white/5">
      {telemetry?.logs.map((log) => (
        <tr
          key={log.id}
          className="group hover:bg-white/[0.01] transition-all"
        >
          <td className="py-4 text-[10px] font-mono text-white/40 tabular-nums">
            {new Date(log.created_at).toLocaleTimeString([], {
              hour12: false,
              hour: "2-digit",
              minute: "2-digit",
              second: "2-digit",
            })}
          </td>
          <td className="py-4 text-[11px] text-white/80 font-light group-hover:translate-x-1
transition-transform">
            {log.rule_id}
          </td>
          <td className="py-4">
            <span
              className={`text-[8px] font-mono uppercase tracking-widest ${
                log.severity === "critical"
                  ? "text-red-500"
                  : log.severity === "high"
                  ? "text-orange-500"
                  : "text-white/20"
              }`}
            >
              {log.severity}
            </span>
          </td>
          <td className="py-4 text-[10px] font-mono text-white/40 text-right tabular-nums">
            {log.evaluation_time_ms?.toFixed(2)}ms
          </td>
        </tr>
      ))}
      {(!telemetry || telemetry.logs.length === 0) && (
        <tr>
          <td
            colSpan={4}
            className="py-12 text-center text-[10px] font-mono text-white/10 uppercase tracking-widest"
          >
            No forensic data available.
          </td>
        </tr>
      )
    </tbody>
  </table>
</div>
```

Repository Audit

```
        </tr>
      })
    </tbody>
  </table>
</div>
</div>
</div>
);
}
```

File: src/components/landing/LandingTerminal.tsx

```
"use client";

import { useState, useEffect, useRef } from "react";

const LOGS = [
  { text: "> INITIALIZING KAIROS ENGINE...", type: "info" },
  { text: "> SYNCING OPERATIONAL BASELINE...", type: "info" },
  { text: "> ANALYZING TRANSACTION TAXONOMY...", type: "info" },
  { text: "> CALCULATING STRUCTURAL RUNWAY...", type: "info" },
  { text: "> [WARNING] INFLOW CONCENTRATION RISK DETECTED.", type: "warning" },
  { text: "> 85% OF YIELD RELIES ON A SINGLE FLOW.", type: "warning" },
  { text: "> STRUCTURAL SOLVENCY: OPTIMAL", type: "success" },
  { text: "> DEPLOYMENT STRATEGY: CALCULATED", type: "info" },
];

export function LandingTerminal() {
  const [visibleLogs, setVisibleLogs] = useState<number[]>([]);
  const [isTyping, setIsTyping] = useState(true);
  const containerRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    let timeoutId: ReturnType<typeof setTimeout>;

    const showNextLog = (index: number) => {
      if (index >= LOGS.length) {
        setIsTyping(false);
        // Reset after a delay to loop the animation
        timeoutId = setTimeout(() => {
          setVisibleLogs([]);
          setIsTyping(true);
          showNextLog(0);
        }, 5000);
        return;
      }

      timeoutId = setTimeout(
        () => {
          setVisibleLogs((prev) => [...prev, index]);
          showNextLog(index + 1);
        },
        index === 0 ? 500 : 1200,
      );
    };

    showNextLog(0);

    return () => clearTimeout(timeoutId);
  }, []);

  return (
    <div className="w-full max-w-2xl mx-auto font-mono text-sm">
```

Repository Audit

```
<div className="bg-[#0a0a0a] border border-white/10 rounded-lg overflow-hidden shadow-2xl shadow-white/5">
  /* Terminal Header */
  <div className="bg-white/5 px-4 py-2 border-b border-white/10 flex items-center justify-between">
    <div className="flex gap-1.5">
      <div className="w-2.5 h-2.5 rounded-full bg-white/10" />
      <div className="w-2.5 h-2.5 rounded-full bg-white/10" />
      <div className="w-2.5 h-2.5 rounded-full bg-white/10" />
    </div>
    <div className="text-[10px] uppercase tracking-widest text-white/30">
      Kairos // Audit Terminal
    </div>
  </div>

  /* Terminal Body */
  <div
    ref={containerRef}
    className="p-6 h-64 overflow-y-auto space-y-2 scrollbar-hide"
  >
    {visibleLogs.map((logIndex) => {
      const log = LOGS[logIndex];
      return (
        <div
          key={logIndex}
          className={`
            ${log.type === "warning" ? "text-orange-500" : ""}
            ${log.type === "success" ? "text-emerald-500" : ""}
            ${log.type === "info" ? "text-white/70" : ""}
            animate-in fade-in slide-in-from-left-2 duration-300
          `}
        >
          {log.text}
        </div>
      );
    })}
    {isTyping && (
      <div className="w-2 h-4 bg-white/40 animate-pulse inline-block align-middle ml-1" />
    )}
  </div>
</div>

/* Decorative footer for the terminal */
<div className="mt-4 flex justify-between items-center px-4 text-[10px] text-white/20 uppercase tracking-[0.2em]">
  <div>Status: Running Clinical Audit</div>
  <div>Memory: 0.124ms / 256MB</div>
</div>
</div>
);
}
```

File: src/components/landing/LuxuryHero.tsx

```
"use client";

import Link from "next/link";
import { useEffect, useState } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { HoverButton } from "@/components/ui/hover-glow-button";
import { RippleButton } from "@/components/ui/multi-type-ripple-buttons";

export function LuxuryHero() {
  const [isVisible, setIsVisible] = useState(false);
```

Repository Audit

```
useEffect(() => {
  const timer = setTimeout(() => setIsVisible(true), 100);
  return () => clearTimeout(timer);
}, []);

return (
  <section className="relative min-h-screen flex flex-col justify-center items-center px-6 text-center
overflow-hidden bg-black selection:bg-white selection:text-black">
    /* Dynamic Background Atmosphere */
    <div className="absolute inset-0 z-0">
      <motion.div
        initial={{ opacity: 0, scale: 0.8 }}
        animate={isVisible ? { opacity: 1, scale: 1.1 } : {}}
        transition={{ duration: 4, ease: "easeOut" }}
        className="absolute top-[-20%] left-[-10%] w-[60%] h-[60%] bg-white/[0.03] rounded-full blur-[180px]"
      />
      <motion.div
        initial={{ opacity: 0, scale: 0.8 }}
        animate={isVisible ? { opacity: 1, scale: 1.1 } : {}}
        transition={{ duration: 4, ease: "easeOut", delay: 0.7 }}
        className="absolute bottom-[-20%] right-[-10%] w-[60%] h-[60%] bg-white/[0.03] rounded-full
blur-[180px]"
      />
      <div
        className="absolute inset-0
bg-[url('https://www.transparenttextures.com/patterns/carbon-fibre.png')] opacity-5 pointer-events-none
mix-blend-overlay" />
    </div>

    <div className="relative z-10 max-w-6xl mx-auto pt-20">
      /* Editorial Subtitle */
      <motion.div
        initial={{ opacity: 0, y: 20 }}
        animate={isVisible ? { opacity: 1, y: 0 } : {}}
        transition={{ duration: 1, delay: 0.3 }}
        className="inline-flex items-center gap-6 mb-12"
      >
        <div className="h-[1px] w-16 bg-gradient-to-r from-transparent via-white/20 to-transparent" />
        <span className="font-mono text-[9px] tracking-[0.8em] text-white/30 uppercase">
          Axiom Strategic Wealth Architecture
        </span>
        <div className="h-[1px] w-16 bg-gradient-to-r from-transparent via-white/40 to-transparent" />
      </motion.div>

      /* Main Display Headline */
      <h1 className="font-cormorant italic text-7xl md:text-[10rem] lg:text-[12rem] text-white leading-[0.8]
tracking-tighter mb-12 overflow-hidden">
        <motion.span
          initial={{ y: "100%" }}
          animate={isVisible ? { y: 0 } : {}}
          transition={{
            duration: 1.2,
            delay: 0.5,
            ease: [0.215, 0.61, 0.355, 1],
          }}
          className="block"
        >
          Financial Progress,
        </motion.span>
        <motion.span
          initial={{ y: "100%" }}
          animate={isVisible ? { y: 0 } : {}}
          transition={{
            duration: 1.2,
```

Repository Audit

```
        delay: 0.7,
        ease: [0.215, 0.61, 0.355, 1],
      }}
      className="block not-italic font-medium text-white/90"
    >
      For Your Future.
    </motion.span>
  </h1>

  { /* Narrative Body */ }
  <motion.p
    initial={{ opacity: 0 }}
    animate={isVisible ? { opacity: 1 } : {}}
    transition={{ duration: 1.5, delay: 1.2 }}
    className="max-w-2xl mx-auto text-white/40 text-xl md:text-2xl font-light leading-relaxed mb-16"
  >
    Beyond passive tracking. Axiom deploys deterministic intelligence to
    secure your structural solvency and amplify liquidity in the Kenyan
    ecosystem.
  </motion.p>

  { /* Elegant Action */ }
  <motion.div
    initial={{ opacity: 0, y: 20 }}
    animate={isVisible ? { opacity: 1, y: 0 } : {}}
    transition={{ duration: 1, delay: 1.5 }}
    className="flex justify-center"
  >
    <Link href="/signup">
      <HoverButton
        glowColor="rgba(255,255,255,0.5)"
        className="px-24 py-8 rounded-full border border-white/20"
      >
        Initialize Access
      </HoverButton>
    </Link>
  </motion.div>
</div>

{ /* Luxury Scroll Anchor */ }
<motion.div
  initial={{ opacity: 0 }}
  animate={isVisible ? { opacity: 1 } : {}}
  transition={{ duration: 1, delay: 2 }}
  className="absolute bottom-12 left-1/2 -translate-x-1/2 flex flex-col items-center gap-6"
>
  <div className="w-[1px] h-24 bg-gradient-to-b from-white/40 via-white/10 to-transparent animate-pulse"
  />
</motion.div>
</section>
);
}
```

File: src/components/landing/ProcessFlow.tsx

```
"use client";

import { ScrollReveal } from "@components/ui/scroll-reveal";

const STEPS = [
  {
    roman: "I",
    title: "Initial Consultation",
```

Repository Audit

```
    desc: "We begin with a deep-dive audit of your current financial architecture, identifying hidden liabilities and leakage points.",
  },
  {
    roman: "II",
    title: "Taxonomy Deployment",
    desc: "Every transaction stream is mapped to our five gates. We build a deterministic model of your capital flow.",
  },
  {
    roman: "III",
    title: "Structural Optimization",
    desc: "We reorganize your assets and liabilities to maximize liquidity and secure your long-term structural solvency.",
  },
  {
    roman: "IV",
    title: "Continuous Intelligence",
    desc: "Kairos, our AI analyst, monitors your portfolio 24/7, providing clinical insights and suggesting high-alpha pivots.",
  },
];
```

```
export function ProcessFlow() {
  return (
    <section className="py-40 px-6 bg-[#050505] relative overflow-hidden selection:bg-white selection:text-black">
      /* Editorial Decorative Layer */
      <div className="absolute top-0 left-0 w-full h-[1px] bg-gradient-to-r from-transparent via-white/10 to-transparent" />

      <div className="max-w-7xl mx-auto">
        <div className="text-center space-y-8 mb-32">
          <ScrollReveal direction="up">
            <span className="font-mono text-[10px] tracking-[0.6em] text-white/20 uppercase">
              The Methodology
            </span>
          </ScrollReveal>
          <ScrollReveal direction="up" delay={0.2}>
            <h2 className="font-cormorant text-7xl md:text-9xl text-white">
              Operational <span className="italic">Process.</span>
            </h2>
          </ScrollReveal>
          <ScrollReveal direction="up" delay={0.4}>
            <div className="w-[1px] h-24 bg-gradient-to-b from-white/20 to-transparent mx-auto" />
          </ScrollReveal>
        </div>

        <div className="grid md:grid-cols-2 lg:grid-cols-4 gap-px bg-white/5 border border-white/5 rounded-3xl overflow-hidden">
          {STEPS.map((step, idx) => (
            <ScrollReveal
              key={step.roman}
              direction="up"
              delay={0.1 * idx}
              distance={20}
            >
              <div className="bg-[#050505] h-full p-12 md:p-16 space-y-12 group hover:bg-white transition-all duration-700 cursor-default">
                <div className="space-y-4">
                  <span className="font-cormorant italic text-6xl text-white/10 group-hover:text-black/10 transition-colors">
                    {step.roman}.
                  </span>
                </div>
              </div>
            </div>
          ))}
```

Repository Audit

```
      <h3 className="font-cormorant text-3xl text-white group-hover:text-black transition-colors
leading-tight">
        {step.title}
      </h3>
    </div>

    <div className="h-[1px] w-12 bg-white/20 group-hover:bg-black/20 group-hover:w-full
transition-all duration-700" />

    <p className="text-white/40 group-hover:text-black/60 text-lg font-light leading-relaxed
transition-colors">
      {step.desc}
    </p>
  </div>
</ScrollReveal>
)}}
</div>

{/* Final Manifest Quote */}
<div className="mt-40 text-center max-w-4xl mx-auto space-y-12">
  <ScrollReveal direction="up">
    <div className="h-[1px] w-24 bg-white/10 mx-auto" />
  </ScrollReveal>
  <ScrollReveal direction="up" delay={0.2}>
    <h3 className="font-cormorant text-4xl md:text-6xl text-white italic leading-snug">
      &quot;Sovereignty is the ultimate asset. Precision is the ultimate
      weapon.&quot;
    </h3>
  </ScrollReveal>
  <ScrollReveal direction="up" delay={0.4}>
    <p className="text-white/20 font-mono text-[10px] tracking-[0.5em] uppercase">
      Axiom Strategic Wealth Intelligence
    </p>
  </ScrollReveal>
</div>
</div>
</section>
);
}
```

File: src/components/landing/RunwaySimulator.tsx

```
"use client";

import { useState, useMemo } from "react";

export function RunwaySimulator() {
  const [liquidity, setLiquidity] = useState(50000);
  const [monthlyBurn, setMonthlyBurn] = useState(30000);

  const runwayResult = useMemo(() => {
    if (monthlyBurn <= 0) return { months: 99, days: 0, isInfinite: true };

    const totalMonths = liquidity / monthlyBurn;
    const months = Math.floor(totalMonths);
    const days = Math.round((totalMonths - months) * 30);

    return { months, days, isInfinite: false };
  }, [liquidity, monthlyBurn]);

  const getVerdict = () => {
    const totalMonths = runwayResult.isInfinite
      ? 99
```

Repository Audit

```
      : runwayResult.months + runwayResult.days / 30;
    if (totalMonths < 3) {
      return {
        label: "CRITICAL",
        text: "Severe solvency risk. Capital depletion imminent.",
        color: "text-orange-500",
      };
    }
    if (totalMonths >= 3 && totalMonths < 6) {
      return {
        label: "ADVISORY",
        text: "Fragile runway. One systemic shock away from illiquidity.",
        color: "text-white/60",
      };
    }
    return {
      label: "OBSERVATION",
      text: "Runway stable. Capital ready for strategic deployment.",
      color: "text-emerald-500/80",
    };
  };
};

const verdict = getVerdict();

const formatCurrency = (val: number) => {
  return new Intl.NumberFormat("en-KE", {
    style: "currency",
    currency: "KES",
    maximumFractionDigits: 0,
  }).format(val);
};

return (
  <div className="w-full max-w-2xl mx-auto space-y-12 p-8 md:p-12 border border-white/10 bg-[#050505] rounded-2xl relative overflow-hidden group">
    {/* Background decoration */}
    <div className="absolute top-0 right-0 w-32 h-32 bg-white/[0.02] blur-3xl rounded-full" />

    <div className="space-y-8 relative z-10">
      <div className="flex justify-between items-end">
        <div className="space-y-1">
          <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40">
            Operational Variable 01
          </div>
          <div className="text-sm font-black uppercase tracking-widest">
            Available Liquidity
          </div>
        </div>
        <div className="font-mono text-2xl font-bold">
          {formatCurrency(liquidity)}
        </div>
      </div>
      <input
        type="range"
        min="0"
        max="1000000"
        step="5000"
        value={liquidity}
        onChange={(e) => setLiquidity(Number(e.target.value))}
        className="w-full h-[1px] bg-white/20 appearance-none cursor-crosshair accent-white"
      />
    </div>

    <div className="space-y-8 relative z-10">
```

Repository Audit

```
<div className="flex justify-between items-end">
  <div className="space-y-1">
    <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40">
      Operational Variable 02
    </div>
    <div className="text-sm font-black uppercase tracking-widest">
      Monthly Burn Rate
    </div>
  </div>
  <div className="font-mono text-2xl font-bold">
    {formatCurrency(monthlyBurn)}
  </div>
</div>
<input
  type="range"
  min="1000"
  max="500000"
  step="1000"
  value={monthlyBurn}
  onChange={(e) => setMonthlyBurn(Number(e.target.value))}
  className="w-full h-[1px] bg-white/20 appearance-none cursor-crosshair accent-white"
/>
</div>

<div className="pt-8 border-t border-white/5 space-y-6">
  <div className="flex justify-between items-baseline">
    <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40">
      Calculated Outcome
    </div>
    <div className="text-4xl font-black font-mono tracking-tighter flex items-baseline gap-2">
      {runwayResult.isInfinite ? (
        "?"
      ) : (
        <>
          <span>{runwayResult.months}</span>
          <span className="text-xs uppercase tracking-widest text-white/40">
            Months
          </span>
          {runwayResult.days > 0 && (
            <>
              <span className="ml-2">{runwayResult.days}</span>
              <span className="text-xs uppercase tracking-widest text-white/40">
                Days
              </span>
            </>
          )}
        </>
      )}
    </div>
  </div>
</div>

<div
  className={`p-6 border border-white/5 bg-white/[0.02] font-mono text-xs leading-relaxed space-y-2
  animate-in fade-in slide-in-from-top-1 duration-500`}
>
  <div className="flex items-center gap-2">
    <div
      className={`w-1.5 h-1.5 rounded-full animate-pulse ${verdict.label === "CRITICAL" ?
      "bg-orange-500" : verdict.label === "ADVISORY" ? "bg-white/40" : "bg-emerald-500"}`}
    />
    <span className={`font-black tracking-[0.2em] ${verdict.color}`}>
      {verdict.label}
    </span>
  </div>
</div>
```

Repository Audit

```
        <div className="text-white/60">{verdict.text}</div>
      </div>
    </div>

    <div className="absolute bottom-4 right-8 text-[8px] font-mono uppercase tracking-[0.4em] text-white/10">
      Deterministic Solver v0.4
    </div>
  </div>
);
}
```

File: src/components/landing/SolvencyCalculator.tsx

```
"use client";

import { useState, useMemo } from "react";
import { RippleButton } from "@components/ui/multi-type-ripple-buttons";
import { ScrollReveal } from "@components/ui/scroll-reveal";

export function SolvencyCalculator() {
  const [balance, setBalance] = useState(5000000);
  const [burn, setBurn] = useState(800000);

  const runway = useMemo(() => {
    if (burn === 0) return 99;
    const months = balance / burn;
    return months > 99 ? 99 : Math.floor(months);
  }, [balance, burn]);

  const formattedBalance = new Intl.NumberFormat("en-KE", {
    style: "currency",
    currency: "KES",
    maximumFractionDigits: 0,
  }).format(balance);

  const formattedBurn = new Intl.NumberFormat("en-KE", {
    style: "currency",
    currency: "KES",
    maximumFractionDigits: 0,
  }).format(burn);

  return (
    <section
      id="calculator"
      className="py-40 px-6 bg-[#050505] border-y border-white/5 relative overflow-hidden"
    >
      { /* Background Detail */ }
      <div className="absolute top-0 right-0 w-[40%] h-full bg-gradient-to-l from-white/[0.01] to-transparent
        pointer-events-none" />

      <div className="max-w-7xl mx-auto grid lg:grid-cols-[1.2fr_1fr] gap-24 items-center">
        <ScrollReveal direction="right" delay={0.2}>
          <div>
            <h2 className="font-cormorant text-6xl md:text-8xl text-white mb-12 leading-[0.9]">
              Measure your <br />
              <span className="italic">structural solvency.</span>
            </h2>

            <div className="space-y-16 mt-20">
              { /* Input 1 */ }
              <div className="space-y-8 group">
                <div className="flex justify-between items-end border-b border-white/10 pb-4 transition-colors
                  group-hover:border-white/20">
```

Repository Audit

```
<label className="text-white/30 font-mono text-[10px] uppercase tracking-[0.4em]">
  Total Liquid Assets
</label>
<span className="font-cormorant text-4xl text-white">
  {formattedBalance}
</span>
</div>
<input
  type="range"
  min="50000"
  max="100000000"
  step="50000"
  value={balance}
  onChange={(e) => setBalance(Number(e.target.value))}
  className="w-full"
/>
    <div className="flex justify-between text-[9px] text-white/10 font-mono uppercase
tracking-widest">
      <span>KES 50K</span>
      <span>KES 10M+</span>
    </div>
  </div>

  {/* Input 2 */}
  <div className="space-y-8 group">
    <div className="flex justify-between items-end border-b border-white/10 pb-4 transition-colors
group-hover:border-white/20">
      <label className="text-white/30 font-mono text-[10px] uppercase tracking-[0.4em]">
        Monthly Deployment
      </label>
      <span className="font-cormorant text-4xl text-white">
        {formattedBurn}
      </span>
    </div>
    <input
      type="range"
      min="10000"
      max="20000000"
      step="10000"
      value={burn}
      onChange={(e) => setBurn(Number(e.target.value))}
      className="w-full"
    />
      <div className="flex justify-between text-[9px] text-white/10 font-mono uppercase
tracking-widest">
        <span>KES 10K</span>
        <span>KES 2M+</span>
      </div>
    </div>
  </div>
</div>
</ScrollReveal>

{/* Results Card */}
<ScrollReveal direction="left" delay={0.4}>
  <div className="relative group">
    <div className="absolute -inset-4 bg-white/[0.02] blur-3xl rounded-full group-hover:bg-white/[0.04]
transition-all duration-1000" />

    <div className="relative bg-[#0a0a0a] border border-white/10 p-16 md:p-24 space-y-12 rounded-3xl
overflow-hidden">
      {/* Roman Decoration */}
      <div className="absolute top-0 right-0 p-8 font-cormorant italic text-white/5 text-[12rem]
leading-none select-none pointer-events-none">
```

Repository Audit

```
      II
    </div>

    <div>
      <p className="text-white/40 font-mono text-[10px] uppercase tracking-[0.5em] mb-4">
        Deterministic Runway
      </p>
      <div className="flex items-baseline gap-4">
        <span className="font-cormorant text-[10rem] md:text-[12rem] text-white leading-none tracking-tighter">
          {runway}
        </span>
        <span className="font-cormorant italic text-4xl text-white/20">
          Months
        </span>
      </div>
    </div>

    <div className="pt-12 border-t border-white/5 space-y-10">
      <p className="text-white/50 text-sm leading-relaxed font-light max-w-sm">
        This projection is based on your current liquidity
        architecture. Axiom provides the tools to optimize these
        variables and extend your runway indefinitely.
      </p>

      <RippleButton className="w-full py-6 text-xs">
        Establish Baseline Architecture
      </RippleButton>

      <p className="text-[9px] font-mono text-white/10 uppercase tracking-widest text-center italic">
        * Estimated from clinical mathematical truth
      </p>
    </div>
  </div>
</ScrollReveal>
</div>
</section>
);
}
```

File: src/components/landing/StrategicPillars.tsx

```
"use client";

import { useState } from "react";
import { ScrollReveal } from "@/components/ui/scroll-reveal";

const PILLARS = [
  {
    roman: "I",
    title: "Clinical Precision",
    desc: "We replace guesswork with deterministic financial modeling. Every shilling is audited against our five taxonomy gates to ensure zero leakage.",
  },
  {
    roman: "II",
    title: "Sovereign Liquidity",
    desc: "Axiom optimizes your capital for maximum availability. We ensure your assets remain liquid while maintaining their yield-generating potential.",
  },
  {
    roman: "III",
```

Repository Audit

```
    title: "Skill Multipliers",
    desc: "We prioritize investment in human capital. Our engine identifies high-alpha opportunities for skill acquisition that compound your earning power.",
  },
  {
    roman: "IV",
    title: "Risk Mitigation",
    desc: "Our automated risk-assessment protocols monitor global and local market trends, shielding your portfolio from systemic institutional failure.",
  },
  {
    roman: "V",
    title: "Legacy Architecture",
    desc: "Wealth is more than numbers. We help you build a durable financial legacy through strategic experience-equity and asset compounding.",
  },
];

export function StrategicPillars() {
  const [active, setActive] = useState<number | null>(0);

  return (
    <section className="py-40 px-6 bg-black border-b border-white/5 selection:bg-white selection:text-black">
      <div className="max-w-7xl mx-auto">
        <div className="grid lg:grid-cols-[1fr_1.5fr] gap-32 items-start">
          <div className="sticky top-40 space-y-12">
            <ScrollReveal direction="right">
              <div className="space-y-4">
                <span className="font-mono text-[10px] tracking-[0.5em] text-white/20 uppercase">
                  Strategic Foundation
                </span>
                <h2 className="font-cormorant text-7xl md:text-9xl text-white leading-[0.9]">
                  Why work <br />
                  <span className="italic">with Axiom?</span>
                </h2>
              </div>
            </ScrollReveal>

            <ScrollReveal direction="right" delay={0.2}>
              <div className="h-[1px] w-24 bg-white/20" />
            </ScrollReveal>

            <ScrollReveal direction="right" delay={0.3}>
              <p className="text-white/40 max-w-sm text-lg font-light leading-relaxed">
                At Axiom, we maintain a simple yet rigorous philosophy: Wealth
                is the delta between your earning power and your structural
                leakage.
              </p>
            </ScrollReveal>
          </div>

          <div className="space-y-2 border-t border-white/10">
            {PILLARS.map((pillar, idx) => (
              <ScrollReveal key={pillar.roman} direction="up" delay={0.1 * idx}>
                <div
                  className={`border-b border-white/10 transition-all duration-700 cursor-pointer group ${
                    active === idx
                      ? "bg-white/[0.02] px-8 py-12"
                      : "py-10 px-4 hover:bg-white/[0.01]"
                  }`}
                  onClick={() => setActive(active === idx ? null : idx)}
                >
                  <div className="flex items-center gap-12">
                    <span className="font-cormorant italic text-3xl text-white/10 group-hover:text-white/30
```

Repository Audit

```
transition-colors">
    {pillar.roman}.
  </span>
  <h3
    className={`font-cormorant text-4xl md:text-5xl text-white transition-all ${
      active === idx
        ? "italic translate-x-2"
        : "group-hover:translate-x-2"
    }`}
  >
    {pillar.title}
  </h3>
</div>

<div
  className={`overflow-hidden transition-all duration-700 ease-in-out ${
    active === idx
      ? "max-h-[500px] opacity-100 mt-12"
      : "max-h-0 opacity-0"
    }`}
>
  <p className="text-white/40 text-xl font-light leading-relaxed max-w-2xl pl-20">
    {pillar.desc}
  </p>
  <div className="mt-12 pl-20">
    <button className="text-[10px] font-mono tracking-[0.4em] text-white/60 hover:text-white
transition-colors uppercase">
      Learn More ?
    </button>
  </div>
</div>
</div>
</ScrollReveal>
  )})
</div>
</div>
</div>
</section>
);
}
```

File: src/components/ui/brand-mark.tsx

```
"use client";

import { motion } from "framer-motion";

export function BrandMark({ className = "w-10 h-10" }: { className?: string }) {
  return (
    <motion.div
      initial="initial"
      whileHover="hover"
      whileTap="tap"
      className={`${className} relative flex items-center justify-center cursor-pointer`}
    >
      {/* Main Rounded Background - Subtle Tilt Right */}
      <motion.div
        variants={{
          initial: { rotate: 0, scale: 1 },
          hover: {
            rotate: 4,
            scale: 1.08,
            boxShadow: "0 15px 30px rgba(255, 255, 255, 0.1)",
          },
        }}
      />
    </motion.div>
  );
}
```

Repository Audit

```
    },
    tap: { scale: 0.95 },
  }}
  transition={{
    type: "spring",
    stiffness: 300,
    damping: 25,
  }}
  className="absolute inset-0 bg-white rounded-[32%] shadow-lg"
/>

{/* Pop-out Frame - More rotation and scale for depth */}
<motion.div
  variants={{
    initial: { rotate: 0, scale: 1, opacity: 0 },
    hover: {
      rotate: 10,
      scale: 1.25,
      opacity: 0.3,
    },
  }},
  transition={{
    type: "spring",
    stiffness: 250,
    damping: 20,
  }}
  className="absolute inset-0 border border-white rounded-[32%] pointer-events-none"
/>

{/* Static Text - San Francisco (SF Pro) */}
<span
  className="relative z-10 text-black font-black text-xl select-none pointer-events-none"
  style={{
    fontFamily:
      '-apple-system, BlinkMacSystemFont, "SF Pro Display", "SF Pro Text", "Helvetica Neue", Helvetica,
Arial, sans-serif',
  }}
>
  A
</span>
</motion.div>
);
}
```

File: src/components/ui/hover-glow-button.tsx

```
import React, { useRef, useState, MouseEvent, ReactNode } from 'react';

interface ButtonProps {
  children: ReactNode;
  onClick?: () => void;
  className?: string;
  disabled?: boolean;
  glowColor?: string;
  backgroundColor?: string;
  textColor?: string;
  hoverTextColor?: string;
}

const HoverButton: React.FC<ButtonProps> = ({
  children,
  onClick,
  className = '',
```

Repository Audit

```
disabled = false,
glowColor = 'ffffff', // Default to white for luxury feel
backgroundColor = '#000000',
textColor = 'ffffff',
hoverTextColor = 'ffffff'
}) => {
  const buttonRef = useRef<HTMLButtonElement>(null);
  const [glowPosition, setGlowPosition] = useState({ x: 50, y: 50 });
  const [isHovered, setIsHovered] = useState(false);

  const handleMouseMove = (e: MouseEvent<HTMLButtonElement>) => {
    if (buttonRef.current) {
      const rect = buttonRef.current.getBoundingClientRect();
      const x = e.clientX - rect.left;
      const y = e.clientY - rect.top;
      setGlowPosition({ x, y });
    }
  };

  const handleMouseEnter = () => {
    setIsHovered(true);
  };

  const handleMouseLeave = () => {
    setIsHovered(false);
  };

  return (
    <button
      ref={buttonRef}
      onClick={onClick}
      disabled={disabled}
      onMouseMove={handleMouseMove}
      onMouseEnter={handleMouseEnter}
      onMouseLeave={handleMouseLeave}
      className={`
        relative inline-block px-8 py-4 border border-white/10
        cursor-pointer overflow-hidden transition-all duration-500
        text-sm tracking-widest uppercase z-10 font-sans
        ${disabled ? 'opacity-50 cursor-not-allowed' : ''}
        ${className}
      `}
      style={{
        backgroundColor: backgroundColor,
        color: isHovered ? hoverTextColor : textColor,
      }}
    >
      { /* Glow effect div */ }
      <div
        className={`
          absolute w-[200px] h-[200px] rounded-full opacity-20 pointer-events-none
          transition-transform duration-700 ease-out -translate-x-1/2 -translate-y-1/2
          ${isHovered ? 'scale-150' : 'scale-0'}
        `}
        style={{
          left: `${glowPosition.x}px`,
          top: `${glowPosition.y}px`,
          background: `radial-gradient(circle, ${glowColor} 0%, transparent 70%)`,
          zIndex: 0,
        }}
      />

      { /* Button content */ }
      <span className="relative z-10">{children}</span>
    </button>
  );
}
```

Repository Audit

```
        </button>
      );
    };

export { HoverButton }
```

File: src/components/ui/luxury-card.tsx

```
"use client";

import React, { useState, useEffect, useRef } from "react";
import type { MouseEvent, ReactNode } from "react";
import { motion } from "framer-motion";

// Type definitions
interface Particle {
  id: number;
  left: number;
  duration: number;
  opacity: number;
}

interface CardTransform {
  rotateX: number;
  rotateY: number;
  translateY: number;
  scale: number;
}

export function LuxuryCard({ children, className = "" }: { children: ReactNode; className?: string }) {
  const [transform, setTransform] = useState<CardTransform>({
    rotateX: 0,
    rotateY: 0,
    translateY: 0,
    scale: 1,
  });

  const cardRef = useRef<HTMLDivElement>(null);
  const [particles, setParticles] = useState<Particle[]>([]);
  const particleId = useRef<number>(0);
  const [ripples, setRipples] = useState<Array<{ id: number; x: number; y: number }>>([]);
  const rippleId = useRef<number>(0);

  // 3D Transform Logic
  const handleMouseMove = (e: MouseEvent<HTMLDivElement>): void => {
    if (!cardRef.current) return;

    const rect = cardRef.current.getBoundingClientRect();
    const x = e.clientX - rect.left - rect.width / 2;
    const y = e.clientY - rect.top - rect.height / 2;

    const rotateX = (y / rect.height) * -10; // More subtle for forms
    const rotateY = (x / rect.width) * 10;

    setTransform({
      rotateX,
      rotateY,
      translateY: -10,
      scale: 1.01,
    });
  };

  const handleMouseLeave = (): void => {
```

Repository Audit

```
setTransform({
  rotateX: 0,
  rotateY: 0,
  translateY: 0,
  scale: 1,
});
};

// Click Ripple Logic
const handleCardClick = (e: MouseEvent<HTMLDivElement>): void => {
  const rect = e.currentTarget.getBoundingClientRect();
  const size = 60;
  const x = e.clientX - rect.left - size / 2;
  const y = e.clientY - rect.top - size / 2;

  const newRipple = { id: rippleId.current++, x, y };
  setRipples((prev) => [...prev, newRipple]);
  setTimeout(() => {
    setRipples((current) => current.filter((r) => r.id !== newRipple.id));
  }, 600);
};

// Particle System
useEffect(() => {
  const interval = setInterval(() => {
    const newParticle: Particle = {
      id: particleId.current++,
      left: Math.random() * 100,
      duration: Math.random() * 3 + 2,
      opacity: Math.random() * 0.3 + 0.1,
    };

    setParticles((prev) => [...prev, newParticle]);

    setTimeout(() => {
      setParticles((prev) => prev.filter((p) => p.id !== newParticle.id));
    }, newParticle.duration * 1000);
  }, 400);

  return () => clearInterval(interval);
}, []);

const transformStyle = `perspective(1000px) translateY(${transform.translateY}px)
rotateX(${transform.rotateX}deg) rotateY(${transform.rotateY}deg) scale(${transform.scale})`;

return (
  <div
    ref={cardRef}
    onMouseMove={handleMouseMove}
    onMouseLeave={handleMouseLeave}
    onClick={handleCardClick}
    className={`relative rounded-[3rem] backdrop-blur-3xl transition-all duration-500 transform-gpu
preserve-3d border border-white/5 bg-white/[0.02] shadow-2xl ${className}`}
    style={{
      transform: transformStyle,
      animation: "luxury-float 6s ease-in-out infinite",
    }}
  >
    {/* Particles Overlay */}
    <div className="absolute inset-0 rounded-[3rem] overflow-hidden pointer-events-none z-0">
      {particles.map((particle) => (
        <div
          key={particle.id}
          className="absolute w-0.5 h-0.5 rounded-full bg-white/20"
```

Repository Audit

```
        style={{
          left: `${particle.left}%`,
          opacity: particle.opacity,
          animation: `luxury-particle-float ${particle.duration}s linear infinite`,
        }}
      />
    ))}
  </div>

  {/* Ripples Overlay */}
  {ripples.map((ripple) => (
    <span
      key={ripple.id}
      className="absolute w-15 h-15 rounded-full pointer-events-none bg-white/10 z-0"
      style={{
        left: ripple.x,
        top: ripple.y,
        animation: "luxury-ripple 0.6s ease-out",
      }}
    />
  ))}

  {/* Card Highlight */}
  <div className="absolute inset-0 bg-gradient-to-br from-white/[0.03] to-transparent pointer-events-none rounded-[3rem]" />

  {/* Content */}
  <div className="relative z-10 h-full flex flex-col">
    {children}
  </div>

  {/* Internal Styles */}
  <style jsx global>{`
    @keyframes luxury-float {
      0%, 100% { transform: ${transformStyle} translateY(0px); }
      50% { transform: ${transformStyle} translateY(-8px); }
    }

    @keyframes luxury-particle-float {
      0% { transform: translateY(100%) scale(0); opacity: 0; }
      10% { opacity: 1; }
      90% { opacity: 1; }
      100% { transform: translateY(-100%) scale(1); opacity: 0; }
    }

    @keyframes luxury-ripple {
      0% { transform: scale(0); opacity: 1; }
      100% { transform: scale(4); opacity: 0; }
    }
  `}</style>
</div>
);
}
```

File: src/components/ui/multi-type-ripple-buttons.tsx

```
import React, { ReactNode, useState, useMemo, MouseEvent, CSSProperties } from 'react';

interface RippleState {
  key: number;
  x: number;
  y: number;
  size: number;
}
```

Repository Audit

```
    color: string;
  }

interface RippleButtonProps {
  children: ReactNode;
  onClick?: (event: MouseEvent<HTMLButtonElement>) => void;
  className?: string;
  disabled?: boolean;
  variant?: 'default' | 'hover' | 'ghost' | 'hoverborder';
  rippleColor?: string; // User override for the JS click ripple color
  rippleDuration?: number; // Duration for the JS click ripple (all variants)

  // For 'hover' variant
  hoverBaseColor?: string;
  hoverRippleColor?: string;

  // For 'hoverborder' variant
  hoverBorderEffectColor?: string; // Color of the visual effect forming the border
  hoverBorderEffectThickness?: string; // Thickness of the border effect (e.g., "0.3em", "2px")
}

const hexToRgba = (hex: string, alpha: number): string => {
  let hexValue = hex.startsWith('#') ? hex.slice(1) : hex;
  if (hexValue.length === 3) {
    hexValue = hexValue.split('').map(char => char + char).join('');
  }
  const r = parseInt(hexValue.slice(0, 2), 16);
  const g = parseInt(hexValue.slice(2, 4), 16);
  const b = parseInt(hexValue.slice(4, 6), 16);
  return `rgba(${r}, ${g}, ${b}, ${alpha})`;
};

const GRID_HOVER_NUM_COLS = 36;
const GRID_HOVER_NUM_ROWS = 12;
const GRID_HOVER_TOTAL_CELLS = GRID_HOVER_NUM_COLS * GRID_HOVER_NUM_ROWS;
const GRID_HOVER_RIPPLE_EFFECT_SIZE = "18.973665961em";

const JS_RIPPLE_KEYFRAMES = `
  @keyframes js-ripple-animation {
    0% { transform: scale(0); opacity: 1; }
    100% { transform: scale(1); opacity: 0; }
  }
  .animate-js-ripple-effect {
    animation: js-ripple-animation var(--ripple-duration) ease-out forwards;
  }
`;

const RippleButton: React.FC<RippleButtonProps> = ({
  children,
  onClick,
  className = '',
  disabled = false,
  variant = 'default',
  rippleColor: userProvidedRippleColor,
  rippleDuration = 600,
  hoverBaseColor = '#ffffff',
  hoverRippleColor: customHoverRippleColor,
  hoverBorderEffectColor = 'rgba(255, 255, 255, 0.2)',
  hoverBorderEffectThickness = '0.1em',
}) => {
  const [jsRipples, setJsRipples] = useState<RippleState[]>([]);

  const determinedJsRippleColor = useMemo(() => {
    if (userProvidedRippleColor) {
```

Repository Audit

```
    return userProvidedRippleColor;
  }
  return 'var(--button-ripple-color, rgba(255, 255, 255, 0.1))';
}, [userProvidedRippleColor]);

const dynamicGridHoverStyles = useMemo(() => {
  let nthChildHoverRules = '';
  const cellDim = 0.25;
  const initialTopOffset = 0.125;
  const initialLeftOffset = 0.1875;

  // Standardized hover transition duration for width and height
  const hoverEffectDuration = '0.9s';

  for (let r = 0; r < GRID_HOVER_NUM_ROWS; r++) {
    for (let c = 0; c < GRID_HOVER_NUM_COLS; c++) {
      const childIndex = r * GRID_HOVER_NUM_COLS + c + 1;
      const topPos = initialTopOffset + r * cellDim;
      const leftPos = initialLeftOffset + c * cellDim;

      if (variant === 'hover') {
        nthChildHoverRules += `
        .hover-variant-grid-cell:nth-child(${childIndex}):hover ~ .hover-variant-visual-ripple {
          top: ${topPos}em; left: ${leftPos}em;
          transition: width ${hoverEffectDuration} ease, height ${hoverEffectDuration} ease, top 0s linear,
left 0s linear;
        }`;
      } else if (variant === 'hoverborder') {
        nthChildHoverRules += `
        .hoverborder-variant-grid-cell:nth-child(${childIndex}):hover ~ .hoverborder-variant-visual-ripple
{
          top: ${topPos}em; left: ${leftPos}em;
          transition: width ${hoverEffectDuration} ease-out, height ${hoverEffectDuration} ease-out, top 0s
linear, left 0s linear;
        }`;
      }
    }
  }

  if (variant === 'hover') {
    const actualHoverRippleColor = customHoverRippleColor
      ? customHoverRippleColor
      : hexToRgba(hoverBaseColor, 0.15);
    return `
    .hover-variant-visual-ripple {
      background-color: ${actualHoverRippleColor};
      transition: width ${hoverEffectDuration} ease, height ${hoverEffectDuration} ease, top 99999s linear,
left 99999s linear;
    }
    .hover-variant-grid-cell:hover ~ .hover-variant-visual-ripple {
      width: ${GRID_HOVER_RIPPLE_EFFECT_SIZE}; height: ${GRID_HOVER_RIPPLE_EFFECT_SIZE};
    }
    ${nthChildHoverRules}
  `;
  } else if (variant === 'hoverborder') {
    return `
    .hoverborder-variant-ripple-container {
      padding: ${hoverBorderEffectThickness};
      mask: linear-gradient(#fff 0 0) content-box, linear-gradient(#fff 0 0);
      mask-composite: exclude;
      -webkit-mask: linear-gradient(#fff 0 0) content-box, linear-gradient(#fff 0 0);
      -webkit-mask-composite: xor;
    }
    .hoverborder-variant-visual-ripple {

```

Repository Audit

```
        background-color: ${hoverBorderEffectColor};
        /* Ensure the base transition also uses the standardized duration for width/height */
        transition: width ${hoverEffectDuration} ease-out, height ${hoverEffectDuration} ease-out, top 99999s
linear, left 9999s linear;
    }
    .hoverborder-variant-grid-cell:hover ~ .hoverborder-variant-visual-ripple {
        width: ${GRID_HOVER_RIPPLE_EFFECT_SIZE}; height: ${GRID_HOVER_RIPPLE_EFFECT_SIZE};
    }
    ${nthChildHoverRules}
`;
}
return '';
}, [variant, hoverBaseColor, customHoverRippleColor, hoverBorderEffectColor, hoverBorderEffectThickness]);

const createJsRipple = (event: MouseEvent<HTMLButtonElement>) => {
    const button = event.currentTarget;
    const rect = button.getBoundingClientRect();
    const size = Math.max(rect.width, rect.height) * 2;
    const x = event.clientX - rect.left - size / 2;
    const y = event.clientY - rect.top - size / 2;
    const newRipple: RippleState = { key: Date.now(), x, y, size, color: determinedJsRippleColor };
    setJsRipples(prev => [...prev, newRipple]);
    setTimeout(() => {
        setJsRipples(currentRipples => currentRipples.filter(r => r.key !== newRipple.key));
    }, rippleDuration);
};

const handleButtonClick = (event: MouseEvent<HTMLButtonElement>) => {
    if (!disabled) {
        createJsRipple(event);
        if (onClick) onClick(event);
    }
};

const jsRippleElements = (
    <div className="absolute inset-0 pointer-events-none z-[5]">
        {jsRipples.map(ripple => (
            <span
                key={ripple.key}
                className="absolute rounded-full animate-js-ripple-effect"
                style={{
                    left: ripple.x, top: ripple.y, width: ripple.size, height: ripple.size,
                    backgroundColor: ripple.color,
                    ['--ripple-duration' as any]: `${rippleDuration}ms`,
                } as CSSProperties}
            />
        ))}
    </div>
);

if (variant === 'hover') {
    const hoverButtonFinalClassName = [
        "relative", "rounded-full", "text-[10px] tracking-widest uppercase font-mono", "px-8", "py-3",
        "border border-white/10", "bg-transparent", "isolate", "overflow-hidden", "cursor-pointer",
        disabled ? "opacity-50 cursor-not-allowed pointer-events-none" : "",
        className,
    ].filter(Boolean).join(" ");
    return (
        <>
            <style dangerouslySetInnerHTML={{ __html: JS_RIPPLE_KEYFRAMES }} />
            <style dangerouslySetInnerHTML={{ __html: dynamicGridHoverStyles }} />
            <button className={hoverButtonFinalClassName} onClick={handleButtonClick} disabled={disabled}>
                <span className="relative z-[10] pointer-events-none">{children}</span>
                {jsRippleElements}
            </button>
        </>
    );
}
```

Repository Audit

```

    <div
      className="hover-variant-grid-container absolute inset-0 grid overflow-hidden pointer-events-none
z-[0]"
      style={{ gridTemplateColumns: `repeat(${GRID_HOVER_NUM_COLS}, 0.25em)` }}
    >
      {Array.from({ length: GRID_HOVER_TOTAL_CELLS }, (_, index) => (
        <span key={index} className="hover-variant-grid-cell relative flex justify-center items-center
pointer-events-auto" />
      ))}
      <div className="hover-variant-visual-ripple pointer-events-none absolute w-0 h-0 rounded-full
transform -translate-x-1/2 -translate-y-1/2 top-0 left-0 z-[-1]" />
    </div>
  </button>
</>
);
}

if (variant === 'hoverborder') {
  const hoverBorderButtonFinalClassName = [
    "relative", "rounded-full", "overflow-hidden", "text-[10px] tracking-widest uppercase font-mono", "px-8",
"py-3",
    "border-none", "bg-transparent", "isolate", "cursor-pointer",
    disabled ? "opacity-50 cursor-not-allowed pointer-events-none" : "",
    className,
  ].filter(Boolean).join(" ");

  return (
    <>
      <style dangerouslySetInnerHTML={{ __html: JS_RIPPLE_KEYFRAMES }} />
      <style dangerouslySetInnerHTML={{ __html: dynamicGridHoverStyles }} />
      <button
        className={hoverBorderButtonFinalClassName}
        onClick={handleButtonClick}
        disabled={disabled}
      >
        <span className="relative z-[10] pointer-events-none">{children}</span>
        {jsRippleElements}
        <div
          className="hoverborder-variant-ripple-container absolute inset-0 grid rounded-full overflow-hidden
pointer-events-none z-[0]"
          style={{ gridTemplateColumns: `repeat(${GRID_HOVER_NUM_COLS}, 0.25em)` }}
        >
          {Array.from({ length: GRID_HOVER_TOTAL_CELLS }, (_, index) => (
            <span
              key={index}
              className="hoverborder-variant-grid-cell relative flex justify-center items-center
pointer-events-auto"
            />
          ))}
          <div className="hoverborder-variant-visual-ripple pointer-events-none absolute w-0 h-0 rounded-full
transform -translate-x-1/2 -translate-y-1/2 top-0 left-0 z-[-1]" />
        </div>
      </button>
    </>
  );
}

if (variant === 'ghost') {
  const ghostButtonFinalClassName = [
    "relative", "border-none", "bg-transparent", "isolate", "overflow-hidden", "cursor-pointer",
    "px-6", "py-2", "rounded-full", "text-[10px] tracking-widest uppercase font-mono",
    disabled ? "opacity-50 cursor-not-allowed pointer-events-none" : "",
    className,
  ].filter(Boolean).join(" ");
```

Repository Audit

```
return (
  <>
    <style dangerouslySetInnerHTML={{ __html: JS_RIPPLE_KEYFRAMES }} />
    <button className={ghostButtonFinalClassName} onClick={handleButtonClick} disabled={disabled}>
      <span className="relative z-10 pointer-events-none">{children}</span>
      {jsRippleElements}
    </button>
  </>
);
}

// Default variant
const baseClasses = "relative border-none overflow-hidden isolate transition-all duration-500 cursor-pointer px-10 py-3 bg-white hover:bg-neutral-200 text-black rounded-full font-medium tracking-widest uppercase text-[10px]";
const disabledClasses = disabled ? "opacity-50 cursor-not-allowed" : "";
const buttonClasses = `${baseClasses} ${disabledClasses} ${className}`;
return (
  <>
    <style dangerouslySetInnerHTML={{ __html: JS_RIPPLE_KEYFRAMES }} />
    <button className={buttonClasses} onClick={handleButtonClick} disabled={disabled}>
      <span className="relative z-[1] pointer-events-none">{children}</span>
      {jsRippleElements}
    </button>
  </>
);
};

export { RippleButton };
```

File: src/components/ui/scroll-reveal.tsx

```
"use client";

import { motion } from "framer-motion";
import { ReactNode } from "react";

interface ScrollRevealProps {
  children: ReactNode;
  delay?: number;
  direction?: "up" | "down" | "left" | "right" | "none";
  distance?: number;
  duration?: number;
}

export function ScrollReveal({
  children,
  delay = 0,
  direction = "up",
  distance = 40,
  duration = 0.8,
}: ScrollRevealProps) {
  const directions = {
    up: { y: distance },
    down: { y: -distance },
    left: { x: distance },
    right: { x: -distance },
    none: {},
  };

  return (
    <motion.div
      initial={{
```

Repository Audit

```
        opacity: 0,
        ...directions[direction]
      }}
      whileInView={{
        opacity: 1,
        x: 0,
        y: 0
      }}
      viewport={{ once: true, margin: "-10%" }}
      transition={{
        duration: duration,
        delay: delay,
        ease: [0.215, 0.61, 0.355, 1], // Luxury cubic bezier (easeOutCirc)
      }}
    >
      {children}
    </motion.div>
  );
}
```

File: src/components/ui/smooth-scroll-provider.tsx

```
"use client";

import { ReactLenis } from "lenis/react";
import { ReactNode } from "react";

export function SmoothScrollProvider({ children }: { children: ReactNode }) {
  return (
    <ReactLenis
      root
      options={{
        duration: 1.5,
        easing: (t) => Math.min(1, 1.001 - Math.pow(2, -10 * t)),
        orientation: "vertical",
        gestureOrientation: "vertical",
        smoothWheel: true,
        wheelMultiplier: 1,
        touchMultiplier: 2,
        infinite: false,
      }}
    >
      {children}
    </ReactLenis>
  );
}
```

File: src/lib/ai/kairos.test.ts

```
import assert from "node:assert/strict";
import test from "node:test";
import { generateSystemInsights } from "../kairos";
import { AnalyticsSummary } from "../analytics/types";
import { buildBehavioralContext } from "../context/buildBehavioralContext";
import { evaluateInsights } from "../insights/generateInsights";

const MOCK_METADATA_QUALITY = {
  averageScore: 1,
  lowQualityCount: 0,
  lowQualityRatio: 0,
  confidenceMultiplier: 1,
};
```

Repository Audit

```
const MOCK_ANALYTICS: AnalyticsSummary = {
  totalDeployed: 1000,
  averageDeployment: 100,
  dailyBurnRate: 10,
  runwayDays: 100,
  categoryBreakdown: { Leakage: 500, Asset: 500 },
  deploymentCount: 10,
  metadataQuality: MOCK_METADATA_QUALITY,
  totalLiabilities: 0,
  totalAssets: 2000,
  netWorth: 2000,
  liquidity: 1000,
  totalMonthlyIncome: 300,
  recurringIncome: 300,
  irregularIncome: 0,
  incomeConcentration: {},
  maxIncomeConcentrationRatio: 0,
  adjustedDailyBurn: 0,
  totalStrategicTargets: 1000,
  totalCurrentProgress: 500,
  averageGoalProgress: 50,
  criticalGoalCount: 0,
  fundingGap: 500,
  // Strategic Objectives v1
  strategicAlignment: {
    fundingRatios: {},
    alignmentPressure: 0,
    liquiditySufficiency: {
      isSufficient: true,
      message:
        "Current liquidity structure sufficiently supports short-term obligations.",
      shortfall: 0,
    },
    objectiveStarvationSignals: [],
    strategicAllocationSignals: [],
    velocity: {},
  },
  // Operational Baseline v1
  totalStructuralMonthlyBurn: 0,
  totalSystemicMonthlyAllocation: 0,
};

test("Kairos Insights: correctly invokes rule engine and returns primary insight", async () => {
  const context = buildBehavioralContext(
    { currentAnalytics: MOCK_ANALYTICS },
    [100, 200, 300],
  );
  const { primaryInsight } = await evaluateInsights(context);

  assert.ok(primaryInsight.message);
  assert.ok(primaryInsight.severity);
  assert.ok(primaryInsight.category);
});

test("Kairos Insights: critical runway trigger", async () => {
  const analytics = {
    ...MOCK_ANALYTICS,
    runwayDays: 10,
  };
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    [100, 200, 300],
  );
});
```

Repository Audit

```
const { primaryInsight } = await evaluateInsights(context);

assert.equal(primaryInsight.severity, "critical");
assert.equal(primaryInsight.category, "solvency_pressure");
assert.match(primaryInsight.message, /runway has contracted/);
});

test("Kairos Insights: efficiency crisis trigger", async () => {
  const analytics = {
    ...MOCK_ANALYTICS,
    categoryBreakdown: { Leakage: 1000, Asset: 0 },
    runwayDays: 10, // Force low runway to plummet efficiency score < 40
  };
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    [100, 2000, 50, 3000],
  );

  // Rule condition: capitalEfficiencyScore < 40
  // buildBehavioralContext for runway < 90 gives -40 penalty.
  // Volatility penalty is vol * 60.

  const { primaryInsight } = await evaluateInsights(context);

  // Solvency pressure (runway) usually has higher priority than efficiency crisis
  // if both are triggered, since runway is status: 'critical'.
  // However, in registry.ts, both have priority: 'high'.
  // evaluateInsights sorts by priority and then maintains order.
  // runway_critical is ID 1, efficiency_crisis is ID 2.

  assert.equal(primaryInsight.severity, "critical");
  assert.match(
    primaryInsight.message,
    /(runway has contracted|efficiency crisis)/,
  );
});

test("Kairos Insights: unclassified data advisory", async () => {
  const analytics = {
    ...MOCK_ANALYTICS,
    categoryBreakdown: { Unknown: 500, Asset: 500 },
  };
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    [500, 500],
  );
  const { allInsights } = await evaluateInsights(context);

  const unclassifiedInsight = allInsights.find((i) =>
    i.message.includes("Unclassified"),
  );
  assert.ok(unclassifiedInsight);
  assert.equal(unclassifiedInsight.severity, "advisory");
});

test("Kairos Insights: high discipline pattern", async () => {
  // Healthy state with stable spending
  const context = buildBehavioralContext(
    { currentAnalytics: MOCK_ANALYTICS },
    [100, 100, 100],
  );
  const { primaryInsight } = await evaluateInsights(context);

  assert.equal(primaryInsight.severity, "observation");
```

Repository Audit

```
    assert.match(primaryInsight.message, /High operational discipline/);
    assert.equal(primaryInsight.isSilent, true);
  });

test("Kairos Insights: silence state default pattern", async () => {
  // State that doesn't trigger High Discipline but falls into Default Pattern
  const analytics = {
    ...MOCK_ANALYTICS,
    deploymentCount: 1, // High Discipline requires count > 1
  };
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    [100],
  );
  const { primaryInsight } = await evaluateInsights(context);

  assert.equal(primaryInsight.severity, "observation");
  assert.match(primaryInsight.message, /No material behavioral shifts/);
  assert.equal(primaryInsight.isSilent, true);
});
```

File: src/lib/ai/kairos.ts

```
import { createClient } from "../supabase/server";
import { getDeployments } from "../db/deployments";
import { getAccounts } from "../db/accounts";
import { getLiabilities } from "../db/liabilities";
import { getIncomeStreams } from "../db/income";
import { getGoals } from "../db/goals";
import { getStrategicObjectives } from "../db/objectives";
import { getOperationalBaseline } from "../db/baseline";
import { getUserSettings } from "../db/settings";
import { generateSummary } from "../analytics/engine";
import {
  Deployment,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  OperationalBaseline,
} from "../analytics/types";
import { MetadataQualitySummary } from "../finance/metadataQuality";
import { buildBehavioralContext } from "../context/buildBehavioralContext";
import { evaluateInsights } from "../insights/generateInsights";

export type InsightSeverity =
  | "observation"
  | "advisory"
  | "warning"
  | "critical";

export interface KairosInsight {
  type: "warning" | "info" | "pattern" | "opportunity";
  severity: InsightSeverity;
  category:
    | "capital_efficiency"
    | "solvency_pressure"
    | "strategic_alignment"
    | "objective_starvation";
  message: string;
  supportingSignals: string[];
  timestamp: string;
```

Repository Audit

```
runway: number | null;
capitalEfficiency: number;
isSilent: boolean;
metadataQuality?: MetadataQualitySummary;
is_new_signal?: boolean;
}

export interface KairosTelemetry {
  deployments?: Deployment[];
  liquidity?: number;
  accounts?: Account[];
  liabilities?: Liability[];
  incomeStreams?: IncomeStream[];
  goals?: FinancialGoal[];
  objectives?: StrategicObjective[];
  baseline?: OperationalBaseline[];
  previousInsight?: KairosInsight | null;
}

/**
 * SERVER-SIDE ORCHESTRATOR: generateSystemInsights
 * Safely hydrates the Kairos AI layer with analytical context.
 */
export async function generateSystemInsights(
  telemetry: KairosTelemetry = {},
): Promise<KairosInsight> {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();

  if (!user) {
    throw new Error("Authentication required for Kairos hydration.");
  }

  // 1. Resolve Canonical Context (Hydrate missing pieces only)
  const deps =
    (telemetry.deployments ?? (await getDeployments(supabase))) || [];
  const accounts = (telemetry.accounts ?? (await getAccounts(supabase))) || [];
  const liabilities =
    (telemetry.liabilities ?? (await getLiabilities(supabase))) || [];
  const income =
    (telemetry.incomeStreams ?? (await getIncomeStreams(supabase))) || [];
  const goals = (telemetry.goals ?? (await getGoals(supabase))) || [];
  const objectives =
    (telemetry.objectives ?? (await getStrategicObjectives(supabase))) || [];
  const baseline =
    (telemetry.baseline ?? (await getOperationalBaseline(supabase))) || [];

  let liquidity = telemetry.liquidity;
  if (liquidity === undefined) {
    const settings = await getUserSettings(supabase, user.id);
    liquidity = Number(
      (settings as { total_liquidity: number | string }).total_liquidity,
    );
  }

  // 2. Build Deterministic Analytics
  const analytics = generateSummary(
    deps,
    liquidity,
    accounts,
    liabilities,
    income,
```

Repository Audit

```
goals,
objectives,
baseline,
);

// 3. Build Behavioral Context for Insight Engine
const context = buildBehavioralContext(
  { currentAnalytics: analytics },
  deps.map((d) => Number(d.amount)),
);

// 4. Evaluate Insights via Rule Registry (V1.1)
console.log("KAHIROS TELEMETRY:", {
  deploymentsCount: deps.length,
  runwayDays: analytics.runwayDays,
  alignmentPressure: analytics.strategicAlignment.alignmentPressure,
  netWorth: analytics.netWorth,
});

const { primaryInsight } = await evaluateInsights(context);

const previousInsight = telemetry.previousInsight;
const isMessageChanged = primaryInsight.message !== previousInsight?.message;
const isSeverityChanged =
  primaryInsight.severity !== previousInsight?.severity;
const isSilentChanged = primaryInsight.isSilent !== previousInsight?.isSilent;

const isNewSignal = isMessageChanged || isSeverityChanged || isSilentChanged;

return {
  ...primaryInsight,
  is_new_signal: isNewSignal,
};
}

/**
 * Bridge support for streamed telemetry from Dashboard Actions.
 */
export async function generateKairosAIInsight(
  telemetry: KairosTelemetry,
): Promise<KairosInsight> {
  return generateSystemInsights(telemetry);
}
```

File: src/lib/analytics/engine.test.ts

```
import assert from "node:assert/strict";
import test from "node:test";
import {
  projectRunway,
  calculateBurnRate,
  calculateIncomeMetrics,
} from "../engine";
import type { Deployment, IncomeStream } from "../types";

test("projectRunway handles dimensional math correctly", () => {
  // Scenario: 1000 balance, 10 daily burn, 0 income
  // Runway = 1000 / 10 = 100 days
  assert.equal(projectRunway(1000, 10, 0), 100);

  // Scenario: 1000 balance, 10 daily burn, 150 monthly income
  // adjustedDailyBurn = 10 - (150 / 30) = 5
  // Runway = 1000 / 5 = 200 days
```

Repository Audit

```
    assert.equal(projectRunway(1000, 10, 150), 200);
  });

test("projectRunway returns null when state is stable (income >= burn)", () => {
  // Scenario: 10 daily burn, 300 monthly income
  // adjustedDailyBurn = 10 - (300 / 30) = 0
  assert.equal(projectRunway(1000, 10, 300), null);

  // Scenario: 10 daily burn, 450 monthly income
  // adjustedDailyBurn = 10 - (450 / 30) = -5
  assert.equal(projectRunway(1000, 10, 450), null);
});

test("projectRunway returns 0 when balance is 0 or negative (and burn > income)", () => {
  assert.equal(projectRunway(0, 10, 0), 0);
  assert.equal(projectRunway(-100, 10, 0), 0);
});

test("projectRunway returns null when dailyBurnRate is 0 or negative (unstable state but no burn)", () => {
  assert.equal(projectRunway(1000, 0, 0), null);
  assert.equal(projectRunway(1000, -5, 0), null);
});

test("calculateBurnRate remains deterministic", () => {
  const deployments: Deployment[] = [
    {
      id: "1",
      amount: 300,
      title: "Rent",
      category: "Leakage",
      created_at: new Date().toISOString(),
    },
    {
      id: "2",
      amount: 150,
      title: "Cloud",
      category: "Leverage",
      created_at: new Date().toISOString(),
    },
  ];

  // 450 total / 30 days = 15/day
  assert.equal(calculateBurnRate(deployments, 30), 15);
});

test("calculateMaxIncomeConcentration: 90% dependency", () => {
  const streams: Partial<IncomeStream>[] = [
    {
      income_name: "Source A",
      amount: 900,
      cadence: "monthly",
      income_type: "salary",
      is_recurring: true,
    },
    {
      income_name: "Source B",
      amount: 100,
      cadence: "monthly",
      income_type: "freelance",
      is_recurring: true,
    },
  ];

  // total = 1000
```

Repository Audit

```
// max = 900
// ratio = 0.9
const metrics = calculateIncomeMetrics(streams as IncomeStream[]);
assert.equal(metrics.maxRatio, 0.9);
});

test("calculateMaxIncomeConcentration: balanced split", () => {
  const streams: Partial<IncomeStream>[] = [
    {
      income_name: "Source A",
      amount: 100,
      cadence: "monthly",
      income_type: "salary",
      is_recurring: true,
    },
    {
      income_name: "Source B",
      amount: 100,
      cadence: "monthly",
      income_type: "salary",
      is_recurring: true,
    },
    {
      income_name: "Source C",
      amount: 100,
      cadence: "monthly",
      income_type: "salary",
      is_recurring: true,
    },
  ];

  // total = 300
  // max = 100
  // ratio = 0.333...
  const metrics = calculateIncomeMetrics(streams as IncomeStream[]);
  assert.ok(Math.abs(metrics.maxRatio - 0.3333333333333333) < 0.0001);
});

test("calculateMaxIncomeConcentration: zero income", () => {
  const metrics = calculateIncomeMetrics([]);
  assert.equal(metrics.maxRatio, 0);
});
```

File: src/lib/analytics/engine.ts

```
import {
  Deployment,
  AnalyticsSummary,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  StrategicAlignment,
  OperationalBaseline,
} from "../types";

/**
 * Normalizes cadences into monthly values.
 */
export const normalizeToMonthly = (amount: number, cadence: string): number => {
  switch (cadence) {
    case "daily":
```

Repository Audit

```
    return amount * 30.4375;
  case "weekly":
    return amount * 4.345;
  case "biweekly":
    return amount * 2.1725;
  case "monthly":
    return amount;
  case "quarterly":
    return amount / 3;
  case "yearly":
    return amount / 12;
  default:
    return amount;
}
};

/**
 * Calculates monthly structural burn and systemic allocation metrics.
 */
export const calculateBaselineMetrics = (baseline: OperationalBaseline[]) => {
  return baseline.reduce(
    (acc, item) => {
      if (!item.is_active) return acc;
      const monthlyAmount = normalizeToMonthly(item.amount, item.cadence);

      if (item.baseline_type === "expense") {
        acc.monthlyBurn += monthlyAmount;
      } else {
        acc.monthlyAllocation += monthlyAmount;
      }
      return acc;
    },
    { monthlyBurn: 0, monthlyAllocation: 0 },
  );
};

import { summarizeMetadataQuality } from "../finance/metadataQuality";
import { calculateMonthlyInflow } from "../finance/income";
import { calculateGoalProgressPercentage } from "../finance/goals";
import { calculateObjectiveFundingRatio } from "../finance/objectives";

/**
 * Pure, deterministic engine for financial intelligence.
 * Rule: The database is truth. This engine only interprets.
 */

export const calculateTotal = (deployments: Deployment[]): number => {
  return deployments.reduce((sum, d) => sum + Number(d.amount), 0);
};

export const calculateAccountTotal = (accounts: Account[]): number => {
  return accounts.reduce((sum, a) => sum + Number(a.current_balance), 0);
};

export const calculateLiabilityTotal = (liabilities: Liability[]): number => {
  return liabilities.reduce((sum, l) => sum + Number(l.outstanding_balance), 0);
};

export const calculateIncomeMetrics = (streams: IncomeStream[]) => {
  const result = streams.reduce(
    (acc, stream) => {
      const monthly = calculateMonthlyInflow(stream);
      acc.total += monthly;
      if (stream.is_recurring) {
        acc.recurring += monthly;
      }
    },
    { total: 0, recurring: 0 },
  );
  return result;
};
```

Repository Audit

```
    } else {
      acc.irregular += monthly;
    }
    acc.concentration[stream.income_type] =
      (acc.concentration[stream.income_type] || 0) + monthly;

    // Group by origin source (income_name) for concentration risk analysis
    acc.sourceTotals[stream.income_name] =
      (acc.sourceTotals[stream.income_name] || 0) + monthly;

    return acc;
  },
  {
    total: 0,
    recurring: 0,
    irregular: 0,
    concentration: {} as Record<string, number>,
    sourceTotals: {} as Record<string, number>,
  },
);

const highestSourceSum = Math.max(0, ...Object.values(result.sourceTotals));
const maxRatio = result.total > 0 ? highestSourceSum / result.total : 0;

return {
  total: result.total,
  recurring: result.recurring,
  irregular: result.irregular,
  concentration: result.concentration,
  maxRatio,
};
};

export const calculateGoalMetrics = (
  goals: FinancialGoal[],
  objectives: StrategicObjective[] = [],
) => {
  const goalStats = goals.reduce(
    (acc, goal) => {
      acc.totalTargets += Number(goal.target_amount);
      acc.totalProgress += Number(goal.current_progress);
      if (goal.priority === "critical" && goal.status === "active") {
        acc.criticalCount += 1;
      }
      acc.progressSum += calculateGoalProgressPercentage(goal);
      return acc;
    },
    {
      totalTargets: 0,
      totalProgress: 0,
      criticalCount: 0,
      progressSum: 0,
    },
  );

  return objectives.reduce((acc, obj) => {
    acc.totalTargets += Number(obj.target_amount);
    acc.totalProgress += Number(obj.current_amount);
    if (obj.priority_level === "critical" && obj.status === "active") {
      acc.criticalCount += 1;
    }
    acc.progressSum += calculateObjectiveFundingRatio(obj);
    return acc;
  }, goalStats);
```

Repository Audit

```
};

export const calculateAverage = (deployments: Deployment[]): number => {
  if (deployments.length === 0) return 0;
  return calculateTotal(deployments) / deployments.length;
};

/**
 * Calculates burn rate over a specific window (in days).
 * Defaulting to a 30-day window for normalized metrics.
 */
export const calculateBurnRate = (
  deployments: Deployment[],
  windowDays: number = 30,
): number => {
  if (windowDays <= 0) return 0;
  const total = calculateTotal(deployments);
  return total / windowDays;
};

/**
 * Projects runway based on deterministic liquidity and income offset.
 * Correct Formula: Runway = balance / (dailyBurnRate - (monthlyIncome / 30))
 * Note: Returns days. Returns null if adjusted burn is <= 0 (stable state).
 */
export const projectRunway = (
  balance: number,
  dailyBurnRate: number,
  monthlyIncome: number = 0,
  netWorth: number = 0,
): number | null => {
  const adjustedDailyBurn = dailyBurnRate - monthlyIncome / 30;

  // If net worth is negative, the structural foundation is critical regardless of cash flow
  if (netWorth < 0) return 0;

  if (adjustedDailyBurn <= 0) return null;
  if (balance <= 0) return 0;

  return balance / adjustedDailyBurn;
};

/**
 * Aggregates deployments into categories.
 */
export const getCategoryBreakdown = (
  deployments: Deployment[],
): Record<string, number> => {
  return deployments.reduce(
    (acc, d) => {
      const cat = d.category || "Unclassified";
      acc[cat] = (acc[cat] || 0) + Number(d.amount);
      return acc;
    },
    {} as Record<string, number>,
  );
};

/**
 * Deterministic Strategic Alignment Logic
 */
export function calculateStrategicAlignment(
  objectives: StrategicObjective[],
```

Repository Audit

```
deployments: Deployment[],
liquidity: number,
incomeTotal: number,
burnRate: number,
liabilities: Liability[],
baseline: OperationalBaseline[] = [],
): StrategicAlignment {
  const fundingRatios: Record<string, number> = {};
  const velocity: Record<
    string,
    "stable" | "accelerating" | "stagnant" | "regressing"
  > = {};
  const objectiveStarvationSignals: string[] = [];
  const strategicAllocationSignals: string[] = [];
  const now = new Date();

  // 1. Funding Ratios & Starvation
  objectives.forEach((obj) => {
    const ratio = calculateObjectiveFundingRatio(obj);
    fundingRatios[obj.id] = ratio;

    const ageInDays = Math.max(
      1,
      (now.getTime() - new Date(obj.created_at).getTime()) /
        (1000 * 60 * 60 * 24),
    );

    // Starvation detection
    if (obj.status === "active" && ratio < 10 && ageInDays > 90) {
      objectiveStarvationSignals.push(
        `${obj.objective_name} objective has remained below 10% funded for ${Math.floor(ageInDays)} days.`
      );
    }

    // Velocity (Simplified deterministic approach based on age)
    const dailyRate = obj.current_amount / ageInDays;
    if (dailyRate === 0) {
      velocity[obj.id] = "stagnant";
    } else if (ratio >= 100) {
      velocity[obj.id] = "stable"; // Completed
    } else {
      velocity[obj.id] = "stable"; // Default for now as we don't have historical delta
    }
  });

  // 2. Strategic Allocation Awareness
  const categories = getCategoryBreakdown(deployments);
  const leakage = categories["Leakage"] || 0;
  const assets = categories["Asset"] || 0;
  const hasActiveAccumulation = objectives.some(
    (o) =>
      o.status === "active" &&
      (o.objective_type === "liquidity" ||
        o.objective_type === "emergency_reserve"),
  );

  if (hasActiveAccumulation && leakage > assets) {
    strategicAllocationSignals.push(
      "Leakage deployments exceeded Asset deployments during an active liquidity accumulation objective."
    );
  }

  // 3. Liquidity Sufficiency
  const criticalObjectives = objectives.filter(
```

Repository Audit

```
(o) => o.priority_level === "critical" && o.status === "active",
);
const totalCriticalTarget = criticalObjectives.reduce(
  (sum, o) => sum + (o.target_amount - o.current_amount),
  0,
);
const isSufficient = liquidity >= totalCriticalTarget;
const shortfall = Math.max(0, totalCriticalTarget - liquidity);

const liquiditySufficiency = {
  isSufficient,
  message: isSufficient
    ? "Current liquidity structure sufficiently supports short-term obligations."
    : `Emergency reserve target exceeds current liquidity capacity by ${shortfall.toLocaleString()} KSh.`,
  shortfall,
};

// 4. Alignment Pressure (Weighted Scoring 0-100)
let pressureScore = 0;

// Weight: Leakage vs Assets (up to 25 points)
if (leakage > assets) pressureScore += 25;
else if (leakage > 0) pressureScore += 10;

// Weight: Liquidity Sufficiency (up to 30 points)
if (!isSufficient) pressureScore += 30;

// Weight: Starvation (up to 20 points)
if (objectiveStarvationSignals.length > 0) pressureScore += 20;

// Weight: High Liabilities (up to 25 points)
const totalLiabilities = liabilities.reduce(
  (sum, l) => sum + Number(l.outstanding_balance),
  0,
);
if (totalLiabilities > liquidity * 2) pressureScore += 25;

return {
  fundingRatios,
  velocity,
  alignmentPressure: Math.min(100, pressureScore),
  liquiditySufficiency,
  objectiveStarvationSignals,
  strategicAllocationSignals,
};
}

/**
 * Main entry point for generating a full analytics context.
 */
export const generateSummary = (
  deployments: Deployment[],
  liquidity: number = 0,
  accounts: Account[] = [],
  liabilities: Liability[] = [],
  incomeStreams: IncomeStream[] = [],
  goals: FinancialGoal[] = [],
  objectives: StrategicObjective[] = [],
  baseline: OperationalBaseline[] = [],
): AnalyticsSummary => {
  const total = calculateTotal(deployments);
  const baselineMetrics = calculateBaselineMetrics(baseline);

  // Total daily burn = (Deployment Burn) + (Baseline Burn / 30)
```

Repository Audit

```
const deploymentBurn = calculateBurnRate(deployments);
const baselineDailyBurn = baselineMetrics.monthlyBurn / 30;
const burnRate = deploymentBurn + baselineDailyBurn;

const totalAssets = calculateAccountTotal(accounts);
const totalLiabilities = calculateLiabilityTotal(liabilities);
const netWorth = totalAssets - totalLiabilities;

const income = calculateIncomeMetrics(incomeStreams);
const goalMetrics = calculateGoalMetrics(goals, objectives);

const strategicAlignment = calculateStrategicAlignment(
  objectives,
  deployments,
  liquidity,
  income.total,
  burnRate,
  liabilities,
  baseline,
);

const totalIntentionsCount = goals.length + objectives.length;

return {
  totalDeployed: total,
  averageDeployment: calculateAverage(deployments),
  dailyBurnRate: burnRate,
  runwayDays: projectRunway(liquidity, burnRate, income.total, netWorth),
  categoryBreakdown: getCategoryBreakdown(deployments),
  deploymentCount: deployments.length,
  metadataQuality: summarizeMetadataQuality(deployments),
  // Liability System v1
  totalLiabilities,
  totalAssets,
  netWorth,
  liquidity,
  // Income Engine v1
  totalMonthlyIncome: income.total,
  recurringIncome: income.recurring,
  irregularIncome: income.irregular,
  incomeConcentration: income.concentration,
  maxIncomeConcentrationRatio: income.maxRatio,
  adjustedDailyBurn: Math.max(0, burnRate - income.total / 30),
  // Goal System v1
  totalStrategicTargets: goalMetrics.totalTargets,
  totalCurrentProgress: goalMetrics.totalProgress,
  averageGoalProgress:
    totalIntentionsCount > 0
      ? goalMetrics.progressSum / totalIntentionsCount
      : 0,
  criticalGoalCount: goalMetrics.criticalCount,
  fundingGap: Math.max(
    0,
    goalMetrics.totalTargets - goalMetrics.totalProgress,
  ),
  // Strategic Objectives v1
  strategicAlignment,
  // Operational Baseline v1
  totalStructuralMonthlyBurn: baselineMetrics.monthlyBurn,
  totalSystemicMonthlyAllocation: baselineMetrics.monthlyAllocation,
};
};
```

Repository Audit

File: src/lib/analytics/index.ts

```
export * from "./types";
export * from "./engine";
```

File: src/lib/analytics/strategic.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { calculateStrategicAlignment } from "./engine";
import type { Deployment, StrategicObjective, Liability } from "./types";

test("strategic alignment funding ratios are deterministic", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Emergency Fund",
      objective_type: "emergency_reserve",
      target_amount: 1000,
      current_amount: 500,
      target_date: null,
      priority_level: "critical",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ],
  ];

  const alignment = calculateStrategicAlignment(objectives, [], 0, 0, 0, []);
  assert.strictEqual(alignment.fundingRatios["obj1"], 50);
});

test("alignment pressure scoring reflects structural conflict", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Liquidity",
      objective_type: "liquidity",
      target_amount: 1000,
      current_amount: 0,
      target_date: null,
      priority_level: "critical",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ],
  ];

  const deployments: Deployment[] = [
    {
      id: "d1",
      amount: 500,
      title: "Bad Spending",
      category: "Leakage",
      created_at: new Date().toISOString(),
    },
  ],
  ];

  const liabilities: Liability[] = [
    {
```

Repository Audit

```
    id: "l1",
    user_id: "u1",
    liability_name: "Debt",
    liability_type: "personal_loan",
    outstanding_balance: 10000,
    created_at: "",
    updated_at: "",
    interest_rate: 0,
    minimum_payment: 0,
    due_date: null,
    currency: "KSh",
  },
];

// Scenario: High Leakage, No Assets, Insufficient Liquidity, High Liabilities
const alignment = calculateStrategicAlignment(
  objectives,
  deployments,
  1000,
  0,
  0,
  liabilities,
);

// Leakage > Assets (+25)
// !isSufficient (+30) - totalCriticalTarget is 1000, liquidity is 1000. Wait, 1000 >= 1000 is true.
// totalLiabilities (10000) > liquidity * 2 (2000) (+25)
// Score should be at least 50.

assert.ok(alignment.alignmentPressure >= 50);
});

test("liquidity sufficiency detects shortfalls", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Reserve",
      objective_type: "emergency_reserve",
      target_amount: 5000,
      current_amount: 1000,
      target_date: null,
      priority_level: "critical",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ],
};

const alignment = calculateStrategicAlignment(objectives, [], 2000, 0, 0, []);

assert.strictEqual(alignment.liquiditySufficiency.isSufficient, false);
assert.strictEqual(alignment.liquiditySufficiency.shortfall, 2000); // 4000 remaining target - 2000 liquidity
});

test("objective starvation detection works for old stagnant goals", () => {
  const sixMonthsAgo = new Date();
  sixMonthsAgo.setMonth(sixMonthsAgo.getMonth() - 6);

  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Starved Goal",
    },
  ],
};
```

Repository Audit

```
    objective_type: "investment",
    target_amount: 10000,
    current_amount: 100, // 1%
    target_date: null,
    priority_level: "high",
    status: "active",
    created_at: sixMonthsAgo.toISOString(),
    updated_at: sixMonthsAgo.toISOString(),
  },
];

const alignment = calculateStrategicAlignment(objectives, [], 0, 0, 0, []);

assert.ok(alignment.objectiveStarvationSignals.length > 0);
assert.ok(alignment.objectiveStarvationSignals[0].includes("Starved Goal"));
});

test("strategic allocation awareness detects leakage conflict", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Build Cash",
      objective_type: "liquidity",
      target_amount: 1000,
      current_amount: 0,
      target_date: null,
      priority_level: "high",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ],
];

const deployments: Deployment[] = [
  {
    id: "d1",
    amount: 1000,
    title: "Leakage",
    category: "Leakage",
    created_at: new Date().toISOString(),
  },
  {
    id: "d2",
    amount: 500,
    title: "Asset",
    category: "Asset",
    created_at: new Date().toISOString(),
  },
];

const alignment = calculateStrategicAlignment(
  objectives,
  deployments,
  0,
  0,
  0,
  [],
);

assert.ok(alignment.strategicAllocationSignals.length > 0);
assert.ok(
  alignment.strategicAllocationSignals[0].includes(
    "Leakage deployments exceeded Asset deployments",
  ),
);
```

Repository Audit

```
    ),  
  );  
});
```

File: src/lib/analytics/types.ts

```
import { MetadataQualitySummary } from "../finance/metadataQuality";  
import { DeploymentAdvancedContext } from "../finance/deploymentContext";  
import { Account } from "../finance/accounts";  
import { Liability } from "../finance/liabilities";  
import { IncomeStream } from "../finance/income";  
import { FinancialGoal } from "../finance/goals";  
import { StrategicObjective } from "../finance/objectives";  
  
export type { Account, AccountType } from "../finance/accounts";  
export type { Liability, LiabilityType } from "../finance/liabilities";  
export type { IncomeStream, IncomeType } from "../finance/income";  
export type {  
  FinancialGoal,  
  GoalType,  
  GoalPriority,  
  GoalStatus,  
} from "../finance/goals";  
export type {  
  StrategicObjective,  
  ObjectiveType,  
  ObjectivePriority,  
  ObjectiveStatus,  
} from "../finance/objectives";  
  
export type BaselineCadence =  
  | "daily"  
  | "weekly"  
  | "biweekly"  
  | "monthly"  
  | "quarterly"  
  | "yearly";  
  
export type BaselineType = "expense" | "allocation";  
  
export interface OperationalBaseline {  
  id: string;  
  user_id: string;  
  title: string;  
  amount: number;  
  category: string;  
  cadence: BaselineCadence;  
  baseline_type: BaselineType;  
  is_active: boolean;  
  notes?: string | null;  
  created_at: string;  
  updated_at: string;  
  deleted_at?: string | null;  
}  
  
export interface Deployment {  
  id: string;  
  amount: number;  
  created_at: string;  
  category?: string | null;  
  title: string;  
  advanced_context?: DeploymentAdvancedContext | null;  
  account_id?: string | null;
```

Repository Audit

```
    deleted_at?: string | null;
  }

export interface StrategicAlignment {
  fundingRatios: Record<string, number>; // objective_id -> ratio (0-100)
  alignmentPressure: number; // 0-100 deterministic score
  liquiditySufficiency: {
    isSufficient: boolean;
    message: string;
    shortfall: number;
  };
  objectiveStarvationSignals: string[];
  strategicAllocationSignals: string[];
  velocity: Record<
    string,
    "stable" | "accelerating" | "stagnant" | "regressing"
  >;
}

export interface AnalyticsSummary {
  totalDeployed: number;
  averageDeployment: number;
  dailyBurnRate: number;
  runwayDays: number | null;
  categoryBreakdown: Record<string, number>;
  deploymentCount: number;
  metadataQuality: MetadataQualitySummary;
  // Liability System v1
  totalLiabilities: number;
  netWorth: number;
  totalAssets: number;
  liquidity: number;
  // Income Engine v1
  totalMonthlyIncome: number;
  recurringIncome: number;
  irregularIncome: number;
  incomeConcentration: Record<string, number>;
  maxIncomeConcentrationRatio: number;
  adjustedDailyBurn: number;
  // Goal System v1
  totalStrategicTargets: number;
  totalCurrentProgress: number;
  averageGoalProgress: number;
  criticalGoalCount: number;
  fundingGap: number;
  // Strategic Objectives v1
  strategicAlignment: StrategicAlignment;
  // Operational Baseline v1
  totalStructuralMonthlyBurn: number;
  totalSystemicMonthlyAllocation: number;
}
```

File: src/lib/context/behavioralFlags.ts

```
import { TrendState } from "../contextTypes";

/**
 * Identifies discrete behavioral signals for AI interpretation.
 * Objective: High signal, low token usage.
 */

export const generateBehavioralFlags = (params: {
  burnTrend: TrendState;
```

Repository Audit

```
    volatility: number;
    concentration: number;
    runwayDays: number | null;
    dominantCategory: string;
  }): string[] => {
    const flags: string[] = [];

    // 1. Burn Signals
    if (params.burnTrend === "increasing") flags.push("accelerating_burn");
    if (params.burnTrend === "decreasing") flags.push("burn_optimization_detected");

    // 2. Consistency Signals
    if (params.volatility > 0.6) flags.push("erratic_spending_pattern");
    if (params.volatility < 0.2 && params.volatility > 0) flags.push("high_operational_discipline");

    // 3. Allocation Signals
    if (params.concentration > 0.7) flags.push(`heavy_concentration:${params.dominantCategory.toLowerCase()}`);
    if (params.concentration < 0.3) flags.push("fragmented_capital_allocation");

    // 4. Criticality Signals
    if (params.runwayDays !== null && params.runwayDays < 30) flags.push("critical_runway_warning");
    if (params.runwayDays !== null && params.runwayDays > 180) flags.push("extended_operational_security");

    return flags;
  };

/**
 * Analyzes category concentration.
 * Returns a score between 0 (even spread) and 1 (all in one category).
 */
export const calculateConcentrationScore = (distribution: Record<string, number>): number => {
  const values = Object.values(distribution);
  if (values.length === 0) return 0;
  if (values.length === 1) return 1;

  // Use Herfindahl-Hirschman Index (HHI) approach for concentration
  // Sum of squares of shares
  const hhi = values.reduce((sum, val) => sum + Math.pow(val, 2), 0);
  return hhi;
};
```

File: src/lib/context/buildBehavioralContext.ts

```
import { BehavioralContext, ContextInput } from "../contextTypes";

import {
  calculateTrendState,
  calculateVolatility,
  inferDiscipline,
} from "../trendAnalysis";
import {
  generateBehavioralFlags,
  calculateConcentrationScore,
} from "../behavioralFlags";
import { isValidCategory } from "../../finance/taxonomy";

/**
 * Axiom Behavioral Context Builder
 * Transforms analytics signals into a stable behavioral state for Kairos.
 */
export const buildBehavioralContext = (
  input: ContextInput,
  deploymentAmounts: number[],
```

Repository Audit

```
): BehavioralContext => {
  const { currentAnalytics, historicalMetrics } = input;

  // 1. Calculate Trends
  const burnTrend = calculateTrendState(
    currentAnalytics.dailyBurnRate,
    historicalMetrics?.previousBurnRate || 0,
  );

  const runwayDelta =
    currentAnalytics.runwayDays !== null &&
    historicalMetrics?.previousRunwayDays !== undefined
      ? currentAnalytics.runwayDays -
        (historicalMetrics.previousRunwayDays || 0)
      : 0;

  // 2. Allocation Analysis
  // 2. Allocation Analysis
  const total = currentAnalytics.totalDeployed || 1;
  const distribution: Record<string, number> = {};
  let dominantCategory = "None";
  let maxAmt = -1;

  let unclassifiedTotal = 0;

  Object.entries(currentAnalytics.categoryBreakdown).forEach(([cat, amt]) => {
    if (!isValidCategory(cat)) {
      unclassifiedTotal += amt;
    } else {
      distribution[cat] = amt / total;
      if (amt > maxAmt) {
        maxAmt = amt;
        dominantCategory = cat;
      }
    }
  });

  // Add the merged 'Unclassified' group
  if (unclassifiedTotal > 0) {
    distribution["Unclassified"] = unclassifiedTotal / total;
    if (unclassifiedTotal > maxAmt) {
      dominantCategory = "Unclassified";
    }
  }

  const concentrationScore = calculateConcentrationScore(distribution);
  const volatility = calculateVolatility(deploymentAmounts);

  // 3. Efficiency Calculation (Heuristic)
  // Starts at 100, penalized by volatility, accelerating burn, and low runway.
  let efficiency = 100;
  efficiency -= volatility * 60; // Much more sensitive to erratic spending
  if (burnTrend === "increasing") efficiency -= 15;
  if (currentAnalytics.runwayDays && currentAnalytics.runwayDays < 90)
    efficiency -= 40; // Aggressive penalty for low runway

  // 4. Status Determination

  // 4. Status Determination
  let runwayStatus: "healthy" | "concerning" | "critical" = "healthy";
  if (currentAnalytics.runwayDays !== null) {
    if (currentAnalytics.runwayDays < 30) runwayStatus = "critical";
    else if (currentAnalytics.runwayDays < 90) runwayStatus = "concerning";
  }
}
```

Repository Audit

```
return {
  generatedAt: new Date().toISOString(),
  deploymentCount: currentAnalytics.deploymentCount,
  capitalEfficiencyScore: Math.round(Math.max(efficiency, 0)),

  netWorth: currentAnalytics.netWorth,
  liquidity: currentAnalytics.liquidity,
  totalMonthlyIncome: currentAnalytics.totalMonthlyIncome,
  maxIncomeConcentrationRatio: currentAnalytics.maxIncomeConcentrationRatio,

  strategicAlignment: currentAnalytics.strategicAlignment,

  burnRate: {
    current: currentAnalytics.dailyBurnRate,
    trend: burnTrend,
    monthlyProjection: currentAnalytics.dailyBurnRate * 30,
  },

  runway: {
    currentDays: currentAnalytics.runwayDays,
    deltaDays: runwayDelta,
    status: runwayStatus,
  },

  allocation: {
    dominantCategory,
    categoryDistribution: distribution,
    concentrationScore,
  },

  volatilityScore: volatility,

  metadataQuality: currentAnalytics.metadataQuality,

  disciplineTrend: inferDiscipline(burnTrend, runwayDelta),

  behavioralFlags: generateBehavioralFlags({
    burnTrend,
    volatility,
    concentration: concentrationScore,
    runwayDays: currentAnalytics.runwayDays,
    dominantCategory,
  })),
};

/**
 * Context Compression
 * Strips metadata to optimize for LLM token usage while preserving signal.
 */
export const compressContextForAI = (context: BehavioralContext): string => {
  return JSON.stringify({
    score: context.capitalEfficiencyScore,
    burn: `${context.burnRate.current}/day (${context.burnRate.trend})`,
    runway: `${context.runway.currentDays}d (${context.runway.status})`,
    allocation: `${context.allocation.dominantCategory} (conc:
    ${context.allocation.concentrationScore.toFixed(2)})`,
    flags: context.behavioralFlags.join(", "),
    discipline: context.disciplineTrend,
    metadata: `${context.metadataQuality.lowQualityCount}/${context.deploymentCount} low-quality labels`,
  });
};
```

Repository Audit

File: src/lib/context/contextTypes.ts

```
import { StrategicAlignment } from "../analytics/types";
import { MetadataQualitySummary } from "../finance/metadataQuality";

export type TrendState = "increasing" | "stable" | "decreasing";
export type DisciplineState = "improving" | "stable" | "declining";

export interface AllocationState {
  dominantCategory: string;
  categoryDistribution: Record<string, number>; // Percentage based (0-1)
  concentrationScore: number; // 0 to 1, where 1 is total concentration in one category
}

export interface BehavioralContext {
  generatedAt: string;
  deploymentCount: number;

  // High-level aggregate metric (0-100)
  capitalEfficiencyScore: number;

  netWorth: number;
  liquidity: number;
  totalMonthlyIncome: number;
  maxIncomeConcentrationRatio: number;

  strategicAlignment: StrategicAlignment;

  burnRate: {
    current: number;
    trend: TrendState;
    monthlyProjection: number;
  };

  runway: {
    currentDays: number | null;
    deltaDays: number; // Change from previous context
    status: "healthy" | "concerning" | "critical";
  };

  allocation: AllocationState;

  // Discrete behavioral markers
  behavioralFlags: string[];

  // Measure of spending consistency
  volatilityScore: number; // 0 to 1, higher is more erratic

  metadataQuality: MetadataQualitySummary;

  disciplineTrend: DisciplineState;
}

/**
 * Input required for the context builder.
 * Represents the current state vs a previous snapshot.
 */
export interface ContextInput {
  currentAnalytics: {
    totalDeployed: number;
    dailyBurnRate: number;
    runwayDays: number | null;
    categoryBreakdown: Record<string, number>;
```

Repository Audit

```
    deploymentCount: number;
    metadataQuality: MetadataQualitySummary;
    netWorth: number;
    liquidity: number;
    totalMonthlyIncome: number;
    maxIncomeConcentrationRatio: number;
    strategicAlignment: StrategicAlignment;
  };
  historicalMetrics?: {
    previousBurnRate: number;
    previousRunwayDays: number | null;
    previousTotalDeployed: number;
  };
}
```

File: src/lib/context/trendAnalysis.ts

```
import { TrendState, DisciplineState } from "../contextTypes";

/**
 * Determines the trend direction between two values.
 * Threshold (0.05) prevents noise from triggering trend shifts.
 */
export const calculateTrendState = (
  current: number,
  previous: number,
  threshold: number = 0.05
): TrendState => {
  if (!previous || previous === 0) return "stable";

  const change = (current - previous) / previous;

  if (change > threshold) return "increasing";
  if (change < -threshold) return "decreasing";
  return "stable";
};

/**
 * Calculates a volatility score (0-1) based on the variance of deployment sizes.
 * High variance = High volatility (erratic spending).
 */
export const calculateVolatility = (amounts: number[]): number => {
  if (amounts.length < 2) return 0;

  const mean = amounts.reduce((a, b) => a + b, 0) / amounts.length;
  const variance = amounts.reduce((a, b) => a + Math.pow(b - mean, 2), 0) / amounts.length;
  const stdDev = Math.sqrt(variance);

  // Normalize volatility relative to the mean.
  // A standard deviation equal to the mean results in a score of 0.5.
  const score = stdDev / (mean + stdDev);
  return Math.min(Math.max(score, 0), 1);
};

/**
 * Infers discipline trend by observing burn rate vs total deployment.
 */
export const inferDiscipline = (
  burnTrend: TrendState,
  runwayDelta: number
): DisciplineState => {
  if (burnTrend === "decreasing" && runwayDelta > 0) return "improving";
  if (burnTrend === "increasing" && runwayDelta < 0) return "declining";
}
```

Repository Audit

```
    return "stable";  
};
```

File: src/lib/db/accounts.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";  
import { validateAccount, type AccountType } from "../finance/accounts";  
  
export async function getAccounts(supabase: SupabaseClient) {  
  const { data, error } = await supabase  
    .from("accounts")  
    .select("*")  
    .is("deleted_at", null)  
    .order("account_name", { ascending: true });  
  
  if (error) throw error;  
  return data;  
}  
  
export async function createAccount(  
  supabase: SupabaseClient,  
  userId: string,  
  data: {  
    account_name: string;  
    account_type: string;  
    current_balance: number;  
    institution?: string;  
    currency?: string;  
    deleted_at?: string | null;  
  },  
) {  
  const validated = validateAccount({  
    account_name: data.account_name,  
    account_type: data.account_type,  
    current_balance: data.current_balance,  
  });  
  
  const { data: account, error } = await supabase  
    .from("accounts")  
    .insert({  
      user_id: userId,  
      account_name: validated.account_name,  
      account_type: validated.account_type,  
      current_balance: validated.current_balance,  
      institution: data.institution,  
      currency: data.currency || "KSh",  
    })  
    .select()  
    .single();  
  
  if (error) throw error;  
  return account;  
}  
  
export async function updateAccount(  
  supabase: SupabaseClient,  
  id: string,  
  userId: string,  
  updates: {  
    account_name?: string;  
    account_type?: string;  
    current_balance?: number;  
    institution?: string;
```

Repository Audit

```
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.account_name !== undefined ||
    updates.account_type !== undefined ||
    updates.current_balance !== undefined
  ) {
    // Basic validation for what's provided
    if (
      updates.account_name !== undefined &&
      updates.account_name.trim().length === 0
    ) {
      throw new Error("Account name is required.");
    }
    // Note: We don't use full validateAccount here to allow partial updates
  }

  const { data, error } = await supabase
    .from("accounts")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteAccount(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("accounts")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

File: src/lib/db/audit.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";

export type AuditRecordType = "deployment" | "income" | "liability";

export interface AuditRecord {
  id: string;
  type: AuditRecordType;
  name: string;
  amount: number;
  cadence?: string;
  deleted_at: string;
  original_created_at: string;
}
```

Repository Audit

```
/**
 * Retrieves all soft-deleted records from the ledger tables (deployments, income_streams, liabilities).
 * Returns a unified, chronologically sorted timeline of deletions.
 */
export async function getDeletedLedgerRecords(
  supabase: SupabaseClient,
  userId: string,
): Promise<AuditRecord[]> {
  const [deployments, incomeStreams, liabilities] = await Promise.all([
    supabase
      .from("deployments")
      .select("id, title, amount, deleted_at, created_at")
      .eq("user_id", userId)
      .not("deleted_at", "is", null),
    supabase
      .from("income_streams")
      .select("id, income_name, amount, cadence, deleted_at, created_at")
      .eq("user_id", userId)
      .not("deleted_at", "is", null),
    supabase
      .from("liabilities")
      .select("id, liability_name, outstanding_balance, deleted_at, created_at")
      .eq("user_id", userId)
      .not("deleted_at", "is", null),
  ]);

  if (deployments.error) throw deployments.error;
  if (incomeStreams.error) throw incomeStreams.error;
  if (liabilities.error) throw liabilities.error;

  const normalizedDeployments: AuditRecord[] = (deployments.data || []).map(
    (d) => ({
      id: d.id,
      type: "deployment",
      name: d.title,
      amount: d.amount,
      deleted_at: d.deleted_at!,
      original_created_at: d.created_at,
    })),
  );

  const normalizedIncome: AuditRecord[] = (incomeStreams.data || []).map(
    (i) => ({
      id: i.id,
      type: "income",
      name: i.income_name,
      amount: i.amount,
      cadence: i.cadence,
      deleted_at: i.deleted_at!,
      original_created_at: i.created_at,
    })),
  );

  const normalizedLiabilities: AuditRecord[] = (liabilities.data || []).map(
    (l) => ({
      id: l.id,
      type: "liability",
      name: l.liability_name,
      amount: l.outstanding_balance,
      deleted_at: l.deleted_at!,
      original_created_at: l.created_at,
    })),
  );
```

Repository Audit

```
return [
  ...normalizedDeployments,
  ...normalizedIncome,
  ...normalizedLiabilities,
].sort(
  (a, b) =>
    new Date(b.deleted_at).getTime() - new Date(a.deleted_at).getTime(),
);
}
```

File: src/lib/db/baseline.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { OperationalBaseline } from "../analytics/types";

export async function getOperationalBaseline(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("operational_baseline")
    .select("*")
    .is("deleted_at", null)
    .order("created_at", { ascending: true });

  if (error) throw error;
  return data as OperationalBaseline[];
}

export async function createOperationalBaseline(
  supabase: SupabaseClient,
  userId: string,
  data: Omit<
    OperationalBaseline,
    "id" | "user_id" | "created_at" | "updated_at" | "deleted_at"
  >,
) {
  const { data: baseline, error } = await supabase
    .from("operational_baseline")
    .insert({
      user_id: userId,
      ...data,
    })
    .select()
    .single();

  if (error) throw error;
  return baseline as OperationalBaseline;
}

export async function updateOperationalBaseline(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: Partial<
    Omit<
      OperationalBaseline,
      "id" | "user_id" | "created_at" | "updated_at" | "deleted_at"
    >
  >,
) {
  const { data: baseline, error } = await supabase
    .from("operational_baseline")
    .update(updates)
    .eq("id", id)
    .eq("user_id", userId)
```

Repository Audit

```
        .select()
        .single();

    if (error) throw error;
    return baseline as OperationalBaseline;
}

export async function deleteOperationalBaseline(
    supabase: SupabaseClient,
    id: string,
    userId: string,
) {
    const { error } = await supabase
        .from("operational_baseline")
        .update({ deleted_at: new Date().toISOString() })
        .eq("id", id)
        .eq("user_id", userId);

    if (error) throw error;
    return true;
}
```

File: src/lib/db/deployments.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateDeployment } from "../finance/validateDeployment";
import {
    hasDeploymentAdvancedContext,
    type DeploymentAdvancedContextInput,
} from "../finance/deploymentContext";

function isMissingAdvancedContextColumn(error: { message?: string }) {
    return Boolean(error.message?.toLowerCase().includes("advanced_context"));
}

export async function getDeployments(supabase: SupabaseClient) {
    const { data, error } = await supabase
        .from("deployments")
        .select("*")
        .is("deleted_at", null)
        .order("created_at", { ascending: false });

    if (error) throw error;
    return data;
}

/**
 * Creates a new deployment record.
 * Guaranteed to pass through the Taxonomy Enforcement Gate.
 */
export async function createDeployment(
    supabase: SupabaseClient,
    title: string,
    amount: number,
    userId: string,
    category: string,
    impactScore: number = 0,
    advancedContext: DeploymentAdvancedContextInput = {},
    accountId?: string,
) {
    // 1. Pass through Taxonomy Enforcement Gate
    const validated = validateDeployment({
        title,
```

Repository Audit

```
    amount,
    category,
    impactScore,
    advancedContext,
  });

// 2. Database Write (Verified Truth)
const deploymentRecord = {
  title: validated.title,
  amount: validated.amount,
  user_id: userId,
  category: validated.category,
  impact_score: validated.impactScore,
  account_id: accountId || null,
  ...(hasDeploymentAdvancedContext(validated.advancedContext)
    ? { advanced_context: validated.advancedContext }
    : {}),
};

const { data, error } = await supabase
  .from("deployments")
  .insert(deploymentRecord);

if (error) {
  if (
    hasDeploymentAdvancedContext(validated.advancedContext) &&
    isMissingAdvancedContextColumn(error)
  ) {
    throw new Error(
      "Advanced Context schema unavailable. Apply supabase-setup.sql before saving advanced deployment metadata.",
    );
  }

  throw error;
}
return data;
}

/**
 * Updates an existing deployment record.
 * Enforces partial taxonomy validation for updated fields.
 */
export async function updateDeployment(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    title?: string;
    amount?: number;
    category?: string;
    impact_score?: number;
    advancedContext?: DeploymentAdvancedContextInput;
    accountId?: string;
  },
) {
  // 1. Enforcement for provided fields
  const dbUpdates: Record<string, unknown> = {};

  if (updates.accountId !== undefined) {
    dbUpdates.account_id = updates.accountId || null;
  }

  const shouldValidate =
```

Repository Audit

```
updates.title !== undefined ||
updates.amount !== undefined ||
updates.category !== undefined ||
updates.advancedContext !== undefined ||
updates.impact_score !== undefined;

if (shouldValidate) {
  // We simulate a full input to reuse the Gate logic,
  // but only use the validated fields for the update.
  const validated = validateDeployment({
    title: updates.title ?? "Temporary Title",
    amount: updates.amount ?? 1,
    category: updates.category ?? "Leakage", // Explicit default for gate reuse
    impactScore: updates.impact_score,
    advancedContext: updates.advancedContext,
  });

  if (updates.title !== undefined) dbUpdates.title = validated.title;
  if (updates.amount !== undefined) dbUpdates.amount = validated.amount;
  if (updates.category !== undefined) dbUpdates.category = validated.category;
  if (updates.impact_score !== undefined)
    dbUpdates.impact_score = validated.impactScore;
  if (updates.advancedContext !== undefined)
    dbUpdates.advanced_context = validated.advancedContext;
}

// 2. Database Update
const { data, error } = await supabase
  .from("deployments")
  .update(dbUpdates)
  .eq("id", id)
  .eq("user_id", userId);

if (error) {
  if (
    updates.advancedContext !== undefined &&
    isMissingAdvancedContextColumn(error)
  ) {
    throw new Error(
      "Advanced Context schema unavailable. Apply supabase-setup.sql before saving advanced deployment metadata.",
    );
  }

  throw error;
}
return data;
}

export async function deleteDeployment(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("deployments")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

Repository Audit

File: src/lib/db/goals.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateGoal } from "../finance/goals";

export async function getGoals(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("financial_goals")
    .select("*")
    .is("deleted_at", null)
    .order("priority", { ascending: false });

  if (error) throw error;
  return data;
}

export async function createGoal(
  supabase: SupabaseClient,
  userId: string,
  data: {
    goal_name: string;
    goal_type: string;
    target_amount: number;
    current_progress?: number;
    target_date?: string | null;
    priority: string;
    status: string;
    notes?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateGoal({
    goal_name: data.goal_name,
    goal_type: data.goal_type,
    target_amount: data.target_amount,
    current_progress: data.current_progress,
    priority: data.priority,
    status: data.status,
  });

  const { data: goal, error } = await supabase
    .from("financial_goals")
    .insert({
      user_id: userId,
      goal_name: validated.goal_name,
      goal_type: validated.goal_type,
      target_amount: validated.target_amount,
      current_progress: validated.current_progress,
      target_date: data.target_date,
      priority: validated.priority,
      status: validated.status,
      notes: data.notes,
    })
    .select()
    .single();

  if (error) throw error;
  return goal;
}

export async function updateGoal(
  supabase: SupabaseClient,
  id: string,
```

Repository Audit

```
    userId: string,
    updates: {
      goal_name?: string;
      goal_type?: string;
      target_amount?: number;
      current_progress?: number;
      target_date?: string | null;
      priority?: string;
      status?: string;
      notes?: string | null;
      deleted_at?: string | null;
    },
  ) {
    const dbUpdates: Record<string, unknown> = { ...updates };

    if (
      updates.goal_name !== undefined &&
      updates.goal_name.trim().length === 0
    ) {
      throw new Error("Goal name is required.");
    }

    const { data, error } = await supabase
      .from("financial_goals")
      .update(dbUpdates)
      .eq("id", id)
      .eq("user_id", userId)
      .select()
      .single();

    if (error) throw error;
    return data;
  }

export async function deleteGoal(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("financial_goals")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

File: src/lib/db/income.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateIncomeStream } from "../finance/income";

export async function getIncomeStreams(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("income_streams")
    .select("*")
    .is("deleted_at", null)
    .order("income_name", { ascending: true });

  if (error) throw error;
  return data;
}
```

Repository Audit

```
}

export async function createIncomeStream(
  supabase: SupabaseClient,
  userId: string,
  data: {
    income_name: string;
    income_type: string;
    amount: number;
    cadence: string;
    execution_day?: number | null;
    is_recurring?: boolean;
    source?: string;
    currency?: string;
    start_date?: string;
    end_date?: string | null;
    deleted_at?: string | null;
  },
) {
  const validated = validateIncomeStream({
    income_name: data.income_name,
    income_type: data.income_type,
    amount: data.amount,
    cadence: data.cadence,
    execution_day: data.execution_day,
    is_recurring: data.is_recurring,
  });

  const { data: stream, error } = await supabase
    .from("income_streams")
    .insert({
      user_id: userId,
      income_name: validated.income_name,
      income_type: validated.income_type,
      amount: validated.amount,
      cadence: validated.cadence,
      execution_day: validated.execution_day,
      is_recurring: validated.is_recurring,
      source: data.source,
      currency: data.currency || "KSh",
      start_date: data.start_date || new Date().toISOString().split("T")[0],
      end_date: data.end_date,
    })
    .select()
    .single();

  if (error) throw error;
  return stream;
}

export async function createIncomeStreams(
  supabase: SupabaseClient,
  userId: string,
  inputs: {
    income_name: string;
    income_type: string;
    amount: number;
    cadence: string;
    execution_day?: number | null;
    is_recurring?: boolean;
    source?: string;
    currency?: string;
    start_date?: string;
    end_date?: string | null;
  }
```

Repository Audit

```
    deleted_at?: string | null;
  }[],
) {
  const validatedInputs = inputs.map((data) => {
    const validated = validateIncomeStream({
      income_name: data.income_name,
      income_type: data.income_type,
      amount: data.amount,
      cadence: data.cadence,
      execution_day: data.execution_day,
      is_recurring: data.is_recurring,
    });

    return {
      user_id: userId,
      income_name: validated.income_name,
      income_type: validated.income_type,
      amount: validated.amount,
      cadence: validated.cadence,
      execution_day: validated.execution_day,
      is_recurring: validated.is_recurring,
      source: data.source,
      currency: data.currency || "KSh",
      start_date: data.start_date || new Date().toISOString().split("T")[0],
      end_date: data.end_date,
    };
  });

  const { data: streams, error } = await supabase
    .from("income_streams")
    .insert(validatedInputs)
    .select();

  if (error) throw error;
  return streams;
}

export async function updateIncomeStream(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    income_name?: string;
    income_type?: string;
    amount?: number;
    cadence?: string;
    execution_day?: number | null;
    last_executed_at?: string | null;
    is_recurring?: boolean;
    source?: string;
    start_date?: string;
    end_date?: string | null;
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.income_name !== undefined &&
    updates.income_name.trim().length === 0
  ) {
    throw new Error("Income name is required.");
  }
}
```

Repository Audit

```
const { data, error } = await supabase
  .from("income_streams")
  .update(dbUpdates)
  .eq("id", id)
  .eq("user_id", userId)
  .select()
  .single();

if (error) throw error;
return data;
}

export async function deleteIncomeStream(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("income_streams")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

File: src/lib/db/insights.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import type { KairosInsight } from "@lib/ai/kairos";

export async function saveInsight(
  supabase: SupabaseClient,
  insight: KairosInsight,
  userId: string,
) {
  const { data, error } = await supabase.from("kairos_insights").insert({
    user_id: userId,
    type: insight.type,
    category: insight.category,
    confidence: 1.0, // Deterministic logic has absolute confidence
    message: insight.message,
    metadata: {
      related_ids: [],
      metadata_quality: insight.metadataQuality || null,
      severity: insight.severity,
      supporting_signals: insight.supportingSignals,
      runway: insight.runway,
      capital_efficiency: insight.capitalEfficiency,
      is_silent: insight.isSilent,
      timestamp: insight.timestamp,
      source: "server_orchestrator_v5e",
    },
  });

  if (error) {
    console.error("Error saving insight:", error);
    throw error;
  }
  return data;
}
```

Repository Audit

```
export async function getInsights(supabase: SupabaseClient, limit: number = 5) {
  const { data, error } = await supabase
    .from("kairos_insights")
    .select(".*")
    .order("created_at", { ascending: false })
    .limit(limit);

  if (error) {
    console.error("Error fetching insights:", error);
    throw error;
  }
  return data;
}
```

File: src/lib/db/liabilities.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateLiability } from "../finance/liabilities";

export async function getLiabilities(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("liabilities")
    .select(".*")
    .is("deleted_at", null)
    .order("liability_name", { ascending: true });

  if (error) throw error;
  return data;
}

export async function createLiability(
  supabase: SupabaseClient,
  userId: string,
  data: {
    liability_name: string;
    liability_type: string;
    outstanding_balance: number;
    interest_rate?: number;
    minimum_payment?: number;
    due_date?: string | null;
    institution?: string;
    currency?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateLiability({
    liability_name: data.liability_name,
    liability_type: data.liability_type,
    outstanding_balance: data.outstanding_balance,
    interest_rate: data.interest_rate,
    minimum_payment: data.minimum_payment,
  });

  const { data: liability, error } = await supabase
    .from("liabilities")
    .insert({
      user_id: userId,
      liability_name: validated.liability_name,
      liability_type: validated.liability_type,
      outstanding_balance: validated.outstanding_balance,
      interest_rate: validated.interest_rate,
      minimum_payment: validated.minimum_payment,
      due_date: data.due_date,
    })
```

Repository Audit

```
        institution: data.institution,
        currency: data.currency || "KSh",
    })
    .select()
    .single();

    if (error) throw error;
    return liability;
}

export async function updateLiability(
    supabase: SupabaseClient,
    id: string,
    userId: string,
    updates: {
        liability_name?: string;
        liability_type?: string;
        outstanding_balance?: number;
        interest_rate?: number;
        minimum_payment?: number;
        due_date?: string | null;
        institution?: string;
        deleted_at?: string | null;
    },
) {
    const dbUpdates: Record<string, unknown> = { ...updates };

    if (
        updates.liability_name !== undefined &&
        updates.liability_name.trim().length === 0
    ) {
        throw new Error("Liability name is required.");
    }

    const { data, error } = await supabase
        .from("liabilities")
        .update(dbUpdates)
        .eq("id", id)
        .eq("user_id", userId)
        .select()
        .single();

    if (error) throw error;
    return data;
}

export async function deleteLiability(
    supabase: SupabaseClient,
    id: string,
    userId: string,
) {
    const { error } = await supabase
        .from("liabilities")
        .update({ deleted_at: new Date().toISOString() })
        .eq("id", id)
        .eq("user_id", userId);

    if (error) throw error;
    return true;
}
```

File: src/lib/db/objectives.ts

Repository Audit

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateObjective } from "../finance/objectives";

export async function getStrategicObjectives(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("strategic_objectives")
    .select("*")
    .is("deleted_at", null)
    .order("priority_level", { ascending: false })
    .order("created_at", { ascending: false });

  if (error) throw error;
  return data;
}

export async function createStrategicObjective(
  supabase: SupabaseClient,
  userId: string,
  data: {
    objective_name: string;
    objective_type: string;
    target_amount: number;
    current_amount?: number;
    target_date?: string | null;
    priority_level: string;
    status: string;
    notes?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateObjective({
    objective_name: data.objective_name,
    objective_type: data.objective_type,
    target_amount: data.target_amount,
    current_amount: data.current_amount,
    priority_level: data.priority_level,
    status: data.status,
  });

  const { data: objective, error } = await supabase
    .from("strategic_objectives")
    .insert({
      user_id: userId,
      objective_name: validated.objective_name,
      objective_type: validated.objective_type,
      target_amount: validated.target_amount,
      current_amount: validated.current_amount,
      target_date: data.target_date,
      priority_level: validated.priority_level,
      status: validated.status,
      notes: data.notes,
    })
    .select()
    .single();

  if (error) throw error;
  return objective;
}

export async function updateStrategicObjective(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
```

Repository Audit

```
    objective_name?: string;
    objective_type?: string;
    target_amount?: number;
    current_amount?: number;
    target_date?: string | null;
    priority_level?: string;
    status?: string;
    notes?: string | null;
    deleted_at?: string | null;
  },
) {
  // If objective_name is provided, validate it's not empty
  if (
    updates.objective_name !== undefined &&
    updates.objective_name.trim().length === 0
  ) {
    throw new Error("Objective name is required.");
  }

  const { data, error } = await supabase
    .from("strategic_objectives")
    .update(updates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteStrategicObjective(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("strategic_objectives")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

File: src/lib/db/settings.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";

export async function getUserSettings(
  supabase: SupabaseClient,
  userId: string,
) {
  const { data, error } = await supabase
    .from("user_settings")
    .select("*")
    .eq("user_id", userId)
    .maybeSingle();

  if (error) throw error;

  // If no settings exist, create default
```

Repository Audit

```
if (!data) {
  const { data: newData, error: createError } = await supabase
    .from("user_settings")
    .insert({ user_id: userId, total_liquidity: 0 })
    .select()
    .single();

  if (createError) throw createError;
  return newData;
}

return data;
}

export async function updateLiquidity(
  supabase: SupabaseClient,
  userId: string,
  amount: number,
) {
  const { data, error } = await supabase
    .from("user_settings")
    .upsert({
      user_id: userId,
      total_liquidity: amount,
      updated_at: new Date().toISOString(),
    })
    .select()
    .single();

  if (error) throw error;
  return data;
}
```

File: src/lib/db/telemetry.ts

```
import { createClient } from "@lib/supabase/server";

/**
 * TELEMETRY SERVICE (Phase 1)
 * Records insight evaluation events for observability.
 */
export async function logKairosEvaluation(data: {
  ruleId: string;
  severity: string;
  durationMs: number;
}) {
  try {
    let supabase;
    try {
      supabase = await createClient();
    } catch (e) {
      // If we're in a test environment or outside a request scope, cookies() will fail.
      // We silently exit as telemetry is non-critical for these contexts.
      return;
    }

    const {
      data: { user },
    } = await supabase.auth.getUser();
    if (!user) return;

    // Record evaluation event to kairos_insights table.
    // This phase uses the rule registry ID and performance metadata.
```

Repository Audit

```
const { error } = await supabase.from("kairos_insights").insert({
  user_id: user.id,
  rule_id: data.ruleId,
  severity: data.severity,
  evaluation_time_ms: data.durationMs,
});

if (error) {
  console.error("[Telemetry] Error recording evaluation:", error.message);
}
} catch (err) {
  // Observability must be non-blocking; we swallow errors to protect primary flows.
  console.error("[Telemetry] Critical failure in logging:", err);
}
}
```

File: src/lib/finance/accounts.ts

```
export const ACCOUNT_TYPES = [
  { value: "checking", label: "Checking" },
  { value: "savings", label: "Savings" },
  { value: "mobile_money", label: "Mobile Money" },
  { value: "brokerage", label: "Brokerage" },
  { value: "crypto", label: "Crypto" },
  { value: "cash", label: "Cash" },
] as const;

export type AccountType = (typeof ACCOUNT_TYPES)[number]["value"];

export interface Account {
  id: string;
  user_id: string;
  account_name: string;
  account_type: AccountType;
  current_balance: number;
  currency: string;
  institution?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidAccountType(type: string): type is AccountType {
  return ACCOUNT_TYPES.some((t) => t.value === type);
}

export function validateAccount(data: {
  account_name: string;
  account_type: string;
  current_balance: number;
}) {
  if (!data.account_name || data.account_name.trim().length === 0) {
    throw new Error("Account name is required.");
  }

  if (!isValidAccountType(data.account_type)) {
    throw new Error(`Invalid account type: ${data.account_type}`);
  }

  if (isNaN(data.current_balance)) {
    throw new Error("Initial balance must be a number.");
  }
}
```

Repository Audit

```
return {
  account_name: data.account_name.trim(),
  account_type: data.account_type as AccountType,
  current_balance: data.current_balance,
};
}
```

File: src/lib/finance/deploymentContext.ts

```
export const EXPECTED_RETURN_HORIZONS = [
  { value: "short-term", label: "Short-term" },
  { value: "medium-term", label: "Medium-term" },
  { value: "long-term", label: "Long-term" },
] as const;

export type ExpectedReturnHorizon =
  (typeof EXPECTED_RETURN_HORIZONS)[number]["value"];

export interface DeploymentAdvancedContext {
  associatedAccount?: string;
  expectedReturnHorizon?: ExpectedReturnHorizon;
  tags?: string[];
}

export interface DeploymentAdvancedContextInput {
  associatedAccount?: string | null;
  expectedReturnHorizon?: string | null;
  tags?: string | string[] | null;
}

const EXPECTED_RETURN_HORIZON_VALUES = new Set(
  EXPECTED_RETURN_HORIZONS.map((horizon) => horizon.value),
);

export function isExpectedReturnHorizon(
  value: string,
): value is ExpectedReturnHorizon {
  return EXPECTED_RETURN_HORIZON_VALUES.has(value as ExpectedReturnHorizon);
}

function normalizeTags(tags: DeploymentAdvancedContextInput["tags"]) {
  const source = Array.isArray(tags) ? tags.join(",") : tags || "";
  const normalizedTags = source
    .split(",")
    .map((tag) => tag.trim().toLowerCase())
    .filter(Boolean);

  return Array.from(new Set(normalizedTags));
}

export function normalizeDeploymentAdvancedContext(
  input: DeploymentAdvancedContextInput | null = {},
): DeploymentAdvancedContext {
  const safeInput = input || {};
  const associatedAccount = (safeInput.associatedAccount || "").trim();
  const expectedReturnHorizon = (
    safeInput.expectedReturnHorizon || ""
  ).trim();
  const tags = normalizeTags(safeInput.tags);

  if (
    expectedReturnHorizon &&
    !isExpectedReturnHorizon(expectedReturnHorizon)
  )
```

Repository Audit

```
) {
  throw new Error(
    "Context Gate: Unknown expected return horizon. Use short-term, medium-term, or long-term.",
  );
}

return {
  ...(associatedAccount ? { associatedAccount } : {}),
  ...(expectedReturnHorizon
    ? { expectedReturnHorizon: expectedReturnHorizon as ExpectedReturnHorizon }
    : {}),
  ...(tags.length > 0 ? { tags } : {}),
};
}

export function hasDeploymentAdvancedContext(
  context: DeploymentAdvancedContext,
) {
  return Boolean(
    context.associatedAccount ||
    context.expectedReturnHorizon ||
    (context.tags && context.tags.length > 0),
  );
}
```

File: src/lib/finance/financeRules.test.ts

```
import assert from "node:assert/strict";
import test from "node:test";

import {
  evaluateMetadataQuality,
  summarizeMetadataQuality,
  weightConfidenceForMetadataQuality,
} from "../metadataQuality";
import {
  normalizeDeploymentAdvancedContext,
} from "../deploymentContext";
import { isValidCategory } from "../taxonomy";
import { validateDeployment } from "../validateDeployment";

test("taxonomy validation accepts only canonical return categories", () => {
  assert.equal(isValidCategory("Asset"), true);
  assert.equal(isValidCategory("Leakage"), true);
  assert.equal(isValidCategory("Unclassified"), false);
  assert.equal(isValidCategory("Unknown"), false);
});

test("metadata quality scoring is deterministic for generic labels", () => {
  assert.deepEqual(evaluateMetadataQuality(" misc ").reasons, [
    "generic_label",
  ]);
  assert.equal(evaluateMetadataQuality("stuff").isLowQuality, true);
  assert.equal(evaluateMetadataQuality("Revenue engine setup").isLowQuality, false);
});

test("metadata quality summary produces stable confidence weighting", () => {
  const summary = summarizeMetadataQuality([
    { title: "misc" },
    { title: "Brokerage allocation" },
  ]);

  assert.equal(summary.lowQualityCount, 1);
});
```

Repository Audit

```
assert.equal(summary.lowQualityRatio, 0.5);
assert.equal(summary.confidenceMultiplier, 0.93);
assert.equal(weightConfidenceForMetadataQuality(0.9, summary), 0.84);
});

test("advanced context normalization preserves optional structured metadata", () => {
  assert.deepEqual(
    normalizeDeploymentAdvancedContext({
      associatedAccount: " Brokerage ",
      expectedReturnHorizon: "long-term",
      tags: "Ops, recurring, ops",
    }),
    {
      associatedAccount: "Brokerage",
      expectedReturnHorizon: "long-term",
      tags: ["ops", "recurring"],
    },
  );
});

test("advanced context rejects unknown return horizons", () => {
  assert.throws(
    () =>
      normalizeDeploymentAdvancedContext({
        expectedReturnHorizon: "eventually",
      }),
    /Unknown expected return horizon/,
  );
});

test("deployment validation remains the write gate", () => {
  assert.throws(
    () =>
      validateDeployment({
        title: "Capital",
        amount: 100,
        category: "Unclassified",
      }),
    /Strategic classification is mandatory/,
  );

  assert.deepEqual(
    validateDeployment({
      title: " Cloud infra ",
      amount: 500,
      category: "Leverage",
      advancedContext: {
        expectedReturnHorizon: "medium-term",
        tags: "ops, infra",
      },
    }).advancedContext,
    {
      expectedReturnHorizon: "medium-term",
      tags: ["ops", "infra"],
    },
  );
});
```

File: src/lib/finance/goals.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateGoal, calculateGoalProgressPercentage } from "../goals";
```

Repository Audit

```
import { generateSummary } from "../analytics/engine";
import type { FinancialGoal } from "../analytics/types";

test("goal validation enforces structural integrity", () => {
  const data = {
    goal_name: "Emergency Fund",
    goal_type: "emergency_fund",
    target_amount: 1000000,
    current_progress: 100000,
    priority: "critical",
    status: "active",
  };

  const validated = validateGoal(data);
  assert.strictEqual(validated.goal_name, "Emergency Fund");
  assert.strictEqual(validated.goal_type, "emergency_fund");
  assert.strictEqual(validated.target_amount, 1000000);
  assert.strictEqual(validated.current_progress, 100000);
});

test("goal progress calculation is deterministic", () => {
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 1000, current_progress: 500 }), 50);
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 1000, current_progress: 1000 }), 100);
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 1000, current_progress: 1500 }), 100);
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 0, current_progress: 500 }), 0);
});

test("strategic metrics are accurately aggregated", () => {
  const goals: FinancialGoal[] = [
    {
      id: "g1",
      user_id: "u1",
      goal_name: "G1",
      goal_type: "emergency_fund",
      target_amount: 1000,
      current_progress: 200,
      priority: "critical",
      status: "active",
      linked_account_ids: [],
      created_at: "",
      updated_at: "",
      target_date: null,
    },
    {
      id: "g2",
      user_id: "u1",
      goal_name: "G2",
      goal_type: "retirement",
      target_amount: 1000,
      current_progress: 800,
      priority: "medium",
      status: "active",
      linked_account_ids: [],
      created_at: "",
      updated_at: "",
      target_date: null,
    },
  ];

  const summary = generateSummary([], 0, [], [], [], goals);

  assert.strictEqual(summary.totalStrategicTargets, 2000);
  assert.strictEqual(summary.totalCurrentProgress, 1000);
  assert.strictEqual(summary.averageGoalProgress, 50); // (20% + 80%) / 2
});
```

Repository Audit

```
    assert.strictEqual(summary.criticalGoalCount, 1);
    assert.strictEqual(summary.fundingGap, 1000);
  });
```

File: src/lib/finance/goals.ts

```
export const GOAL_TYPES = [
  { value: "emergency_fund", label: "Emergency Fund" },
  { value: "retirement", label: "Retirement" },
  { value: "home_purchase", label: "Home Purchase" },
  { value: "investment_target", label: "Investment Target" },
  { value: "business_runway", label: "Business Runway" },
  { value: "education", label: "Education" },
  { value: "debt_payoff", label: "Debt Payoff" },
  { value: "wealth_preservation", label: "Wealth Preservation" },
  { value: "other", label: "Other" },
] as const;

export type GoalType = (typeof GOAL_TYPES)[number]["value"];

export const GOAL_PRIORITIES = [
  { value: "critical", label: "Critical" },
  { value: "high", label: "High" },
  { value: "medium", label: "Medium" },
  { value: "low", label: "Low" },
] as const;

export type GoalPriority = (typeof GOAL_PRIORITIES)[number]["value"];

export const GOAL_STATUSES = [
  { value: "active", label: "Active" },
  { value: "paused", label: "Paused" },
  { value: "achieved", label: "Achieved" },
  { value: "archived", label: "Archived" },
] as const;

export type GoalStatus = (typeof GOAL_STATUSES)[number]["value"];

export interface FinancialGoal {
  id: string;
  user_id: string;
  goal_name: string;
  goal_type: GoalType;
  target_amount: number;
  current_progress: number;
  target_date: string | null;
  priority: GoalPriority;
  status: GoalStatus;
  linked_account_ids: string[];
  notes?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidGoalType(type: string): type is GoalType {
  return GOAL_TYPES.some((t) => t.value === type);
}

export function isValidPriority(priority: string): priority is GoalPriority {
  return GOAL_PRIORITIES.some((p) => p.value === priority);
}
```

Repository Audit

```
export function isValidStatus(status: string): status is GoalStatus {
    return GOAL_STATUSES.some((s) => s.value === status);
}

export function validateGoal(data: {
    goal_name: string;
    goal_type: string;
    target_amount: number;
    current_progress?: number;
    priority: string;
    status: string;
}) {
    if (!data.goal_name || data.goal_name.trim().length === 0) {
        throw new Error("Goal name is required.");
    }

    if (!isValidGoalType(data.goal_type)) {
        throw new Error(`Invalid goal type: ${data.goal_type}`);
    }

    if (isNaN(data.target_amount) || data.target_amount <= 0) {
        throw new Error("Target amount must be a positive number.");
    }

    if (data.current_progress !== undefined && isNaN(data.current_progress)) {
        throw new Error("Current progress must be a number.");
    }

    if (!isValidPriority(data.priority)) {
        throw new Error(`Invalid priority: ${data.priority}`);
    }

    if (!isValidStatus(data.status)) {
        throw new Error(`Invalid status: ${data.status}`);
    }

    return {
        goal_name: data.goal_name.trim(),
        goal_type: data.goal_type as GoalType,
        target_amount: data.target_amount,
        current_progress: data.current_progress || 0,
        priority: data.priority as GoalPriority,
        status: data.status as GoalStatus,
    };
}

/**
 * Calculates deterministic goal progress.
 */
export function calculateGoalProgressPercentage(
    goal: Pick<FinancialGoal, "current_progress" | "target_amount">,
): number {
    if (goal.target_amount <= 0) return 0;
    return Math.min(
        100,
        Math.max(0, (goal.current_progress / goal.target_amount) * 100),
    );
}
```

File: src/lib/finance/income.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
```

Repository Audit

```
import { validateIncomeStream, calculateMonthlyInflow } from "../income";
import { generateSummary } from "../analytics/engine";
import type { Deployment, IncomeStream } from "../analytics/types";

test("income validation enforces structural integrity", () => {
  const data = {
    income_name: "Salary",
    income_type: "salary",
    amount: 100000,
    cadence: "monthly",
  };

  const validated = validateIncomeStream(data);
  assert.strictEqual(validated.income_name, "Salary");
  assert.strictEqual(validated.income_type, "salary");
  assert.strictEqual(validated.amount, 100000);
  assert.strictEqual(validated.cadence, "monthly");
});

test("income normalization is accurate", () => {
  assert.strictEqual(
    calculateMonthlyInflow({ amount: 1000, cadence: "weekly" }),
    1000 * (52 / 12),
  );
  assert.strictEqual(
    calculateMonthlyInflow({ amount: 120000, cadence: "annually" }),
    10000,
  );
  assert.strictEqual(
    calculateMonthlyInflow({ amount: 5000, cadence: "monthly" }),
    5000,
  );
});

test("runway logic incorporates income offset correctly", () => {
  const deployments: Deployment[] = [
    {
      id: "1",
      amount: 3000,
      created_at: new Date().toISOString(),
      title: "Rent",
      category: "Fixed",
    },
  ]; // 100/day burn over 30 days

  const liquidity = 1000;
  const incomeStreams: IncomeStream[] = [
    {
      id: "s1",
      user_id: "u1",
      income_name: "Salary",
      amount: 1500,
      cadence: "monthly",
      is_recurring: true,
      income_type: "salary",
      currency: "KSh",
      start_date: "",
      created_at: "",
      updated_at: "",
    },
  ]; // 50/day offset

  // Correct Formula: Liquidity / (Daily Burn - (Monthly Income / 30))
  // 1000 / (100 - (1500 / 30)) = 1000 / 50 = 20 days
```

Repository Audit

```
const summary = generateSummary(
  deployments,
  liquidity,
  [],
  [],
  incomeStreams,
);

assert.strictEqual(summary.dailyBurnRate, 100);
assert.strictEqual(summary.totalMonthlyIncome, 1500);
assert.strictEqual(summary.runwayDays, 20);
assert.strictEqual(summary.adjustedDailyBurn, 50);
});
```

File: src/lib/finance/income.ts

```
export const INCOME_TYPES = [
  { value: "salary", label: "Salary" },
  { value: "freelance", label: "Freelance" },
  { value: "business", label: "Business" },
  { value: "dividends", label: "Dividends" },
  { value: "rental", label: "Rental" },
  { value: "contract", label: "Contract" },
  { value: "other", label: "Other" },
] as const;

export type IncomeType = (typeof INCOME_TYPES)[number]["value"];

export const CADENCES = [
  { value: "weekly", label: "Weekly", factor: 52 / 12 },
  { value: "biweekly", label: "Biweekly", factor: 26 / 12 },
  { value: "monthly", label: "Monthly", factor: 1 },
  { value: "quarterly", label: "Quarterly", factor: 1 / 3 },
  { value: "annually", label: "Annually", factor: 1 / 12 },
  { value: "irregular", label: "Irregular", factor: 0 },
] as const;

export type Cadence = (typeof CADENCES)[number]["value"];

export interface IncomeStream {
  id: string;
  user_id: string;
  income_name: string;
  income_type: IncomeType;
  amount: number;
  cadence: Cadence;
  execution_day?: number | null;
  last_executed_at?: string | null;
  is_recurring: boolean;
  currency: string;
  source?: string | null;
  start_date: string;
  end_date?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidIncomeType(type: string): type is IncomeType {
  return INCOME_TYPES.some((t) => t.value === type);
}

export function isValidCadence(cadence: string): cadence is Cadence {
```

Repository Audit

```
    return CADENCES.some((c) => c.value === cadence);
  }

export function validateIncomeStream(data: {
  income_name: string;
  income_type: string;
  amount: number;
  cadence: string;
  execution_day?: number | null;
  is_recurring?: boolean;
}) {
  if (!data.income_name || data.income_name.trim().length === 0) {
    throw new Error("Income name is required.");
  }

  if (!isValidIncomeType(data.income_type)) {
    throw new Error(`Invalid income type: ${data.income_type}`);
  }

  if (!isValidCadence(data.cadence)) {
    throw new Error(`Invalid cadence: ${data.cadence}`);
  }

  if (isNaN(data.amount) || data.amount < 0) {
    throw new Error("Amount must be a non-negative number.");
  }

  return {
    income_name: data.income_name.trim(),
    income_type: data.income_type as IncomeType,
    amount: data.amount,
    cadence: data.cadence as Cadence,
    execution_day: data.execution_day,
    is_recurring: data.is_recurring ?? true,
  };
}

/**
 * Normalizes an income stream amount to a monthly value.
 */
export function calculateMonthlyInflow(
  stream: Pick<IncomeStream, "amount" | "cadence">,
): number {
  const cadenceConfig = CADENCES.find((c) => c.value === stream.cadence);
  if (!cadenceConfig) return 0;
  return stream.amount * cadenceConfig.factor;
}
```

File: src/lib/finance/liabilities.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateLiability } from "../liabilities";
import { generateSummary } from "../analytics/engine";
import type { Account, Liability } from "../analytics/types";

test("liability validation enforces structural integrity", () => {
  const data = {
    liability_name: "Credit Card",
    liability_type: "credit_card",
    outstanding_balance: 50000,
  };
});
```

Repository Audit

```
const validated = validateLiability(data);
assert.strictEqual(validated.liability_name, "Credit Card");
assert.strictEqual(validated.liability_type, "credit_card");
assert.strictEqual(validated.outstanding_balance, 50000);
});

test("liability validation rejects empty names", () => {
  assert.throws(() => {
    validateLiability({
      liability_name: "",
      liability_type: "credit_card",
      outstanding_balance: 1000,
    });
  }, /Liability name is required/);
});

test("liability validation rejects invalid types", () => {
  assert.throws(() => {
    validateLiability({
      liability_name: "Test",
      liability_type: "gambling_debt",
      outstanding_balance: 1000,
    });
  }, /Invalid liability type/);
});

test("net worth calculation is deterministic", () => {
  const accounts: Account[] = [
    {
      id: "1",
      user_id: "u1",
      account_name: "Checking",
      account_type: "checking",
      current_balance: 100000,
      currency: "KSh",
      created_at: "",
      updated_at: "",
      institution: null,
    },
  ];

  const liabilities: Liability[] = [
    {
      id: "l1",
      user_id: "u1",
      liability_name: "Loan",
      liability_type: "personal_loan",
      outstanding_balance: 30000,
      interest_rate: 0,
      minimum_payment: 0,
      currency: "KSh",
      created_at: "",
      updated_at: "",
      institution: null,
      due_date: null,
    },
  ];

  const summary = generateSummary([], 100000, accounts, liabilities);

  assert.strictEqual(summary.totalAssets, 100000);
  assert.strictEqual(summary.totalLiabilities, 30000);
  assert.strictEqual(summary.networth, 70000);
});
```

Repository Audit

File: src/lib/finance/liabilities.ts

```
export const LIABILITY_TYPES = [
  { value: "credit_card", label: "Credit Card" },
  { value: "mortgage", label: "Mortgage" },
  { value: "personal_loan", label: "Personal Loan" },
  { value: "student_loan", label: "Student Loan" },
  { value: "business_loan", label: "Business Loan" },
  { value: "line_of_credit", label: "Line of Credit" },
  { value: "other", label: "Other" },
] as const;

export type LiabilityType = (typeof LIABILITY_TYPES)[number]["value"];

export interface Liability {
  id: string;
  user_id: string;
  liability_name: string;
  liability_type: LiabilityType;
  outstanding_balance: number;
  interest_rate: number;
  minimum_payment: number;
  due_date: string | null;
  currency: string;
  institution?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidLiabilityType(type: string): type is LiabilityType {
  return LIABILITY_TYPES.some((t) => t.value === type);
}

export function validateLiability(data: {
  liability_name: string;
  liability_type: string;
  outstanding_balance: number;
  interest_rate?: number;
  minimum_payment?: number;
}) {
  if (!data.liability_name || data.liability_name.trim().length === 0) {
    throw new Error("Liability name is required.");
  }

  if (!isValidLiabilityType(data.liability_type)) {
    throw new Error(`Invalid liability type: ${data.liability_type}`);
  }

  if (isNaN(data.outstanding_balance)) {
    throw new Error("Outstanding balance must be a number.");
  }

  return {
    liability_name: data.liability_name.trim(),
    liability_type: data.liability_type as LiabilityType,
    outstanding_balance: data.outstanding_balance,
    interest_rate: data.interest_rate || 0,
    minimum_payment: data.minimum_payment || 0,
  };
}
```

Repository Audit

File: src/lib/finance/metadataQuality.ts

```
export type MetadataQualityLevel = "specific" | "low";

export interface MetadataQualityEvaluation {
  level: MetadataQualityLevel;
  score: number;
  isLowQuality: boolean;
  normalizedLabel: string;
  reasons: string[];
}

export interface MetadataQualitySummary {
  averageScore: number;
  lowQualityCount: number;
  lowQualityRatio: number;
  confidenceMultiplier: number;
}

type LabelBearingInput = {
  title?: string | null;
};

const LOW_QUALITY_EXACT_LABELS = new Set([
  "misc",
  "stuff",
  "other",
  "random",
  "generic",
  "unclear",
  "unknown",
  "untitled",
  "no title",
  "none",
  "nil",
  "na",
  "n/a",
  "tbd",
  "test",
]);

const EMPTY_LIKE_PATTERNS = [/^-$/, /^\.+$/, /^?+$/, /^_+$/];

const roundQualityScore = (value: number) => Math.round(value * 100) / 100;

export function normalizeMetadataLabel(label: string) {
  return label.trim().toLowerCase().replace(/\s+/g, " ");
}

export function evaluateMetadataQuality(
  label: string | null | undefined,
): MetadataQualityEvaluation {
  const normalizedLabel = normalizeMetadataLabel(label || "");
  const semanticLabel = normalizedLabel
    .replace(/^[a-z0-9?._-]+/g, " ")
    .replace(/\s+/g, " ")
    .trim();
  const comparableLabel = semanticLabel.replace(
    /^[^a-z0-9]+|^[a-z0-9]+$/g,
    "",
  );

  const reasons: string[] = [];
```

Repository Audit

```
if (
  !semanticLabel ||
  EMPTY_LIKE_PATTERNS.some((pattern) => pattern.test(semanticLabel))
) {
  reasons.push("empty_like_label");
}

if (
  LOW_QUALITY_EXACT_LABELS.has(semanticLabel) ||
  LOW_QUALITY_EXACT_LABELS.has(comparableLabel)
) {
  reasons.push("generic_label");
}

const isLowQuality = reasons.length > 0;

return {
  level: isLowQuality ? "low" : "specific",
  score: isLowQuality ? 0.35 : 1,
  isLowQuality,
  normalizedLabel,
  reasons,
};
}

export function summarizeMetadataQuality(
  deployments: LabelBearingInput[],
): MetadataQualitySummary {
  if (deployments.length === 0) {
    return {
      averageScore: 1,
      lowQualityCount: 0,
      lowQualityRatio: 0,
      confidenceMultiplier: 1,
    };
  }

  const evaluations = deployments.map((deployment) =>
    evaluateMetadataQuality(deployment.title),
  );
  const lowQualityCount = evaluations.filter(
    (evaluation) => evaluation.isLowQuality,
  ).length;
  const averageScore =
    evaluations.reduce((sum, evaluation) => sum + evaluation.score, 0) /
    evaluations.length;
  const lowQualityRatio = lowQualityCount / evaluations.length;

  return {
    averageScore: roundQualityScore(averageScore),
    lowQualityCount,
    lowQualityRatio: roundQualityScore(lowQualityRatio),
    confidenceMultiplier: roundQualityScore(
      Math.max(0.85, 1 - lowQualityRatio * 0.15),
    ),
  };
}

export function weightConfidenceForMetadataQuality(
  confidence: number,
  metadataQuality: MetadataQualitySummary,
) {
  return roundQualityScore(
```

Repository Audit

```
    Math.min(1, Math.max(0, confidence * metadataQuality.confidenceMultiplier)),
  );
}
```

File: src/lib/finance/objectives.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateObjective } from "../objectives";

test("strategic objective validation enforces structural integrity", () => {
  const data = {
    objective_name: "Safety Net",
    objective_type: "emergency_reserve",
    target_amount: 500000,
    current_amount: 50000,
    priority_level: "critical",
    status: "active",
  };

  const validated = validateObjective(data);
  assert.strictEqual(validated.objective_name, "Safety Net");
  assert.strictEqual(validated.objective_type, "emergency_reserve");
  assert.strictEqual(validated.target_amount, 500000);
  assert.strictEqual(validated.current_amount, 50000);
  assert.strictEqual(validated.priority_level, "critical");
  assert.strictEqual(validated.status, "active");
});

test("objective validation fails on invalid types", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "invalid_type",
      target_amount: 1000,
      priority_level: "high",
      status: "active",
    });
  }, /Invalid objective type/);
});

test("objective validation fails on invalid priority", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      priority_level: "very_high",
      status: "active",
    });
  }, /Invalid priority level/);
});

test("objective validation fails on invalid status", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      priority_level: "high",
      status: "unknown",
    });
  }, /Invalid status/);
});
```

Repository Audit

```
});

test("objective validation fails on zero or negative target amount", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 0,
      priority_level: "high",
      status: "active",
    });
  }, /Target amount must be a positive number/);

  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: -1,
      priority_level: "high",
      status: "active",
    });
  }, /Target amount must be a positive number/);
});

test("objective validation fails on overfunded state", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      current_amount: 1001,
      priority_level: "high",
      status: "active",
    });
  }, /Current amount cannot exceed target amount/);
});

test("objective validation handles date normalization", () => {
  const validated = validateObjective({
    objective_name: "Test",
    objective_type: "liquidity",
    target_amount: 1000,
    priority_level: "high",
    status: "active",
    target_date: "2026-12-31",
  });
  assert.strictEqual(validated.target_date, "2026-12-31");
});

test("objective validation fails on malformed dates", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      priority_level: "high",
      status: "active",
      target_date: "not-a-date",
    });
  }, /Invalid target date format/);
});

test("objective normalization trims whitespace and handles empty notes", () => {
  const validated = validateObjective({
```

Repository Audit

```
    objective_name: " Safety Net ",
    objective_type: "emergency_reserve",
    target_amount: 1000,
    priority_level: "critical",
    status: "active",
    notes: " Need this for safety ",
  });
assert.strictEqual(validated.objective_name, "Safety Net");
assert.strictEqual(validated.notes, "Need this for safety");

const validatedEmptyNotes = validateObjective({
  objective_name: "Test",
  objective_type: "liquidity",
  target_amount: 1000,
  priority_level: "high",
  status: "active",
  notes: " ",
});
assert.strictEqual(validatedEmptyNotes.notes, null);
});
```

File: src/lib/finance/objectives.ts

```
export const OBJECTIVE_TYPES = [
  { value: "emergency_reserve", label: "Emergency Reserve" },
  { value: "liquidity", label: "Liquidity" },
  { value: "debt_elimination", label: "Debt Elimination" },
  { value: "investment", label: "Investment" },
  { value: "retirement", label: "Retirement" },
  { value: "education", label: "Education" },
  { value: "acquisition", label: "Acquisition" },
  { value: "custom", label: "Custom" },
] as const;

export type ObjectiveType = (typeof OBJECTIVE_TYPES)[number]["value"];

export const OBJECTIVE_PRIORITIES = [
  { value: "critical", label: "Critical" },
  { value: "high", label: "High" },
  { value: "moderate", label: "Moderate" },
  { value: "low", label: "Low" },
] as const;

export type ObjectivePriority = (typeof OBJECTIVE_PRIORITIES)[number]["value"];

export const OBJECTIVE_STATUSES = [
  { value: "active", label: "Active" },
  { value: "paused", label: "Paused" },
  { value: "completed", label: "Completed" },
  { value: "abandoned", label: "Abandoned" },
] as const;

export type ObjectiveStatus = (typeof OBJECTIVE_STATUSES)[number]["value"];

export interface StrategicObjective {
  id: string;
  user_id: string;
  objective_name: string;
  objective_type: ObjectiveType;
  target_amount: number;
  current_amount: number;
  target_date: string | null;
  priority_level: ObjectivePriority;
```

Repository Audit

```
    status: ObjectiveStatus;
    notes?: string | null;
    created_at: string;
    updated_at: string;
    deleted_at?: string | null;
  }

export function isValidObjectiveType(type: string): type is ObjectiveType {
  return OBJECTIVE_TYPES.some((t) => t.value === type);
}

export function isValidPriority(
  priority: string,
): priority is ObjectivePriority {
  return OBJECTIVE_PRIORITIES.some((p) => p.value === priority);
}

export function isValidStatus(status: string): status is ObjectiveStatus {
  return OBJECTIVE_STATUSES.some((s) => s.value === status);
}

/**
 * Validates and normalizes strategic objective data.
 * This is the canonical authority for objective correctness.
 */
/**
 * Calculates deterministic objective progress percentage.
 */
export function calculateObjectiveProgressPercentage(
  objective: Pick<StrategicObjective, "current_amount" | "target_amount">,
): number {
  if (objective.target_amount <= 0) return 0;
  return Math.min(
    100,
    Math.max(0, (objective.current_amount / objective.target_amount) * 100),
  );
}

/**
 * Calculates the funding ratio (alias for progress for semantic clarity in UI).
 */
export function calculateObjectiveFundingRatio(
  objective: Pick<StrategicObjective, "current_amount" | "target_amount">,
): number {
  return calculateObjectiveProgressPercentage(objective);
}

export function validateObjective(data: {
  objective_name: string;
  objective_type: string;
  target_amount: number;
  current_amount?: number;
  priority_level: string;
  status: string;
  target_date?: string | null;
  notes?: string | null;
}) {
  // 1. Required Fields & Basic Types
  if (!data.objective_name || data.objective_name.trim().length === 0) {
    throw new Error("Objective name is required.");
  }

  // 2. Enum Validations
  if (!isValidObjectiveType(data.objective_type)) {
```

Repository Audit

```
    throw new Error(`Invalid objective type: ${data.objective_type}`);
  }

  if (!isValidPriority(data.priority_level)) {
    throw new Error(`Invalid priority level: ${data.priority_level}`);
  }

  if (!isValidStatus(data.status)) {
    throw new Error(`Invalid status: ${data.status}`);
  }

  // 3. Numeric Validations
  const targetAmount = Number(data.target_amount);
  const currentAmount = Number(data.current_amount || 0);

  if (isNaN(targetAmount) || targetAmount <= 0) {
    throw new Error("Target amount must be a positive number.");
  }

  if (isNaN(currentAmount) || currentAmount < 0) {
    throw new Error("Current amount must be a non-negative number.");
  }

  if (currentAmount > targetAmount) {
    throw new Error(
      "Current amount cannot exceed target amount (overfunding requires manual strategic adjustment).",
    );
  }

  // 4. Date Validation
  let normalizedDate: string | null = null;
  if (data.target_date) {
    const d = new Date(data.target_date);
    if (isNaN(d.getTime())) {
      throw new Error("Invalid target date format.");
    }
    normalizedDate = d.toISOString().split("T")[0]; // Store as YYYY-MM-DD
  }

  // 5. Normalization
  const normalizedNotes = data.notes?.trim() || null;
  const finalNotes = normalizedNotes === "" ? null : normalizedNotes;

  return {
    objective_name: data.objective_name.trim(),
    objective_type: data.objective_type as ObjectiveType,
    target_amount: targetAmount,
    current_amount: currentAmount,
    priority_level: data.priority_level as ObjectivePriority,
    status: data.status as ObjectiveStatus,
    target_date: normalizedDate,
    notes: finalNotes,
  };
}
```

File: src/lib/finance/taxonomy.ts

```
/**
 * AXIOM RETURN-BASED TAXONOMY
 * Centralized definitions for the financial intelligence system.
 */

export const VALID_CATEGORIES = [
```

Repository Audit

```
"Asset",
"Skill",
"Leverage",
"Experience",
"Maintenance",
"Leakage",
] as const;
export type ValidCategory = (typeof VALID_CATEGORIES)[number];

export interface TaxonomyCategory {
  value: ValidCategory;
  label: string;
  definition: string;
  interpretation: string;
}

export const TAXONOMY_CATEGORIES: TaxonomyCategory[] = [
  {
    value: "Asset",
    label: "Investments",
    definition: "Future value generation (Stocks, Crypto, Property)",
    interpretation:
      "Strategic capital foundation. These deployments are expected to produce tangible appreciation or yield over time.",
  },
  {
    value: "Skill",
    label: "Growth",
    definition: "Learning and high-income skill development",
    interpretation:
      "Human capital investment. Deployments into skills increase your intrinsic market value and future inflow capacity.",
  },
  {
    value: "Leverage",
    label: "Efficiency",
    definition: "Buying back time or multiplying output",
    interpretation:
      "Operational efficiency. Leverage deployments are designed to buy back time or multiply the results of your effort.",
  },
  {
    value: "Experience",
    label: "Lifestyle",
    definition: "Intentional quality-of-life and memories",
    interpretation:
      "Strategic utility. Non-yielding deployments that provide intentional lived-value and mental sustainability.",
  },
  {
    value: "Maintenance",
    label: "Essentials",
    definition: "Bills, rent, food, and survival core",
    interpretation:
      "Operational core. Survival-layer capital needed to maintain your current system (Rent, utilities, basic nourishment).",
  },
  {
    value: "Leakage",
    label: "Waste",
    definition: "Non-strategic drift or impulse spending",
    interpretation:
      "Systemic inefficiency. Tracked separately to identify recurring capital drift. Excess leakage compresses operational runway.",
  }
]
```

Repository Audit

```
    },
  ];

export const isValidCategory = (value: string): boolean => {
  return VALID_CATEGORIES.includes(value as ValidCategory);
};

export const formatTaxonomyCategoryList = (): string => {
  return VALID_CATEGORIES.join(", ");
};

export const getTaxonomyDefinition = (value: string) => {
  return TAXONOMY_CATEGORIES.find((c) => c.value === value)?.definition || "";
};

export const getTaxonomyInterpretation = (value: string) => {
  return (
    TAXONOMY_CATEGORIES.find((c) => c.value === value)?.interpretation || ""
  );
};
```

File: src/lib/finance/validateDeployment.ts

```
/**
 * TAXONOMY ENFORCEMENT GATE
 * Ensures every deployment written to the ledger follows the strict return-based taxonomy.
 */

import {
  formatTaxonomyCategoryList,
  isValidCategory,
} from "../taxonomy";
import {
  DeploymentAdvancedContextInput,
  normalizeDeploymentAdvancedContext,
} from "../deploymentContext";
import { evaluateMetadataQuality } from "../metadataQuality";

export { VALID_CATEGORIES, type ValidCategory } from "../taxonomy";

export interface DeploymentInput {
  title: string;
  amount: number;
  category: string;
  impactScore?: number;
  advancedContext?: DeploymentAdvancedContextInput;
}

/**
 * Pre-write Validation Layer
 * Rejects any input that violates taxonomy integrity or data truth.
 */
export function validateDeployment(input: DeploymentInput) {
  const { title, amount, category, impactScore, advancedContext } = input;

  // 1. Structural Integrity
  if (!title || !title.trim()) {
    throw new Error("Taxonomy Gate: Title is required for deployment verification");
  }

  if (amount === undefined || amount === null || isNaN(amount)) {
    throw new Error("Taxonomy Gate: Numeric amount is required");
  }
}
```

Repository Audit

```
if (amount <= 0) {
  throw new Error("Taxonomy Gate: Capital deployment must be a positive value (> 0)");
}

// 2. Taxonomy Enforcement
if (!isValidCategory(category)) {
  // Specifically reject 'Unclassified' and legacy categories
  if (category === "Unclassified") {
    throw new Error("Taxonomy Gate: Strategic classification is mandatory. Entry rejected.");
  }

  throw new Error(`Taxonomy Gate: Unknown category '${category}'. Use ${formatTaxonomyCategoryList()}.`);
}

// 3. Normalization
return {
  title: title.trim(),
  amount: Number(amount),
  category,
  advancedContext: normalizeDeploymentAdvancedContext(advancedContext),
  metadataQuality: evaluateMetadataQuality(title),
  impactScore: Math.min(Math.max(impactScore || 0, 0), 10) // Scale 0-10
};
}
```

File: src/lib/insights/generateInsights.ts

```
import { BehavioralContext } from "../context/contextTypes";
import { InsightEngineResult, InsightPriority } from "../types";
import { KairosInsight } from "../ai/kairos";
import { rules } from "../rules/registry";
import { logKairosEvaluation } from "../db/telemetry";

/**
 * Axiom Insight Engine Orchestrator
 * Evaluates behavioral context against the rule registry to generate
 * prioritized strategic insights.
 *
 * V1.2: Added asynchronous telemetry logging for performance audit.
 */
export const evaluateInsights = async (
  context: BehavioralContext,
): Promise<InsightEngineResult> => {
  // 1. Process all rules and capture telemetry
  const evaluationResults = await Promise.all(
    rules.map(async (rule) => {
      const start = Date.now();
      const isMatched = rule.condition(context);

      let insight: KairosInsight | null = null;
      if (isMatched) {
        insight = rule.generate(context);
      }

      const duration = Date.now() - start;

      // Persist forensic telemetry (Non-blocking but awaited for database integrity)
      await logKairosEvaluation({
        ruleId: rule.id,
        severity: insight?.severity || "none",
        durationMs: duration,
      });
    })
  );
}
```

Repository Audit

```
    if (isMatched && insight) {
      return {
        priority: rule.priority,
        insight,
      };
    }
    return null;
  })),
);

const generatedInsights = evaluationResults.filter(
  (result): result is { priority: InsightPriority; insight: KairosInsight } =>
    result !== null,
);

// Axiom Standard: Gracefully handle the "Quiet" state
if (generatedInsights.length === 0) {
  return {
    primaryInsight: {
      type: "info",
      severity: "observation",
      category: "strategic_alignment",
      message:
        "No material structural deterioration detected since previous evaluation.",
      supportingSignals: [],
      timestamp: new Date().toISOString(),
      runway: null,
      capitalEfficiency: context.capitalEfficiencyScore ?? 0,
      isSilent: true, // This is the "Quiet System" output
    },
    allInsights: [],
  };
}

// 2. Prioritize
// Sort by priority (High > Medium > Low) and then by confidence
const sortedInsights = [...generatedInsights].sort((a, b) => {
  const priorityScore: Record<InsightPriority, number> = {
    critical: 4,
    high: 3,
    medium: 2,
    low: 1,
  };

  if (priorityScore[a.priority] !== priorityScore[b.priority]) {
    return priorityScore[b.priority] - priorityScore[a.priority];
  }

  return 0; // Maintain order within same priority
});

return {
  primaryInsight: sortedInsights[0].insight,
  allInsights: generatedInsights.map((result) => result.insight),
};
};
```

File: src/lib/insights/rules/registry.ts

```
import { InsightRule } from "../../types";

/**
```

Repository Audit

```
* KAIROS RULE REGISTRY (V1.1)
* Behavioral Presence implementation.
* Identity: Restrained Operational Analyst.
*/

export const rules: InsightRule[] = [
  // 1. SOLVENCY CRISIS (Critical Severity)
  {
    id: "solvency_crisis",
    priority: "critical",
    condition: (ctx) => ctx.netWorth < 0,
    generate: (ctx) => ({
      type: "warning",
      severity: "critical",
      category: "solvency_pressure",
      timestamp: new Date().toISOString(),
      message:
        "Severe solvency crisis detected. Total credit facilities (e.g., Fuliza, M-Shwari, SACCO Loans) exceed combined capital assets, resulting in a negative net worth state.",
      supportingSignals: [
        `Net Worth: KSh ${Math.round(ctx.netWorth).toLocaleString()}`,
        `Liquid Reserves (MMF/SACCO): KSh ${Math.round(ctx.liquidity).toLocaleString()}`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    })),
  },

  // 2. STRUCTURAL DEFICIT (Critical Severity)
  {
    id: "structural_deficit",
    priority: "high",
    condition: (ctx) => {
      const monthlyBurn = ctx.burnRate.monthlyProjection;
      const monthlyReplenishment = ctx.totalMonthlyIncome;

      // Trigger if structural burn exceeds income (Structural Deficit)
      return monthlyBurn > monthlyReplenishment && monthlyBurn > 0;
    },
    generate: (ctx) => {
      const deficit = ctx.burnRate.monthlyProjection - ctx.totalMonthlyIncome;
      return {
        type: "warning",
        severity: "critical",
        category: "solvency_pressure",
        timestamp: new Date().toISOString(),
        message:
          "Structural deficit detected. Baseline monthly outflow (including M-Pesa leakage) currently outpaces aggregate inbound yield (M-Pesa flows / Hustle & Salary).",
        supportingSignals: [
          `Monthly Deficit: KSh ${Math.round(deficit).toLocaleString()}`,
          `Inbound Yield Coverage: ${Math.round((ctx.totalMonthlyIncome / ctx.burnRate.monthlyProjection) * 100)}% of monthly burn.`,
          `Survival Window: ${ctx.runway.currentDays ? Math.round(ctx.runway.currentDays) : "Unknown"} days remaining.`,
        ],
        runway: ctx.runway.currentDays,
        capitalEfficiency: ctx.capitalEfficiencyScore,
        isSilent: false,
      };
    },
  },

  // 3. SOLVENCY PRESSURE (Critical/Warning Severity)
```

Repository Audit

```
{
  id: "solvency_pressure",
  priority: "high",
  condition: (ctx) =>
    !ctx.strategicAlignment.liquiditySufficiency.isSufficient,
  generate: (ctx) => {
    const { shortfall, message } =
      ctx.strategicAlignment.liquiditySufficiency;

    const isCritical =
      ctx.liquidity === 0 ? shortfall > 0 : shortfall / ctx.liquidity > 0.5;

    return {
      type: "warning",
      severity: isCritical ? "critical" : "warning",
      category: "solvency_pressure",
      timestamp: new Date().toISOString(),
      message:
        "Liquid reserves (MMF, SACCO Deposits) are insufficient to satisfy all critical strategic obligations.",
      supportingSignals: [
        message,
        `Combined Available Liquid Reserves: KSh ${Math.round(ctx.liquidity).toLocaleString()}`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    };
  },
},

// 4. OBJECTIVE STARVATION (Advisory Severity)
{
  id: "objective_starvation",
  priority: "medium",
  condition: (ctx) =>
    ctx.strategicAlignment.objectiveStarvationSignals.length > 0,
  generate: (ctx) => ({
    type: "info",
    severity: "advisory",
    category: "objective_starvation",
    timestamp: new Date().toISOString(),
    message: `Strategic starvation detected: ${ctx.strategicAlignment.objectiveStarvationSignals[0]}`,
    supportingSignals:
      ctx.strategicAlignment.objectiveStarvationSignals.slice(1, 3),
    runway: ctx.runway.currentDays,
    capitalEfficiency: ctx.capitalEfficiencyScore,
    isSilent: false,
  })),
},

// 5. STRATEGIC CONFLICT (Warning Severity)
{
  id: "strategic_conflict",
  priority: "medium",
  condition: (ctx) =>
    ctx.strategicAlignment.strategicAllocationSignals.length > 0,
  generate: (ctx) => ({
    type: "warning",
    severity: "warning",
    category: "strategic_alignment",
    timestamp: new Date().toISOString(),
    message: ctx.strategicAlignment.strategicAllocationSignals[0],
    supportingSignals: [
```

Repository Audit

```
`Alignment Pressure: ${ctx.strategicAlignment.alignmentPressure}/100`,
],
runway: ctx.runway.currentDays,
capitalEfficiency: ctx.capitalEfficiencyScore,
isSilent: false,
}),
},

// 6. RUNWAY DEPLETION (Critical Severity)
{
  id: "runway_critical",
  priority: "critical",
  condition: (ctx) => ctx.runway.status === "critical",
  generate: (ctx) => ({
    type: "warning",
    severity: "critical",
    category: "solvency_pressure",
    timestamp: new Date().toISOString(),
    message: `Operational runway has contracted to ${Math.round(ctx.runway.currentDays || 0)} days. Immediate
capital preservation (minimizing M-Pesa leakage) required.`,
    supportingSignals: [
      `Runway Delta: ${ctx.runway.deltaDays >= 0 ? "+" : ""}${Math.round(ctx.runway.deltaDays)} days since
last evaluation.`,
    ],
    runway: ctx.runway.currentDays,
    capitalEfficiency: ctx.capitalEfficiencyScore,
    isSilent: false,
  }),
},

// 7. LEAKAGE CRISIS (Critical Severity)
{
  id: "efficiency_crisis",
  priority: "high",
  condition: (ctx) => ctx.capitalEfficiencyScore < 40,
  generate: (ctx) => {
    const isLeakage = ctx.allocation.dominantCategory === "Leakage";
    return {
      type: "warning",
      severity: "critical",
      category: "capital_efficiency",
      timestamp: new Date().toISOString(),
      message: isLeakage
        ? "Capital efficiency crisis detected. Excessive 'Mobile Money (M-Pesa) Leakage' is compromising
long-term sustainability."
        : "Capital efficiency has reached a critical threshold. Current deployment patterns (Chama, MMF,
Shamba) are suboptimal.",
      supportingSignals: [
        `Efficiency Index: ${ctx.capitalEfficiencyScore}/100 ? Dominant: ${ctx.allocation.dominantCategory}
(${ctx.allocation.categoryDistribution[ctx.allocation.dominantCategory] * 100).toFixed(1)}%`.`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    };
  },
},

// 8. UNCLASSIFIED DATA (Advisory Severity)
{
  id: "unclassified_data",
  priority: "medium",
  condition: (ctx) =>
    (ctx.allocation.categoryDistribution["Unclassified"] || 0) > 0.1, // Only alert if > 10%
```

Repository Audit

```
generate: (ctx) => ({
  type: "info",
  severity: "advisory",
  category: "strategic_alignment",
  timestamp: new Date().toISOString(),
  message:
    "Unclassified outflows detected. Reclassify recent M-Pesa flows to restore the accuracy of the
efficiency index.",
  supportingSignals: [
    `Metadata Integrity: ${Math.round((ctx.allocation.categoryDistribution["Unclassified"] || 0) * 100)}%
of flows lack strategic classification.`
  ],
  runway: ctx.runway.currentDays,
  capitalEfficiency: ctx.capitalEfficiencyScore,
  isSilent: false,
}),
},

// 9. ERRATIC VOLATILITY (Warning Severity)
{
  id: "erratic_volatility",
  priority: "medium",
  condition: (ctx) => ctx.volatilityScore > 0.4,
  generate: (ctx) => ({
    type: "warning",
    severity: "warning",
    category: "capital_efficiency",
    timestamp: new Date().toISOString(),
    message:
      "High volatility in capital deployment detected. Irregular outbound M-Pesa flows may disrupt runway
projections.",
    supportingSignals: [
      `Irregularity Index: HIGH ? Volatility score of ${((ctx.volatilityScore * 100).toFixed(1))}% exceeds
variance threshold.`
    ],
    runway: ctx.runway.currentDays,
    capitalEfficiency: ctx.capitalEfficiencyScore,
    isSilent: false,
  }),
},

// 10. HIGH DISCIPLINE (Observation Severity)
{
  id: "high_discipline",
  priority: "medium",
  condition: (ctx) => ctx.volatilityScore < 0.2 && ctx.deploymentCount > 1,
  generate: (ctx) => {
    const isAsset = ctx.allocation.dominantCategory === "Asset";
    return {
      type: "info",
      severity: "observation",
      category: "capital_efficiency",
      timestamp: new Date().toISOString(),
      message: isAsset
        ? "High operational discipline in Capital Deployment (Chama, MMF, Shamba) detected. Consistent
deployment provides a stable foundation for scaling."
        : "High operational discipline detected. Consistent outflow patterns provide a stable foundation for
capital scaling.",
      supportingSignals: [
        `Stability Metric: ${((100 - ctx.volatilityScore * 100).toFixed(1))}% consistency across
${ctx.deploymentCount} deployments.`
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
    }
  }
}
```

Repository Audit

```
        isSilent: true,
    };
},
},

// 11. HEAVY CONCENTRATION (Advisory Severity)
{
    id: "heavy_concentration",
    priority: "low",
    condition: (ctx) => ctx.allocation.concentrationScore > 0.7,
    generate: (ctx) => ({
        type: "info",
        severity: "advisory",
        category: "strategic_alignment",
        timestamp: new Date().toISOString(),
        message: `Significant capital concentration detected in ${ctx.allocation.dominantCategory}. Ensure this
deployment (Chama, MMF, Shamba) aligns with strategic intent.`,
        supportingSignals: [
            `Concentration    Score:    ${ctx.allocation.concentrationScore.toFixed(2)}    ?
${ctx.allocation.dominantCategory}                                represents
${(ctx.allocation.categoryDistribution[ctx.allocation.dominantCategory] * 100).toFixed(1)}%    of    total
deployment.`,
        ],
        runway: ctx.runway.currentDays,
        capitalEfficiency: ctx.capitalEfficiencyScore,
        isSilent: false,
    }),
},

// 12. DEFAULT PATTERN (Legible Silence - Observation)
{
    id: "default_pattern",
    priority: "low",
    condition: () => true,
    generate: (ctx) => ({
        type: "info",
        severity: "observation",
        category: "strategic_alignment",
        timestamp: new Date().toISOString(),
        message:
            "No material behavioral shifts detected in M-Pesa flows or capital deployment. Silence is
intentional.",
        supportingSignals: [
            "Observing capital behavior: State remains within normal variance parameters.",
        ],
        runway: ctx.runway.currentDays,
        capitalEfficiency: ctx.capitalEfficiencyScore,
        isSilent: true,
    }),
},

// 13. INCOME CONCENTRATION RISK (Warning Severity)
{
    id: "income_concentration_risk",
    priority: "medium",
    condition: (ctx) => ctx.maxIncomeConcentrationRatio > 0.8,
    generate: (ctx) => ({
        type: "warning",
        severity: "warning",
        category: "capital_efficiency",
        timestamp: new Date().toISOString(),
        message:
            "High inbound concentration risk detected. Over 80% of total yield relies on a single primary flow,
creating severe structural fragility.",
    })
}
```

Repository Audit

```
    supportingSignals: [
      `Concentration Ratio: ${((ctx.maxIncomeConcentrationRatio * 100).toFixed(1))}% from primary source.`
    ],
    runway: ctx.runway.currentDays,
    capitalEfficiency: ctx.capitalEfficiencyScore,
    isSilent: false,
  })),
},
];
```

File: src/lib/insights/types.ts

```
import { BehavioralContext } from "../context/contextTypes";
import { KairosInsight } from "../ai/kairos";

export type InsightPriority = "critical" | "high" | "medium" | "low";

export interface InsightRule {
  id: string;
  priority: InsightPriority;
  condition: (context: BehavioralContext) => boolean;
  generate: (context: BehavioralContext) => KairosInsight;
}

export interface InsightEngineResult {
  primaryInsight: KairosInsight;
  allInsights: KairosInsight[];
}
```

File: src/lib/supabase.ts

```
import { createClient } from "../supabase/client";

export const supabase = createClient();
```

File: src/lib/supabase/client.ts

```
import { createBrowserClient } from '@supabase/ssr'

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}
```

File: src/lib/supabase/middleware.ts

```
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function updateSession(request: NextRequest) {
  let supabaseResponse = NextResponse.next({
    request,
  });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
```

Repository Audit

```
getAll() {
  return request.cookies.getAll();
},
setAll(cookiesToSet) {
  cookiesToSet.forEach(({ name, value }) =>
    request.cookies.set(name, value),
  );
  supabaseResponse = NextResponse.next({
    request,
  });
  cookiesToSet.forEach(({ name, value, options }) =>
    supabaseResponse.cookies.set(name, value, options),
  );
},
},
},
);

// refreshing the auth token
await supabase.auth.getUser();

return supabaseResponse;
}
```

File: src/lib/supabase/server.ts

```
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options),
            );
          } catch (error) {
            // This is expected if calling from a Server Component that is already streaming.
            // We log it only for debugging in dev mode.
            if (process.env.NODE_ENV === "development") {
              console.warn(
                "Supabase SSR: Unable to set cookies from Server Component. Ensure middleware is configured.",
                error,
              );
            }
          }
        },
      },
    },
  );
}
```

File: src/lib/utils/formatters.ts

Repository Audit

```
/**
 * Formatting utilities for the Axiom presentation layer.
 * Localized for the Kenyan market (KES / en-KE).
 */

export const formatCurrency = (amount: number): string => {
  return new Intl.NumberFormat("en-KE", {
    style: "currency",
    currency: "KES",
    maximumFractionDigits: 0,
  }).format(amount);
};

export const formatNumber = (num: number): string => {
  return new Intl.NumberFormat("en-KE").format(num);
};

export const formatDate = (dateString: string): string => {
  return new Date(dateString).toLocaleDateString("en-KE", {
    year: "numeric",
    month: "short",
    day: "numeric",
  });
};
```

File: src/lib/utills/taxonomy.ts

```
import { AccountType } from "../finance/accounts";
import { LiabilityType } from "../finance/liabilities";
import { IncomeType } from "../finance/income";
import { ValidCategory } from "../finance/taxonomy";

/**
 * TAXONOMY MAPPING DICTIONARY
 * Localizes abstract database enums to Kenyan market realities.
 */

export const AccountMap: Record<AccountType, string> = {
  checking: "M-Pesa / Bank",
  savings: "Savings / MMF",
  mobile_money: "M-Pesa",
  brokerage: "Investment / CDS",
  crypto: "Digital Assets",
  cash: "Cash on Hand",
};

export const LiabilityMap: Record<LiabilityType, string> = {
  credit_card: "Credit Card / Digital Loan",
  mortgage: "Mortgage / Shamba Loan",
  personal_loan: "Fuliza / SACCO Loan",
  student_loan: "HELB / Education Loan",
  business_loan: "Business / SME Loan",
  line_of_credit: "Overdraft facility",
  other: "Other Credit Facilities",
};

export const IncomeMap: Record<IncomeType, string> = {
  salary: "Salary / Hustle",
  freelance: "Freelance / Gig Work",
  business: "Business Yield",
  dividends: "Dividends / Chama Payout",
  rental: "Rental Income",
  contract: "Contract Fee",
};
```

Repository Audit

```
    other: "Other Inflows",
  };

export const DeploymentMap: Record<ValidCategory, string> = {
  Asset: "Investments",
  Skill: "Growth",
  Leverage: "Efficiency",
  Experience: "Lifestyle",
  Maintenance: "Essentials",
  Leakage: "Waste",
};
```

File: src/proxy.ts

```
import { type NextRequest, NextResponse } from "next/server";
import { updateSession } from "@lib/supabase/middleware";
import { createServerClient } from "@supabase/ssr";

export async function proxy(request: NextRequest) {
  // Update the session (refreshes the token)
  const response = await updateSession(request);

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll() {
          // No-op for read-only session check
        },
      },
    },
  );

  const {
    data: { user },
  } = await supabase.auth.getUser();

  // ROUTE PROTECTION

  // If user is not logged in and trying to access dashboard, redirect to home
  if (!user && request.nextUrl.pathname.startsWith("/dashboard")) {
    const url = request.nextUrl.clone();
    url.pathname = "/";
    const redirectResponse = NextResponse.redirect(url);

    // Copy all cookies from the session update response to the redirect response
    response.cookies.getAll().forEach((cookie) => {
      redirectResponse.cookies.set(cookie.name, cookie.value, cookie);
    });

    return redirectResponse;
  }

  // If user is logged in and trying to access home (login page), redirect to dashboard
  if (user && request.nextUrl.pathname === "/") {
    const url = request.nextUrl.clone();
    url.pathname = "/dashboard";
    const redirectResponse = NextResponse.redirect(url);
```

Repository Audit

```
// Copy all cookies from the session update response to the redirect response
response.cookies.getAll().forEach((cookie) => {
  redirectResponse.cookies.set(cookie.name, cookie.value, cookie);
});

return redirectResponse;
}

return response;
}

export const config = {
  matcher: [
    /*
     * Match all request paths except for the ones starting with:
     * - _next/static (static files)
     * - _next/image (image optimization files)
     * - favicon.ico (favicon file)
     * Feel free to modify this pattern to include more paths.
     */
    "/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)",
  ],
};
```

File: tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next",
      },
    ],
    "paths": {
      "@/*": [".src/*"],
    },
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts",
  ],
  "exclude": ["node_modules", "mobile"],
}
```